The MiniMPC Scheme

Design documentation
Submission version v1.0

The MINIons Team:
Hongrui Cui (Shanghai Jiao Tong University)
Chun Guo (Shandong University)
XiaoJie Guo (Shanghai Qi Zhi Institute)
David Heath (University of Illinois Urbana-Champaign)
Jonathan Katz (University of Maryland)
Vladimir Kolesnikov (Georgia Institute of Technology)
Alex Malozemoff (Galois Inc.)
Samuel Ranellucci (Coinbase)
Mike Rosulek (Oregon State University)
Lawrence Roy (Aarhus University)
Xiao Wang (Northwestern University)
Chenkai Weng (Arizona State University)
Kang Yang (State Key Laboratory of Cryptology)
Yu Yu (Shanghai Jiao Tong University, Shanghai Qi Zhi Institute)

The MiniMPC Scheme

Design documentation
Submission version v1.0

Submission version v1.0
team mailing list: minimpc@googlegroups.com

Hongrui Cui (Shanghai Jiao Tong University)
Chun Guo (Shandong University)
XiaoJie Guo (Shanghai Qi Zhi Institute)
David Heath (University of Illinois Urbana-Champaign)
Jonathan Katz (University of Maryland)
Vladimir Kolesnikov (Georgia Institute of Technology)
Alex Malozemoff (Galois Inc.)
Samuel Ranellucci (Coinbase)
Mike Rosulek (Oregon State University)
Lawrence Roy (Aarhus University)
Xiao Wang (Northwestern University)
Chenkai Weng (Arizona State University)
Kang Yang (State Key Laboratory of Cryptology)
Yu Yu (Shanghai Jiao Tong University, Shanghai Qi Zhi Institute)

Yu Yu is supported by the National Natural Science Foundation of China (Grant Nos. 92270201 and 62125204). Chun Guo is supported by the National Natural Science Foundation of China (Grant 62372274). David Heath is supported in part by NSF award CNS-2246353. Vladimir Kolesnikov is supported in part by NSF awards CNS-2246354 and CCF-2217070. Mike Rosulek is supported in part by NSF award CNS-2150726. Lawrence Roy is supported by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme under grant agreement number 101124977 (DECRYPSIS). Xiao Wang is supported in part by NSF CNS-2236819.

Submission Scope

This document details a set of protocols for securely computing any Boolean circuit assuming that an adversary corrupting all-but-one participants. It includes the main protocol based on the authenticated garbling framework (Section **??**), a symmetric-key only oblivious extension protocol based on the SoftspokenOT (Section **??**), and a suite of symmetric-key primitives to achieve correlation robust properties (Section **??**). The protocol mainly targets at Category `N3` and can be applied to `S3` if the primitives have efficient Boolean representations.

Patents disclosure

The team is not aware of any active patent, with or without the invovelement of team members, that is in conflict of the proposed protocols.

Abstract

This document describes a suite of cryptographic protocols for efficiently evaluating any Boolean circuits assuming an active adversary corrupting all-but-one parties. As such, it mainly targets N3: Symmetric category and can also be used for S3: Symmetric. The documented protocol is constant-round and has communication complexity roughly quadratic to the number of parties.

This document contains several building blocks: 1) a main protocol based on the authenticated garbling framework, 2) an improved authenticated triple generation protocol, 3) a correlated oblivious transfer extension protocol based on the SoftSpokenOT, and 4) a tweakable correlation-robust hash function that can be composed in above protocols. End-to-end, the whole protocol only requires symmetric-key primitives except for a handle of oblivious transfer correlations at the beginning of the protocol, a.k.a. base oblivious transfer. As a result, when the based oblivious transfer protocol is instantiated from quantum resilient assumptions, the whole protocol is plausibly post-quantum secure. [Xiao: performance]

Contents

Preface

The main goal of this submission is to provide a complete solution for secure multi-party computation of any Boolean circuit assuming a malicious adversary corrupting all-but-one parties. This has particular application to threashold symmetric-key primitives but also useful for various other applications including A, B, and C.

Executive summary

[Xiao: An abridged explanation of the package content, high- lighting relevant properties of the proposed threshold schemes, their applicability and performance, and other key insights. It should also describe the main challenges addressed while preparing the submission, including in the specification (e.g., in proving security), the open-source implementation, and the performance evaluation. Mathematical symbols should be used sparingly in this section, if at all.]

1 Introduction

2 Notations

[Xiao: A section that lists and explains the used acronyms (§2.1), math- ematical symbols (§2.2), and terms (§2.3). It may contain subsections to explain certain relatioships between symbols (e.g., sets, elements, operations).]

In this document, we use λ and ρ to denote the computational and statistical security parameters, respectively. For two integers a, b, we denote by $[a, b]$ the set $\{a, \ldots, b\}$ and by $[a, b)$ the set $\{a, \ldots, b-1\}$. We define $[n] = [1, n]$. We use $x \xleftarrow{\$} S$ to denote sampling x uniformly at random from a finite set S. We use bold lower-case letters like $\boldsymbol{a}$ to denote column vectors and bold upper-case letters like $\mathbf{A}$ to represent matrices. We use a_i to denote the i-th component of $\boldsymbol{a}$ (with a_0 the first entry). We also use $\boldsymbol{a}[i, j]$ (resp. $\boldsymbol{a}[i, j)$) to denote the subvector $(a_i, ..., a_j)$ (resp. $(a_i, ..., a_{j-1})$). For matrices, we use $\boldsymbol{A}^i$ to denote the i-th column and $\boldsymbol{A}_j$ to denote the j-th row.

For an extension field $\mathbb{F}_{2^\lambda}$ of a binary field $\mathbb{F}_2$, we fix some monic, irreducible polynomial $f(X)$ of degree λ and then write $\mathbb{F}_{2^\lambda} \cong \mathbb{F}_2[X]/f(X)$. Thus, every element $a \in \mathbb{F}_{2^\lambda}$ can be denoted uniquely as $a = \sum_{i\in[0,\lambda)} a_i \cdot X^i$ with $a_i \in \mathbb{F}_2$ for all $i \in [0, \lambda)$. We could view elements over $\mathbb{F}_{2^\lambda}$ equivalently as vectors in $\mathbb{F}_2^\lambda$ or strings in $\{0,1\}^\lambda$, and consider a bit $x \in \mathbb{F}_2$ as an element in $\mathbb{F}_{2^\lambda}$. Depending on the context, we use $\{0,1\}^\lambda$, $\mathbb{F}_2^\lambda$ and $\mathbb{F}_{2^\lambda}$ interchangeably, and thus addition in $\mathbb{F}_2^\lambda$ and $\mathbb{F}_{2^\lambda}$ corresponds to XOR in $\{0,1\}^\lambda$.

A Boolean circuit $\mathcal{C}$ is represented as a list of gates of the form $(\alpha, \beta, \gamma, T)$, which denotes a gate with input-wire indices α and β, output-wire index γ and gate type $T \in \{\oplus, \wedge\}$. By $|\mathcal{C}|$, we denote the number of AND gates in a circuit $\mathcal{C}$. We use $\mathcal{I}_i$ to denote the set of circuit-input wire indices with the input from a party P_i, $\mathcal{W}$ to denote the set of output wire indices for all AND gates, and $\mathcal{O}_i$ to denote the set of circuit-output wire indices associated with the output of P_i. Without loss of generality, we assume that the input and output of all parties have the same length, i.e., $|\mathcal{I}_1| = \cdots = |\mathcal{I}_n|$ and $|\mathcal{O}_1| = \cdots = |\mathcal{O}_n|$. We use $|\mathcal{I}|$ and $|\mathcal{O}|$ to denote the length of all circuit-input wires and circuit-output wires, respectively.

Let $\mathcal{F}_{\{1,...,u\}\times\mathcal{W}\times\mathcal{T}\times\{0,1\},\mathcal{W}}$ denote the set of functions from $\{1, ..., u\} \times \mathcal{W} \times \mathcal{T} \times \{0, 1\}$ to $\mathcal{W}$, and let $\mathcal{E}_{\mathcal{T},\mathcal{W}}$ denote the set of blockciphers with keyspace $\mathcal{T}$ and message space $\mathcal{W}$.

3 Related Work and Design Decisions

4 Conventional Primitives and Building Blocks

4.1 Block Cipher

4.2 Cryptographic Hash Function

4.3 Almost Universal Hash Functions

We recall the definition of almost-1 universal linear hash function (UHF) from [31] in Definition 1. Then we present two standard UHF constructions [13, 8], namely, matrix-based and polynomial-based. Finally, we recall the composition and truncation of UHF that would facilitate efficient UHF by composing the two basic designs.

4.3.1 Definitions of UHF

Definition 1. *We say that a family of linear functions $\mathcal{H} = \{\mathsf{H} : \mathbb{F}_2^m \to \mathbb{F}_2^s\}$ is ε-almost 1-universal, if it holds that for every non-zero $x \in \mathbb{F}_2^m$,*

$$\Pr_{\mathsf{H} \xleftarrow{\$} \mathcal{H}}[\mathsf{H}(x) = 0] \leq \varepsilon.$$

We call the hash family ε-uniform if for every $y \in \mathbb{F}_2^s$:

$$\Pr_{\mathsf{H} \xleftarrow{\$} \mathcal{H}}[\mathsf{H}(x) = y] \leq \varepsilon.$$

We can identify functions $\mathsf{H} \in \mathcal{H}$ with their $s \times m$ transformation matrix $\boldsymbol{H}$, and write $\mathsf{H}(x) = \boldsymbol{H} \cdot x$.

Following the definitions in [41, 2] we define the hiding property of the universal hash function as follows.

Definition 2. *An almost 1-universal hash family $\mathcal{H} = \{\mathsf{H} : \mathbb{F}_2^m \to \mathbb{F}_2^s\}$ is $\mathbb{F}_2^n$ hiding if for any $\mathsf{H} \in \mathcal{H}$ and $\boldsymbol{x} \xleftarrow{\$} \mathbb{F}_2^m$, the hash result does not leak the first n bit of the input. In particular, the concatenation of $\boldsymbol{x}[0, n)$ and $\mathsf{H}(\boldsymbol{x})$ follows uniformly random distribution over $\mathbb{F}_2^{n+s}$, or*

$$(\boldsymbol{x}[0, n), \mathsf{H}(\boldsymbol{x})) \sim U_{n+s}$$

where U_{n+s} denotes the uniform random distribution over $\mathbb{F}_2^{n+s}$.

We also note that we can pack the output of UHF from a binary vector into an element in the extension field. This would be convenient when we perform a batch-checking on authenticated bit values, since the same function would be applied to both the bit values and their IT-MAC tags and local keys (which reside in the extension field). By using the matrix form of the packed UHF, the hashed IT-MAC tags (resp. local keys) form a single extension field element rather than a vector, therefore saving communication.

Remark 1. *Let $\boldsymbol{g} := (1, X, \ldots, X^{\lambda-1})$ be the gadget vector and $\mathcal{H} := \{\mathsf{H} : \mathbb{F}_2^m \to \mathbb{F}_2^\lambda\}$ be an ε-almost 1-universal linear hash family. We can define a UHF family that outputs an extension field element as $\mathcal{H}' := \{\mathsf{H}' = \boldsymbol{g} \circ \mathsf{H} \mid \mathsf{H} \in \mathcal{H}\}$ (i.e. multiplying the hash output with the gadget vector). Moreover, we have the following facts.*

- *Since for a binary vector $\boldsymbol{x} \in \mathbb{F}_2^\lambda$ the map $\boldsymbol{x} \mapsto \boldsymbol{g}^T \cdot \boldsymbol{x}$ is a bijection, the compressed UHF preserves ε-almost property.*
- *The hiding property (if any) of $\mathcal{H}$ is also preserved in $\mathcal{H}'$.*
- *The matrix representation of $\mathsf{H} \in \mathcal{H}'$ has dimension $\boldsymbol{H} \in \mathbb{F}_{2^\lambda}^{1 \times m}$.*

4.3.2 Standard Constructions

We present two commonly-used candidate constructions of UHF. The first one is simple matrix hash family, as shown in Algorithm 1.

The second one is the polynomial based construction, as shown in Algorithm 2.

We have the following simple lemmas regarding the properties of the above two hash functions.

Lemma 1. *The matrix-based UHF $\mathcal{H} = \{\mathsf{H} : \mathbb{F}_2^m \to \mathbb{F}_2^\lambda\}$ in Algorithm 1 satisfies $2^{-\lambda}$-almost uniform property.*

Algorithm 1 Matrix-based UHF $\mathsf{H} : \mathbb{F}_2^m \to \mathbb{F}_2^\lambda$

Description: $\mathcal{H} := \mathbb{F}_2^{\lambda \times m}$, hash function $\mathsf{H} \in \mathcal{H}$ is identified by the matrix representation $\boldsymbol{H}$.
Hash Computation: $\mathsf{H}(\boldsymbol{x}) = \boldsymbol{H} \cdot \boldsymbol{x}$.

Algorithm 2 Polynomial-based UHF $\mathsf{H} : \mathbb{F}_2^m \to \mathbb{F}_2^\lambda$

Description: $\mathcal{H} := \mathbb{F}_{2^\lambda}$, hash function $\mathsf{H} \in \mathcal{H}$ is identified by an extension field element $\chi \in \mathbb{F}_{2^\lambda}$.
Hash Computation: Let $\boldsymbol{x} \in \mathbb{F}_2^m$ be the input.

1. Compute $y = \sum_{i=0}^{m-1} \chi^i \cdot x_i$.
2. Let $y = y_0 + y_1 X + ... + y_{\lambda-1} X^{\lambda-1}$, output $(y_0, ..., y_{\lambda-1}) \in \mathbb{F}_2^\lambda$.

Proof. For any $\boldsymbol{x} \in \mathbb{F}_2^m, \boldsymbol{y} \in \mathbb{F}_2^\lambda$ s.t. $\boldsymbol{x} \neq 0$, we have

$$\Pr_{\mathsf{H} \xleftarrow{\$} \mathcal{H}}[\mathsf{H}(\boldsymbol{x}) = \boldsymbol{y}] = \Pr_{\boldsymbol{H} \xleftarrow{\$} \mathbb{F}_2^{\lambda \times m}}[\boldsymbol{H} \cdot \boldsymbol{x} = \boldsymbol{y}] = \Pr_{\boldsymbol{H} \xleftarrow{\$} \mathbb{F}_2^{\lambda \times m}}[\boldsymbol{H}^i = \boldsymbol{y}'] = 2^{-\lambda},$$

where $i \in [0, m)$ is an index s.t. $x_i \neq 0$ and $\boldsymbol{y}'$ captures the rest of the terms. □

Lemma 2. *The polynomial-based UHF $\mathcal{H} = \{\mathsf{H} : \mathbb{F}_2^m \to \mathbb{F}_2^\lambda\}$ in Algorithm 2 satisfies $\frac{m}{2^\lambda}$-almost uniform property.*

Proof. For any $\boldsymbol{x} \in \mathbb{F}_2^m, \boldsymbol{y} \in \mathbb{F}_2^\lambda$ s.t. $\boldsymbol{x} \neq 0$, we have

$$\Pr_{\mathsf{H} \xleftarrow{\$} \mathcal{H}}[\mathsf{H}(\boldsymbol{x}) = \boldsymbol{y}] = \Pr_{\chi \xleftarrow{\$} \mathbb{F}_{2^\lambda}}\left[\sum_{i=0}^{m-1} x_i \cdot \chi^i = y'\right]$$

where $y' = \sum_{j=0}^{\lambda-1} y_i X^i$. We can view the above condition as the evaluation of a degree-$(m-1)$ polynomial on the point $\chi \in \mathbb{F}_{2^\lambda}$ and by the Schwartz-Zippel lemma it has at most m roots. Therefore, we conclude that

$$\Pr_{\mathsf{H} \xleftarrow{\$} \mathcal{H}}[\mathsf{H}(\boldsymbol{x}) = \boldsymbol{y}] = \frac{m}{2^\lambda}.$$

□

4.3.3 Concatenation and Truncation of UHF

We first present a method of adding the hiding property onto any existing uniform hash function family.

Lemma 3. *Let $\mathcal{H} : \{\mathsf{H} : \mathbb{F}_2^m \to \mathbb{F}_2^\lambda\}$ be a family of ε-almost uniform hash functions. Then we can construct a family of ε-almost universal hash functions with $\mathbb{F}_2^m$-hiding property as follows.*

- *The new hash family is denoted as $\mathcal{H}'$ and maps $\mathbb{F}_2^{m+\lambda}$ to $\mathbb{F}_2^\lambda$.*
- *Sampling $\mathsf{H}' \xleftarrow{\$} \mathcal{H}'$ is equivalent to $\mathsf{H} \xleftarrow{\$} \mathcal{H}$.*
- *Computation of $\mathsf{H}'(\boldsymbol{x}, \boldsymbol{r})$ for $\boldsymbol{x} \in \mathbb{F}_2^m, \boldsymbol{r} \in \mathbb{F}_2^\lambda$ returns $\mathsf{H}(\boldsymbol{x}) \oplus \boldsymbol{r}$.*

Proof. For any $(\boldsymbol{x}, \boldsymbol{r}) \in \mathbb{F}_2^{m+\lambda}$ s.t. $(\boldsymbol{x}, \boldsymbol{r}) \neq 0$, we have

$$\Pr_{\mathsf{H}' \xleftarrow{\$} \mathcal{H}'}\left[\mathsf{H}'(\boldsymbol{x}, \boldsymbol{r}) = 0\right] = \Pr_{\mathsf{H} \xleftarrow{\$} \mathcal{H}}\left[\mathsf{H}(\boldsymbol{x}) = \boldsymbol{r}\right] \leq \varepsilon$$

from the ε-almost uniform property of $\mathcal{H}$. Moreover, since $\boldsymbol{r}$ functions as one-time pad, the output perfectly hides the information of the first m-bit of the input. Therefore we have $\mathbb{F}_2^m$-hiding. □

Next we present the following composition and truncation results.

Lemma 4. *Let $\mathcal{H}, \mathcal{H}'$ be ε and ε'-almost universal hash families. Then the concatenation $\{\begin{bmatrix}\boldsymbol{H}\\ \boldsymbol{H}'\end{bmatrix} : \boldsymbol{H} \in \mathcal{H}, \boldsymbol{H}' \in \mathcal{H}'\}$ is $\varepsilon \cdot \varepsilon'$-almost universal.*

Proof. The concatenated hash function returning zero implies the two components returning zeros respectively. Since $\mathcal{H}$ and $\mathcal{H}'$ are independent, the probability of this event is the multiplication of the probability of a random function from $\mathcal{H}$ and an independent random function from $\mathcal{H}'$ returning zero respectively. □

Lemma 5. *Let $\mathcal{H} : \mathbb{F}_2^m \to \mathbb{F}_2^{r'}$ be ε-almost universal and $\mathcal{H}' : \mathbb{F}_2^{r'} \to \mathbb{F}_2^r$ be ε-almost uniform. Then the product $\{\mathsf{H}' \circ \mathsf{H} : \mathsf{H} \in \mathcal{H}, \mathsf{H}' \in \mathcal{H}'\}$ is $(\varepsilon + \varepsilon')$-almost uniform.*

Proof. Let $\boldsymbol{x} \in \mathbb{F}_2^m, \boldsymbol{x} \neq 0$. Then $\boldsymbol{x}' = \mathsf{H}(\boldsymbol{x})$ is non-zero except with probability ε. If $\boldsymbol{x}' \neq 0$ then for any $\boldsymbol{y} \in \mathbb{F}_2^r$, $\mathsf{H}(\boldsymbol{x}') = \boldsymbol{y}$ holds except with probability ε'. Therefore, by union bound we conclude that the product is an $(\varepsilon + \varepsilon')$-almost uniform family. □

Lemma 6. *Let $\delta \in \mathbb{N}, \delta < \lambda$ and $\mathcal{H} = \{\mathsf{H} : \mathbb{F}_2^m \to \mathbb{F}_2^\lambda\}$ be an ε-almost uniform family. Then the truncated hash family $\{\mathsf{trunc}_{\lambda-\delta} \circ \mathsf{H} : \mathsf{H} \in \mathcal{H}\}$ is $\varepsilon \cdot 2^\delta$-almost uniform. Here $\mathsf{trunc}_{\lambda-\delta}$ returns $\boldsymbol{x}[0, \lambda - \delta)$ on input $\boldsymbol{x} \in \mathbb{F}_2^\lambda$.*

Proof. For any $\boldsymbol{y} \in \mathbb{F}_2^{\lambda-\delta}$ there exists 2^δ pre-images of pre-truncated hash output. Therefore, by union-bound and the ε-almost uniform property of $\mathcal{H}$, we conclude that the truncated hash family is $\varepsilon \cdot 2^\delta$-almost uniform. □

5 Overview

In this section, we provide an overview of what we propose in this submission, as well as how our submission is connected to prior works.

5.1 (Tweakable) correlation robust hashing

Garbling [45, 5] and oblivious-transfer (OT) extension [33] are two important building blocks of secure computation protocols. A huge proposition of the proposed schemes were built upon the so-called *correlation robust hash functions*. This notion was first proposed by Ishai et al. [33] for the purpose of OT extension. Roughly, a hash function H is correlation robust, if the function $f_R(x) := H(x \oplus R)$ keyed by R is pseudorandom. This notion was soon adopted by garbling with "free-XOR" technique [35]. However, Choi et al. [19] pointed out that a form of "circularity" is needed in order to support security proofs for the "free-XOR" garbling. They proposed *circular correlation robustness*, which requires $f_R(x, b) := H(x \oplus R) \oplus b \cdot R$ to be pseudorandom (provided that the input x is never repeated).

Bellare et al. [4] proposed to use fixed-key AES in circuit garbling, which results in substantially reduced CPU time costs. This has motivated many subsequent works to use fixed-key AES in secure computations. Concretely, many of them built their protocols over some variants of correlation robust hashing (e.g., the half-gate garbling scheme [46] used a variant termed *circular correlation robustness for naturally derived keys*), and then instantiated the hashing using fixed-key AES. To have a solid foundation, Guo et al. [29] provided a systematic study of the correlation robustness notions. They provided detailed security proofs for the correlation robustness of the folklore construction $\mathsf{MMO}^{\pi}(x) = \pi(x) \oplus x$ and circular correlation robustness of $\widehat{\mathsf{MMO}}^{\pi}_{\sigma}(x) = \pi(\sigma(x)) \oplus \sigma(x)$ using a linear orthomorphism σ.[1] They proposed a further enhanced notion named *tweakable circular correlation robustness* (TCCR), which is necessary for the malicious security of some protocols. Roughly, $H : \{0,1\}^n \times \{0,1\}^t \to \{0,1\}^n$ is TCCR, if $f_R(w, b, i) := H(w \oplus R, i) \oplus b \cdot R$ is pseudorandom. Guo et al. [29] also gave a provably secure construction $\mathsf{TMMO}^{\pi}(x, i) = \pi(\pi(x) \oplus i) \oplus \pi(x)$ using two fixed-key AES calls. Subsequently, Chen and Tessaro [18] proposed two new TCCR hash designs from permutations, including a one-call construction using a field multiplication and a two-call construction with better security against a limited class of distinguishers.

To address security issues due to en mass deployment, Guo et al. [27] formalized *multi-user TCCR security (*muTCCR*)*, which requires the key whitened function $\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}(\mathsf{idx}, w, b, i) := H(w \oplus R_{\mathsf{idx}}, i) \oplus b \cdot R_{\mathsf{idx}}$ using a vector of u keys $\mathbf{R} = (R_1, ..., R_u)$ to be pseudorandom. Guo et al. [27] further gave attacks against fixed-key AES-based constructions and a construction $\widehat{\mathsf{MMO}}^{E}(x, i) := E(i, \sigma(x)) \oplus \sigma(x)$ using a "full-fledged" blockcipher $E : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ and a linear orthomorphism σ. Despite the added key schedule cost, $\widehat{\mathsf{MMO}}^{E}$ enjoys nearly optimal muTCCR bounds (when correctly used), and is thus adopted by some [42, 17].

When used in OT extension schemes in the $\mathcal{F}_{\Delta\text{-}\mathsf{ROT}}$-hybrid model, TCCR hash functions may suffer from key leakages. In detail, in the malicious setting, the global key query of the underlying $\mathcal{F}_{\Delta\text{-}\mathsf{ROT}}$ functionality allows the adversary to choose a predicate P and acquire $P(R)$. To capture this, Roy [41] incorporated a "key leaking" oracle into his security definition of TCR. Roy proved this notion for the random oracle and the aforementioned $\widehat{\mathsf{MMO}}^{E}$ construction, and the proven bounds are comparable with Guo et al. [27, Theorems 6 and 2].

A number of Universally Composability (UC) [12] secure MPC protocols, including distributed point functions, correlated-OT and authenticated bits/shares/triples, are built upon a hash function $H(x)$ and the aforementioned key whitened function $\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}(\mathsf{idx}, w, b, i) := H(w \oplus R_{\mathsf{idx}}, i) \oplus b \cdot R_{\mathsf{idx}}$, and a crucial step of their security proofs is to replace H with a simulator $\mathcal{S}$ and $\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}$ with a random function f, in the presence of the aforementioned key leaking oracle $\mathsf{L}_{\mathbf{R}}$. When H is a random oracle, it is fairly easy to design such a simulator. However, when the hash function is a blockcipher-based construction H^E with sophisticated structural properties, both the design and the soundness of the simulator become much more non-trivial. Note that an obvious solution is to set H^E as an indifferentiable hash function [21], but such constructions usually deliver worse security and efficiency.

Motivated by the above, our notion UC-mTCCRL is formalized as the indistinguishability of a real world and a simulated ideal world experiment. Both experiments proceed in two phases.

- In phase 1, the adversary is given access to four oracles $(O_1, O_2, O_3, \mathsf{mL}_{\mathbf{R}})$. In the real world experiment, the first three oracles are an ideal cipher E, its inverse E^{-1} and the key whitened function $\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}(\mathsf{idx}, w, b, i) := H(w \oplus R_{\mathsf{idx}}, i) \oplus b \cdot R_{\mathsf{idx}}$ using a vector of u keys $\mathbf{R} =$

[1] σ is *linear* if $\sigma(x \oplus y) = \sigma(x) \oplus \sigma(y)$ for all $x, y \in \{0,1\}^k$; σ is an *orthomorphism* [15] if it is a permutation, and the function σ' given by $\sigma'(x) := \sigma(x) \oplus x$ is also a permutation. It has been known [15, 27] that σ can be efficiently instantiated as $\sigma(x_L \| x_R) = x_R \oplus x_L \| x_L$ where x_L and x_R are the left and right halves of the input.

$(R_1, ..., R_u)$, and the key leaking oracle $\mathsf{mL}_{\mathbf{R}}$ is also defined on *this* vector $\mathbf{R}$. In the ideal world experiment, the first three oracles are blockcipher forward and backward interfaces emulated by a simulator and a random function f, and $\mathsf{mL}_{\mathbf{R}}$ is defined on a randomly picked key vector $\mathbf{R} = (R_1, ..., R_u)$.

- At the end of phase 1, the key vector $\mathbf{R}$ (used by $\mathsf{mTCK}_{\mathbf{R}}^{\mathsf{H}^E}$ and $\mathsf{mL}_{\mathbf{R}}$ in the real world, while by $\mathsf{mL}_{\mathbf{R}}$ in the ideal world) is given to the adversary $\mathcal{D}$. Meanwhile, the construction queries and answers collected by $\mathcal{D}$ in phase 1 are given to the simulator for subsequent execution.
- In phase 2, the adversary $\mathcal{D}$ is allowed to query the ideal cipher E (in the real world) or the simulator (in the ideal world). But $\mathcal{D}$ loses access to the construction oracle. (Meanwhile, $\mathsf{mL}_{\mathbf{R}}$ becomes meaningless.) $\mathcal{D}$ is finally asked to decide which world it is in.

Since $\mathcal{D}$ has "known" the keys $\mathbf{R} = (R_1, ..., R_u)$ in phase 2, the simulator has to give answers that are consistent with $\mathbf{R}$ as well. Without any information about $\mathbf{R}$, this is clearly infeasible. Fortunately, in our intended applications there could be some entities providing a *multi-key guessing oracle* $\mathsf{mKGuess}_{\mathbf{R}}$ to the simulator, and the oracle $\mathsf{mKGuess}_{\mathbf{R}}$ accepts inputs (idx, R') and replies if $R' = R_{\mathsf{idx}}$ or not. For example, in our application to authenticated triple generation, such an oracle could be provided by the ideal functionality $\mathcal{F}_{\mathsf{LaAND}}$. By querying such an oracle, the simulator can confirm its key guesses. We thereby add this oracle to our formalism.

5.2 Correlated Oblivious Transfer

5.3 Authenticated Shares and Triples

5.4 Authenticated Garbling

6 Background

6.1 Security Model

We use the standard ideal/real paradigm [11, 24] to prove security of our protocols in the presence of a malicious, static adversary, who can corrupt up to $n-1$ out of n parties. We use $\mathcal{M} \subset [n]$ to denote the set of all corrupted parties.[2]

Ideal versus real model. This stand-alone security model [24] adopts the Ideal/Real simulation paradigm, where an execution in the ideal world is compared to an execution in the real world. The goal of a protocol is defined by an ideal functionality $\mathcal{F}$, which can be considered as a trusted third party that receives the parties' inputs, performs the desired task, and sends the outputs to the parties. In the ideal execution, an ideal adversary (called the simulator) $\mathcal{S}$ interacts with functionality $\mathcal{F}$. In the real execution, adversary $\mathcal{A}$ can on behalf of corrupted parties execute the protocol with honest parties by following any arbitrary strategy. To prove the security of a protocol Π, we need to show that the real execution with protocol Π is as secure as the ideal execution with functionality $\mathcal{F}$. This means that for any adversary $\mathcal{A}$ performing an attack in the real world, there exists a simulator $\mathcal{S}$ who performs the same attack in the ideal world, such that the outputs of the honest parties and $\mathcal{A}$ in the real world are indistinguishable from the outputs of the honest parties and $\mathcal{S}$ in the ideal world.

We denote by $\mathsf{REAL}_{\Pi,\mathcal{A}(z),\mathcal{M}}(1^\lambda, \boldsymbol{x})$ the output of the honest parties and adversary $\mathcal{A}$ in a real world execution of protocol Π with the parties' inputs $\boldsymbol{x} = (x_1, \ldots, x_n)$ and auxiliary input z for

[2]Since the symbol $\mathcal{H}$ is used to denote the set of uniform hash functions, we use $[n] \setminus \mathcal{M}$ to denote the set of honest parties.

Functionality $\mathcal{F}_{\mathsf{BC}}$

This functionality runs with parties $P_1, \ldots, P_n$.

Broadcast: Upon receiving $(\mathsf{BC}, \mathtt{sid}, \mathtt{ssid}, i, m)$ from P_i, where $m \in \{0,1\}^*$, send $(\mathsf{Get}, \mathtt{sid}, \mathtt{ssid}, i, m)$ to the adversary, output $(\mathsf{Msg}, \mathtt{sid}, \mathtt{ssid}, i, m)$ to P_j upon receiving $(\mathsf{OK}, \mathtt{sid}, \mathtt{ssid}, i, j)$ from the adversary for each $j \neq i$, and ignore all subsequent $(\mathsf{BC}, \mathtt{sid}, \mathtt{ssid}, i, \ldots)$ calls with the same $(\mathtt{sid}, \mathtt{ssid}, i)$.

Figure 1: The functionality of broadcast.

$\mathcal{A}$. By $\mathsf{IDEAL}_{\mathcal{F},\mathcal{S}(z),\mathcal{M}}(1^\lambda, \boldsymbol{x})$, we denote the output of the honest parties and simulator $\mathcal{S}$ in the ideal world execution with functionality $\mathcal{F}$, the inputs $\boldsymbol{x} = (x_1, \ldots, x_n)$ to the parties and auxiliary input z to $\mathcal{S}$. In the real (resp., ideal) execution, a corrupted party P_i controlled by adversary $\mathcal{A}$ (resp., simulator $\mathcal{S}$) may either send its specified input x_i, some other x_i' or an abort message.

We prove the security of our protocol in the hybrid model, where the parties execute a protocol with real messages and also have access to a functionality such as $\mathcal{G}$. In particular, we denote the hybrid execution of a protocol Π, which is given access to a functionality $\mathcal{G}$, by $\mathsf{HYB}^{\mathcal{G}}_{\Pi,\mathcal{A}(z),\mathcal{M}}(1^\lambda, \boldsymbol{x})$. This is defined similarly to the real world execution, and is known as the $\mathcal{G}$-hybrid model.

Definition 3. *We say that a protocol Π securely computes a functionality $\mathcal{F}$ in the $\mathcal{G}$-hybrid model if for every non-uniform probabilistic polynomial time (PPT) adversary $\mathcal{A}$, there exists a non-uniform PPT simulator $\mathcal{S}$ such that*

$$\left\{\mathsf{HYB}^{\mathcal{G}}_{\Pi,\mathcal{A}(z),\mathcal{M}}(1^\lambda, \boldsymbol{x})\right\}_{\lambda,z,\mathcal{M},\boldsymbol{x}} \overset{c}{\approx} \left\{\mathsf{IDEAL}_{\mathcal{F},\mathcal{S}(z),\mathcal{M}}(1^\lambda, \boldsymbol{x})\right\}_{\lambda,z,\mathcal{M},\boldsymbol{x}},$$

where $\boldsymbol{x} = (x_1, \ldots, x_n)$ and $|x_1| = \cdots = |x_n|$.

The modular sequential composition theorem in [11] provides security guarantees when protocols are sequentially composed together. This means that if a protocol Υ securely realizes the functionality $\mathcal{G}$, then the protocol Π in the $\mathcal{G}$-hybrid model can be replaced by the sequential composition $\Pi \circ \Upsilon$. Informally, the sequential composition theorem guarantees that $\mathsf{REAL}_{\Pi\circ\Upsilon,\mathcal{A}(z),\mathcal{M}}(1^\lambda, \boldsymbol{x})$ is indistinguishable from $\mathsf{HYB}^{\mathcal{G}}_{\Pi,\mathcal{A}(z),\mathcal{M}}(1^\lambda, \boldsymbol{x})$.

In [36, Theorem 5], it has been shown that any protocol, that is proven secure in the stand-alone model with a black-box straight-line simulator and start synchronization (i.e., the inputs of all parties are fixed before the protocol execution starts), is also secure under the universal composability (UC) framework [12]. All our protocols satisfy these conditions, and thus are also UC-secure. This allows us to run the protocols in parallel and concurrently.

6.2 Communication Model

We assume that all parties are connected via authenticated channels $\mathcal{F}_{\mathsf{AT}}$ (Figure **??**), as well as point-to-point channels $\mathcal{F}_{\mathsf{PT}}$ (Figure **??**) and a broadcast channel $\mathcal{F}_{\mathsf{BC}}$ (Figure 1). Notice that we allow abort in the communication channels since in the respective functionalities the adversary can choose to not send the Deliver message but rather the abort symbol. This is allowed since in this document we consider security with abort.

The default method of communication in our protocols is authenticated channel, unless otherwise specified. Therefore, we do not explicitly mention that the protocols are defined in the $\mathcal{F}_{\mathsf{AT}}$-hybrid model. Also to facilitate less jargons, when the $\mathcal{F}_{\mathsf{PT}}$ is required in the protocol, we say that the message is sent over a private channel.

Protocol Π_{BC}

Send: On input $(\mathsf{Send}, i, \mathsf{mid}, m)$ from P_i where $m \in \{0,1\}^*$, the parties perform the following steps.

1. P_i sends m to all parties over authenticated channel
2. For $j \in [n]$, P_j receives the message m_j from the authenticated channel. Then P_j sends m_j to $\mathcal{F}_{\mathsf{RO}}$ to get h_j and sends h_j to all parties.
3. For $j \in [n]$, P_j checks that the received $n-1$ hash values equal to the previously computed h_j. If not then it aborts. Otherwise it outputs m_j.

Figure 2: The 2-round echo-broadcast protocol in the $\mathcal{F}_{\mathsf{RO}}$-hybrid model

In practice, these channels can be implemented using standard techniques. In particular, a broadcast channel can be efficiently implemented using a standard 2-round echo-broadcast protocol [26], as we allow abort. When each party needs to broadcast m messages of ℓ-bit length, the communication overhead of this broadcast protocol can be improved from $nm\ell$ bits to $m\ell + \rho$ bits using the optimized technique [23] and almost universal linear hash functions [14]. An alternative approach is to replace the almost universal linear hash function with a random oracle $\mathcal{F}_{\mathsf{RO}}$, where the communication overhead is $m\ell + \lambda$. We present the 2-round echo-broadcast protocol in the $\mathcal{F}_{\mathsf{RO}}$-hybrid model in Figure 2.

We also recall a simple lemma bounding the probability of random oracle collision below.

Lemma 7. *Let $t \in \mathbb{N}$ be an upper bound on the number of random oracle queries by $\mathcal{A}$. Then we have (f is the internal random function in $\mathcal{F}_{\mathsf{RO}}$.)*

$$\Pr\left[(y_0, y_1) \xleftarrow{\$} \mathcal{A}(1^\lambda) : y_0 \neq y_1 \wedge f(y_0) = f(y_1)\right] \leq \frac{t^2}{2^\lambda}.$$

Proof. If y_0 or y_1 does not appear on the transcript of $\mathcal{A}$'s query to $\mathcal{F}_{\mathsf{RO}}$ then we have $\Pr[f(y_0) = f(y_1)] \leq \frac{t}{2^\lambda}$, satisfying the bound.

Otherwise, the probability can be upper-bounded by the probability of collision in the hash transcript of $\mathcal{A}$, which can be upper-bounded by $\frac{1}{2^\lambda} + \frac{2}{2^\lambda} + ... + \frac{t-1}{2^\lambda} \leq \frac{t^2}{2^\lambda}$ from union bound. □

Now we prove that the protocol Π_{BC} securely implements $\mathcal{F}_{\mathsf{BC}}$.

Lemma 8. *The protocol Π_{BC} securely implements $\mathcal{F}_{\mathsf{BC}}$ in the $\mathcal{F}_{\mathsf{RO}}$-hybrid model with statistical security error $\frac{t^2}{2^\lambda}$ where t upper-bounds the number of queries to $\mathcal{F}_{\mathsf{RO}}$ by $\mathcal{A}$.*

Proof. The simulator for Π_{BC} simulates $\mathcal{F}_{\mathsf{RO}}$ in the sender-honest case and aborts if receiving inconsistent messages in the sender-corrupted case. From Lemma 7 we conclude that the statistical difference between the real world and the ideal world is $\frac{t^2}{2^\lambda}$. □

6.3 Authenticated Bits and Sharings

[Xiaojie: Extended to vectorized form?]

Authenticated bits. Authenticated bits (or equivalently information-theoretic MACs) were firstly proposed for maliciously secure two-party computation by Nielsen et al. [39], and can also be extended to the multi-party setting [7, 37]. Every party P_i holds a uniform global key $\Delta_i \in \{0,1\}^\lambda$. We say that a party P_i holds a bit $x \in \{0,1\}$ authenticated by P_j, if P_j holds a random local key $\mathsf{K}_j[x] \in \{0,1\}^\lambda$ and P_i holds the MAC $\mathsf{M}_j[x] = \mathsf{K}_j[x] \oplus x\Delta_j$. We write $[\![x]\!]_i^j = (x, \mathsf{M}_j[x], \mathsf{K}_j[x])$

to represent a two-party authenticated bit where x is known to P_i and authenticated to only one party P_j. In the multi-party setting, we let $[\![x]\!]_i = (x, \{\mathsf{M}_j[x]\}_{j\neq i}, \{\mathsf{K}_j[x]\}_{j\neq i})$ denote a multi-party authenticated bit, where the bit x is known by P_i and authenticated to all other parties. In more detail, P_i holds $(x, \{\mathsf{M}_j[x]\}_{j\neq i})$, and P_j holds $\mathsf{K}_j[x]$ for $j \neq i$.

We note that $[\![x]\!]_i$ is XOR-homomorphic. That is, for two authenticated bits $[\![x]\!]_i$ and $[\![y]\!]_i$ held by P_i, it is possible to locally compute an authenticated bit $[\![z]\!]_i$ with $z = x \oplus y$ by each party locally XOR their respective values. That is, P_i computes $z := x \oplus y$ and $\{\mathsf{M}_j[z] := \mathsf{M}_j[x] \oplus \mathsf{M}_j[y]\}_{j\neq i}$; P_j computes $\mathsf{K}_j[z] := \mathsf{K}_j[x] \oplus \mathsf{K}_j[y]$ for each $j \neq i$. We use $[\![z]\!]_i := [\![x]\!]_i \oplus [\![y]\!]_i$ to denote the above operation. As such, $[\![x]\!]_i^j$ is also XOR-homomorphic. With a slight abuse of the notation, we can also authenticate a constant bit b: P_i sets $\{\mathsf{M}_j[b] := 0\}_{j\neq i}$; P_j sets $\mathsf{K}_j[b] := b\Delta_j$ for each $j \neq i$. Similarly, let $[\![b]\!]_i = (b, \{\mathsf{M}_j[b]\}_{j\neq i}, \{\mathsf{K}_j[b]\}_{j\neq i})$. Now we can write $[\![x]\!]_i \oplus b = [\![x]\!]_i \oplus [\![b]\!]_i$, and let $b[\![x]\!]_i$ be equal to $[\![0]\!]_i$ if $b = 0$ and $[\![x]\!]_i$ otherwise.

Authenticated sharings. In most cases, authenticating a bit known to one party is not sufficient. We would like a way to authenticate a bit unknown to all parties, which can be done by secret sharing together with authenticating each share. In detail, to generate an authenticated secret bit x, we can generate XOR shares of x (i.e., shares $\{x^i\}_{i=1}^n$ such that $\bigoplus_{i=1}^n x^i = x$), and then ask every party to authenticate to every other parties about their shares. We use $\langle\!\langle x \rangle\!\rangle = \left([\![x^1]\!]_1, \ldots [\![x^n]\!]_n\right)$ to denote an authenticated share of bit x, i.e., $\langle\!\langle x \rangle\!\rangle$ means that every party P_i holds $\left(x^i, \{\mathsf{M}_j[x^i], \mathsf{K}_i[x^j]\}_{j\neq i}\right)$. It is straightforward to see that authenticated shares are also XOR-homomorphic. For a constant bit $b \in \{0, 1\}$, we let $\langle\!\langle x \rangle\!\rangle \oplus b = \left([\![x^1]\!]_1 \oplus b, [\![x^2]\!]_2, \ldots, [\![x^n]\!]_n\right)$, and define $b\langle\!\langle x \rangle\!\rangle$ to be equal to $\langle\!\langle 0 \rangle\!\rangle = ([\![0]\!]_1, \ldots, [\![0]\!]_n)$ if $b = 0$ and $\langle\!\langle x \rangle\!\rangle$ otherwise.

We define a useful operation called $\mathsf{Bit2Share}$, which takes as input an authenticated bit $[\![\lambda]\!]_i = \left(\lambda, \{\mathsf{M}_k[\lambda]\}_{k\neq i}, \{\mathsf{K}_k[\lambda]\}_{k\neq i}\right)$ with bit λ known by P_i, and extends it to an authenticated share $\langle\!\langle \lambda \rangle\!\rangle$ as follows:

- Set $[\![\lambda^i]\!]_i := [\![\lambda]\!]_i$: P_i sets $\lambda^i := \lambda$ and $\left\{\mathsf{M}_k[\lambda^i] := \mathsf{M}_k[\lambda]\right\}_{k\neq i}$, P_k sets $\mathsf{K}_k[\lambda^i] := \mathsf{K}_k[\lambda]$ for each $k \neq i$;
- Set $[\![\lambda^j]\!]_j := [\![0]\!]_j$ for each $j \neq i$: P_j sets $\lambda^j := 0$ and $\{\mathsf{M}_k[\lambda^j] := 0\}_{k\neq j}$, and P_k defines $\mathsf{K}_k[\lambda^j] := 0$ for each $k \neq j$.

In our protocol, the circuit-input wires are processed using the above procedure instead of a full-fledged authenticated shares. This is partially effective when the input is large (See Section 10). A similar idea is used in the semi-honest MPC protocol [6], where the MACs need not to be considered.

6.4 Basic Functionalities

We use several basic functionalities in our protocols, namely the random coin sampling functionality $\mathcal{F}_{\mathsf{Rand}}$ (Figure 3), the commitment functionality $\mathcal{F}_{\mathsf{Com}}$ (Figure 4), and the random oracle functionality $\mathcal{F}_{\mathsf{RO}}$ (Figure **??**).

We note that when $\mathcal{F}_{\mathsf{Rand}}$ is called in a bigger protocol, we can apply the Fiat-Shamir heuristic to replace the invocation of this functionality. Namely, all parties can hash the public protocol transcript (i.e., the public input and all public messages up until the invocation of $\mathcal{F}_{\mathsf{Rand}}$) to generate public randomness. This would save two rounds of communication at the expense of more computation.

6.5 Tools for Security Proofs

We will rely on a balls-in-bin lemma from [30].

Functionality $\mathcal{F}_{\mathsf{Rand}}$

This functionality runs with parties $P_1, \ldots, P_n$.

Rand: Upon receiving $(\mathsf{Rand}, \mathtt{sid}, \mathtt{ssid}, \mathcal{S})$ from all parties, where $\mathcal{S}$ is a finite set along with a PPT sampling algorithm, sample $r \xleftarrow{\$} \mathcal{S}$, output $(\mathsf{Res}, \mathtt{sid}, \mathtt{ssid}, r)$ to all parties, and ignore all subsequent $(\mathsf{Rand}, \mathtt{sid}, \mathtt{ssid}, \ldots)$ calls with the same $(\mathtt{sid}, \mathtt{ssid})$.

Figure 3: The functionality of coin-tossing.

Functionality $\mathcal{F}_{\mathsf{Com}}$

This functionality runs with parties $P_1, \ldots, P_n$.

Commit: Upon receiving $(\mathsf{Commit}, \mathtt{sid}, \mathtt{ssid}, i, x)$ from P_i, where $x \in \{0,1\}^*$, store $(\mathtt{sid}, \mathtt{ssid}, i, x)$, output $(\mathsf{Committed}, \mathtt{sid}, \mathtt{ssid}, i)$ to all parties, and ignore all subsequent $(\mathsf{Commit}, \mathtt{sid}, \mathtt{ssid}, i, \ldots)$ calls with the same $(\mathtt{sid}, \mathtt{ssid}, i)$.

Open: Given stored $(\mathtt{sid}, \mathtt{ssid}, i, x)$, this functionality outputs the committed value: Upon receiving $(\mathsf{Open}, \mathtt{sid}, \mathtt{ssid}, i)$ from P_i, output $(\mathsf{Opened}, \mathtt{sid}, \mathtt{ssid}, i, x)$ to all parties.

Figure 4: The functionality of commitment.

Functionality $\mathcal{F}_{\mathsf{RO}}$

This functionality runs with parties $P_1, \ldots, P_n$.

Query: Upon receiving $(\mathsf{In}, \mathtt{sid}, \mathtt{ssid}, x)$ from P_i, ignore $\mathtt{ssid}$ and proceed as follows:

- If $(\mathtt{sid}, x, y)$ was stored, output $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, y)$ to P_i.
- Otherwise, sample $y \xleftarrow{\$} \{0,1\}^\lambda$, store $(\mathtt{sid}, x, y)$, and output $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, y)$ to P_i.

Figure 5: The functionality of random oracle.

Functionality $\mathcal{F}_{\mathsf{Zero}}$

This functionality runs with parties $P_1, \ldots, P_n$. Let $\mathcal{M} \subset [n]$ be the subscript set of corrupted parties.

Zero sharing: Upon receiving $(\mathsf{Zero}, \mathtt{sid}, \mathtt{ssid}, \ell)$ from all parties, where $\ell \in \mathbb{N}^+$,

1. Receive $(\mathsf{ZShr}, \mathtt{sid}, \mathtt{ssid}, \{\boldsymbol{z}^i\}_{i \in \mathcal{M}})$ from the adversary, where $\boldsymbol{z}^i \in (\{0,1\}^\lambda)^\ell$ for each $i \in \mathcal{M}$, and sample $\boldsymbol{z}^i \xleftarrow{\$} (\{0,1\}^\lambda)^\ell$ for each $i \notin \mathcal{M}$ such that $\bigoplus_{i \in [n]} \boldsymbol{z}_i = \boldsymbol{0}$.
2. For each $i \in [n]$, output $(\mathsf{Val}, \mathtt{sid}, \mathtt{ssid}, \boldsymbol{z}^i)$ to P_i.
3. Ignore all subsequent $(\mathsf{Zero}, \mathtt{sid}, \mathtt{ssid}, \ldots)$ calls with the same $(\mathtt{sid}, \mathtt{ssid})$.

Figure 6: The functionality of zero sharing.

Lemma 9. *Fix integers n, q and $u \leq q$, a bijective function $\gamma : [2^n - 1] \mapsto \{0,1\}^n$ and a sequence of positive integers $(q_1, \ldots, q_u)$ with $\sum_{i=1}^{u} q_i = q$. Consider the following experiment involving a set of 2^n bins and q balls: for each $i \in [u]$, q_i balls are placed in the bins of indices $\gamma(1) \oplus IV_i, \gamma(2) \oplus IV_i, ..., \gamma(q_i) \oplus IV_i$, where $IV_i \xleftarrow{\$} \{0,1\}^n$ is uniformly picked. If μ^* is the random variable denoting*

Protocol Π_{Zero}

Zero sharing: All parties generate their random shares of zero vector as follows:

1. For each $i, j \in [n]$ and $j \neq i$, P_i samples $\boldsymbol{z}^{i,j} \xleftarrow{\$} (\{0,1\}^\lambda)^\ell$ and sends $\boldsymbol{z}^{i,j}$ to P_j.
2. For each $i \in [n]$, P_i computes $\boldsymbol{z}^i := \bigoplus_{j \neq i}(\boldsymbol{z}^{i,j} \oplus \boldsymbol{z}^{j,i})$ and outputs $(\mathsf{Val}, \mathtt{sid}, \mathtt{ssid}, \boldsymbol{z}^i)$.

the maximum number of balls in any bin, then

$$\Pr[\mu^* > \mu] \leq \frac{q^{\mu+1}}{(\mu+1)! \cdot 2^{\mu n}}.$$

Proof. The proof essentially follows [27, Lemma 1]. Concretely, consider the ith sequence of balls. As per our assumption, these q_i balls are thrown into the bins of indices $\gamma(1) \oplus IV_i, \gamma(2) \oplus IV_i, ..., \gamma(q_i) \oplus IV_i$ with $IV_i \xleftarrow{\$} \{0,1\}^n$. Since γ is bijective, the q_i indices are pairwise distinct. Therefore, for a certain bin, the probability that it gets a ball after the ith experiment is $q_i/2^n$.

Now, consider some μ sequences of balls, i.e., the i_1th, ..., i_μth, and consider the event that there is a $a \in \{0,1\}^n$ such that every one of those sequences hits the ath bin. By the above, the probability is

$$2^n \times \frac{q_{i_1}}{2^n} \times \cdots \times \frac{q_{i_\mu}}{2^n} = \frac{q_{i_1} \times \cdots \times q_{i_\mu}}{2^{n \cdot (\mu-1)}}.$$

Since μ^* is the maximum number of balls in any of the 2^n bins, we have

$$\Pr[\mu^* \geq \mu] \quad \leq \sum_{0 < i_1 < i_2 < \cdots < i_\mu \leq u} \frac{q_{i_1} \times \cdots \times q_{i_\mu}}{2^{n \cdot (\mu-1)}}$$

Observing that

$$(q_1 + q_2 + \cdots + q_u)^\mu \quad \geq \sum_{i_1 \neq i_2 \neq \cdots \neq i_\mu} q_{i_1} \times \cdots \times q_{i_\mu} = \mu! \cdot \sum_{i_1 < i_2 < \cdots < i_\mu} q_{i_1} \times \cdots \times q_{i_\mu},$$

we have

$$\sum_{i_1 < i_2 < \cdots < i_\mu} q_{i_1} \times \cdots \times q_{i_\mu} \leq \frac{(q_1 + q_2 + \cdots + q_u)^\mu}{\mu!}.$$

Therefore,

$$\Pr[\mu^* > \mu] = \Pr[\mu^* \geq \mu + 1] \quad \leq \frac{1}{2^{n \cdot \mu}} \times \frac{(q_1 + \cdots + q_u)^{\mu+1}}{(\mu+1)!} = \frac{q^{\mu+1}}{(\mu+1)! \cdot 2^{n \cdot \mu}}.$$

This complete the proof. □

7 Correlation-Robust Hash Function

7.1 New Notions of TCCR Hashing

We first introduce some additional notations in subsection 7.1.1. We then give the definitions of muTCCRL and UC-mTCCRL in subsections 7.1.2 and 7.1.3 respectively.

7.1.1 Key whiten oracles and transcripts

Given a hash function $\mathsf{H}^E : \mathcal{W} \times \mathcal{T} \to \mathcal{W}$ (defined upon E) and a vector of u secret keys $\mathbf{R} = (R_1, ..., R_u)$ of u users, the *multi-user tweakable circular key-whitened oracle* is defined as

$$\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}(\mathsf{idx}, w, i, b) := \mathsf{H}^E(w \oplus R_{\mathsf{idx}}, i) \oplus b \cdot R_{\mathsf{idx}}, \tag{1}$$

Given a vector of u secret keys $\mathbf{R} = (R_1, ..., R_u)$ of u users, the *multi-user key leaking oracle* $\mathsf{mL}_{\mathbf{R}}(\mathsf{idx}, P)$ takes a user index $\mathsf{idx} \in [u]$ and a predicate $P \in \mathcal{P}$ (for a set $\mathcal{P}$ that is the parameter of the notion) as input and *aborts the session of the idx-th user if and only if* $P(R_{\mathsf{idx}}) \neq 1$. This means if the adversary keeps querying $\mathsf{mL}_{\mathbf{R}}(\mathsf{idx}, P)$ with $P(R_{\mathsf{idx}}) = 1$ then nothing happens (by this, it gains information about R_{idx} by knowing $P(R_{\mathsf{idx}}) = 1$); once it queries $\mathsf{mL}_{\mathbf{R}}(\mathsf{idx}, P)$ with $P(R_{\mathsf{idx}}) = 0$ then all the keyed oracles of the idx-th user never reply queries any more—namely, $\mathsf{mL}_{\mathbf{R}}$ never replies queries of the form $(\mathsf{idx}, \star)$ and $\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}$ never replies queries of the form $(\mathsf{idx}, \star, \star, \star)$.

Our proof will need *transcripts of queries to the oracles* $\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}$ *and* E. Concretely, for any Turing machine TM that has access to the oracle $\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}$ and an ideal cipher E, we will denote by $\mathcal{Q}_E = \{(k_1, x_1, y_1), \ldots\}$ and $\mathcal{Q}_F = \{(\mathsf{idx}_1, w_1, i_1, b_1, z_1), \ldots\}$ the transcripts of queries of TM and the answers. A triple (k, x, y) in the list $\mathcal{Q}_E$ indicates either TM queried $E(k, x)$ and obtained y, or TM queried $E^{-1}(k, y)$ and obtained x. A tuple $(\mathsf{idx}, w, i, b, z)$ indicates TM queried $\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}(\mathsf{idx}, w, i, b)$ and obtained z as the response. Depending on the context, transcripts may be time-dependent.

7.1.2 muTCCRL: Multi-user TCCR with Key Leakages

We first augment the notion muTCCR of Guo et al. [27] with a key leaking oracle, and formalize the obtained security definition as *multi-user TCCR with key leakages* (muTCCRL).

Definition 4 (muTCCRL advantage)**.** *Given a function $H^E : \mathcal{W} \times \mathcal{T} \to \mathcal{W}$, a tuple of key sets $\overrightarrow{\mathcal{R}} = \mathcal{R}_1 \times ... \times \mathcal{R}_u \subseteq \mathcal{W}^u$, a set $\mathcal{P}$ of "allowed" predicates, and a distinguisher $\mathcal{D}$, define*

$$\mathbf{Adv}^{\mathrm{muTCCRL}}_{H,\mathcal{R},\mathcal{P},u}(\mathcal{D}) := \Bigg| \Pr_{E \xleftarrow{\$} \mathcal{E}_{\mathcal{T},\mathcal{W}}, \mathbf{R} \xleftarrow{\$} (\mathcal{R})^u} \left[\mathcal{D}^{E, E^{-1}, \mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}, \mathsf{mL}_{\mathbf{R}}} = 1\right]$$

$$- \Pr_{f \xleftarrow{\$} \mathcal{F}_{\{1,\ldots,u\}\times\mathcal{W}\times\mathcal{T}\times\{0,1\},\mathcal{W}}, E \xleftarrow{\$} \mathcal{E}_{\mathcal{T},\mathcal{W}}, \mathbf{R} \xleftarrow{\$} (\mathcal{R})^u} \left[\mathcal{D}^{E, E^{-1}, f, \mathsf{mL}_{\mathbf{R}}} = 1\right] \Bigg|,$$

where both probabilities are also over choice of E and we require that

1. *$\mathcal{D}$ never queries both $(\mathsf{idx}, w, i, 0)$ and $(\mathsf{idx}, w, i, 1)$ to its second oracle (for any idx, w, i).*
2. *Every query (idx, P) of $\mathcal{D}$ to its third oracle $\mathsf{mL}_{\mathbf{R}}$ has $P \in \mathcal{P}$.*

In the ideal cipher model, we call a muTCCRL adversary/distinguisher $\mathcal{D}$ $(q_C, q_E, M, q_{L,max}, \mu)$-*bounded*, if:

1. $\mathcal{D}$ makes at most q_E queries to E and at most q_C queries to $\mathcal{O}^{\mathrm{muTCCRL}}_{\mathbf{R}}/f$;
2. $\mathcal{D}$ makes key leaking queries with at most M distinct user indices idx, and for any of them, $\mathcal{D}$ makes at most $q_{L,max}$ queries to $\mathsf{mL}_{\mathbf{R}}(\mathsf{idx}, \cdot)$;
3. For all $i \in \mathcal{T}$, the number of queries (across all the u users) of the form $(\star, \star, i, \star)$ to $\mathcal{O}^{\mathrm{muTCCRL}}_{\mathbf{R}}/f$ is at most μ.

As shown in [30], the five parameters give a complicated though fine-grained characterization of adversarial power in the muTCCRL setting.

The UC-mTCCRL experiment $\mathbf{Exp}^{\text{UC-mTCCRL}}_{\mathsf{H}^E,\overrightarrow{\mathcal{R}},\mathcal{P},u,\mathcal{D},\mathcal{S},\beta}$:

1. **Initialization:**
   - If $\beta = 0$: sample an ideal cipher $E \xleftarrow{\$} \mathcal{E}_{\mathcal{T},\mathcal{W}}$ and a vector of u keys $\mathbf{R} \xleftarrow{\$} \overrightarrow{\mathcal{R}}$;
   - Else, i.e., $\beta = 1$: sample a random function $\mathbf{F} \xleftarrow{\$} \mathcal{F}_{\{1,\ldots,u\}\times\mathcal{W}\times\mathcal{T}\times\{0,1\},\mathcal{W}}$ and a vector of u keys $\mathbf{R} \xleftarrow{\$} \overrightarrow{\mathcal{R}}$.
2. **Phase 1:**
   - If $\beta = 0$: run $end \leftarrow \mathcal{D}^{E,\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}},\mathsf{mL}_{\mathbf{R}}}$ with E, the multi-user tweakable circular key-whitened oracle $\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}$, and the key leaking oracle $\mathsf{mL}_{\mathbf{R}}$, till $\mathcal{D}$ outputs *end* indicating the end of phase 1;
   - Else, i.e., $\beta = 1$: run $end \leftarrow \mathcal{D}^{\mathcal{S}^{\mathsf{mKGuess}_{\mathbf{R}}}.O_1,\mathbf{F},\mathsf{mL}_{\mathbf{R}}}$ with the simulator's interface $\mathcal{S}^{\mathsf{mKGuess}_{\mathbf{R}}}.O_1$, the random function $\mathbf{F}$, and the key leaking oracle $\mathsf{mL}_{\mathbf{R}}$, till $\mathcal{D}$ outputs *end* indicating the end of phase 1;
3. **Key revealing:** Give $\mathcal{D}$ the key vector $\mathbf{R}$ used in phase 1. If $\beta = 1$ then invoke $\mathcal{S}^{\mathsf{mKGuess}_{\mathbf{R}}}.PhaseSwitch(\mathcal{Q}_F)$, where $\mathcal{Q}_F$ is the transcript of queries and answers of $\mathcal{D}$ to $\mathbf{F}$ in the first phase (transcript has been formalized in Sect. 7.1.1).
4. **Phase 2:**
   - If $\beta = 0$: run $\beta' \leftarrow \mathcal{D}^E$ with E and obtain its output β';
   - Else, i.e., $\beta = 1$: run $\beta' \leftarrow \mathcal{D}^{\mathcal{S}^{\mathsf{mKGuess}_{\mathbf{R}}}.O_1}$ with the simulator's interface and obtain its output β'.
5. **Finalization:** Output β' (the output of $\mathcal{D}$).

Figure 7: Experiments in the UC-mTCCRL setting. Note that in phase 2 $\mathcal{D}$ has got $\mathbf{R}$, and its access to $\mathsf{mL}_{\mathbf{R}}$ is thus not needed.

7.1.3 UC-TCCR: Multi-user TCCR for UC Security

Consider a function $H^E : \mathcal{W} \times \mathcal{T} \to \mathcal{W}$ defined upon an ideal blockcipher $E \in \mathcal{E}_{\mathcal{T},\mathcal{W}}$. Using the oracles $\mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}$ and $\mathsf{mL}_{\mathbf{R}}$ formalized in Sect. 7.1.1, the *UC variant of* muTCCRL *(*UC-mTCCRL*)* requires $(E, \mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}, \mathsf{mL}_{\mathbf{R}})$ to be indistinguishable from $(\mathcal{S}^{\mathsf{mKGuess}_{\mathbf{R}}}, f, \mathsf{mL}_{\mathbf{R}})$ for some simulator $\mathcal{S}$ in a 2-phase game. The *multi-key guessing oracle* $\mathsf{mKGuess}_{\mathbf{R}}$ is implemented by relevant protocol entities to help the simulator confirm its key guesses, and it accepts inputs (idx, R') and replies if $R' = R_{\mathsf{idx}}$ or not. The formal definition is as follows.

Definition 5 (UC-mTCCRL advantage)**.** *Given a function $H^E : \mathcal{W} \times \mathcal{T} \to \mathcal{W}$ defined upon an ideal cipher $E \in \mathcal{E}_{\mathcal{T},\mathcal{W}}$, a tuple of key sets $\overrightarrow{\mathcal{R}} = \mathcal{R}_1 \times \ldots \times \mathcal{R}_u \subseteq \mathcal{W}^u$, a set $\mathcal{P}$ of "allowed" predicates, a distinguisher $\mathcal{D}$ and a simulator $\mathcal{S}^{\mathsf{mKGuess}_{\mathbf{R}}}$ providing an interface O_1, define*

$$\mathbf{Adv}^{\text{UC-mTCCRL}}_{H,\overrightarrow{\mathcal{R}},\mathcal{P},u}(\mathcal{D},\mathcal{S}) := \left|\Pr\left[\mathbf{Exp}^{\text{UC-mTCCRL}}_{H,\overrightarrow{\mathcal{R}},\mathcal{P},u,\mathcal{D},\mathcal{S},0} = 1\right] - \Pr\left[\mathbf{Exp}^{\text{UC-mTCCRL}}_{H,\overrightarrow{\mathcal{R}},\mathcal{P},u,\mathcal{D},\mathcal{S},1} = 1\right]\right|.$$

Furthermore:

1. *$\mathcal{D}$ never queries both $(\mathsf{idx}, w, i, 0)$ and $(\mathsf{idx}, w, i, 1)$ to its third oracle (for any idx, w, i).*
2. *Every query (idx, P) of $\mathcal{D}$ to its third oracle $\mathsf{mL}_{\mathbf{R}}$ has $P \in \mathcal{P}$.*

Similarly to Sect. 7.1.2, we also use the notion $(q_C, q_E, M, q_{L,max}, \mu)$*-bounded distinguishers*

to quantify adversarial power. Asymptotically, if there exists a simulator $\mathcal{S}$ such that for any distinguisher $\mathcal{D}$ $\mathbf{Adv}^{\text{UC-mTCCRL}}_{H,\vec{\mathcal{R}},\mathcal{P},u}(\mathcal{D},\mathcal{S})$ is negligibly small, then H is UC-mTCCRL-secure.

7.2 muTCCRL Security of $\widehat{\mathsf{MMO}}^E$

Guo et al. [28] proposed a hash function with good mTCCR security bounds. Specifically, define $\widehat{\mathsf{MMO}}^E : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ as

$$\widehat{\mathsf{MMO}}^E(x,i) := E(i,\sigma(x)) \oplus \sigma(x), \tag{2}$$

where $E : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ is a block cipher and σ is a linear orthomorphism. As shown by Guo et al. [28], σ can be efficiently instantiated as $\sigma(x_L\|x_R) = x_R \oplus x_L\|x_L$ where x_L and x_R are the left and right halves of the input, respectively; in assembly code, this becomes $\sigma(x) = \texttt{\_mm\_shuffle\_epi32}(\texttt{x}, 78) \oplus \texttt{and\_si128}(\texttt{x}, \texttt{mask})$, where $\texttt{mask} = 1^{64}\|0^{64}$.

We will prove that $\widehat{\mathsf{MMO}}^E$ is muTCCRL secure. We only consider the case of $\mathcal{D}$ using *oracle-free* predicates for its key leaking oracle queries. Namely, for every query $\mathsf{mL}_{\mathbf{R}}(\mathsf{idx}, P)$, the predicate evaluation $P(R_{\mathsf{idx}})$ does not query the ideal cipher E. Formally, let $\mathcal{P}_{\mathsf{free}}$ be the set of all such oracle-free predicates.

Theorem 1. *Let $C := \min\{|\mathcal{R}_1|, ..., |\mathcal{R}_u|\}$. If σ is a linear orthomorphism and E is modeled as an ideal cipher, then the* muTCCRL *advantage of any $(q_C, q_E, M, q_{L,max}, \mu)$-bounded adversary $\mathcal{D}$ against $\widehat{\mathsf{MMO}}^E$ has upper bound*

$$\mathbf{Adv}^{\text{muTCCRL}}_{H,\vec{\mathcal{R}},\mathcal{P}_{\mathsf{free}},u}(\mathcal{D}) \le \frac{2\mu q_E(M+1)(q_{L,max}+1)}{|\mathcal{R}|} + \frac{(\mu-1)\cdot q_C(M+1)(q_{L,max}+1)}{|\mathcal{R}|}. \tag{3}$$

In the single-user setting, using $M = 1$ yields the bound

$$\mathbf{Adv}^{\text{muTCCRL}}_{H,\vec{\mathcal{R}},\mathcal{P}_{\mathsf{free}},1}(\mathcal{D}) \le \frac{4\mu q_E(q_L+1)}{|\mathcal{R}|} + \frac{2(\mu-1)\cdot q_C(q_L+1)}{|\mathcal{R}|}. \tag{4}$$

This bound can also be obtained by adapting our proof to the single-user setting, but a single analysis in the multi-user setting turns out to be sufficient.

Below we elaborate on the concrete bounds in Sect. 7.2.1. The proof is given in Appendix A.

7.2.1 Bounds in concrete scenarios

As demonstrated by Guo et al. [27, Theorem 3], μ can be limited to $O(n)$ or even $O(1)$ by using somewhat random tweaks in the protocol. In the optimistic case where $\mu = O(1)$, it holds:

- Eq. (4) indicates single-user security up to $q_E q_L \ll C$ and $q_C q_L \ll C$, which is much inferior to Guo et al. [27, Theorem 2].
- Eq. (3) means multi-user security up to $Mq_E q_{L,max} \ll C$ and $Mq_C q_{L,max} \ll C$. In the worst case $M = u$, and it suffers from the well-known multi-user security degradation.

Unfortunately, these are tight: we refer to the attacks in [30]. On the other hand, in concrete scenarios, $q_{L,max}$, the number of maximal key leaking queries per user, may be rather limited, which entails much better concrete security. For example, in an execution of the OT extension protocol, $q_{L,max}$ is equal to the number ι of iterations in OT extension. The number of iterations

depends on the concrete applications and memory, but we often have $\iota = O(1)$ in general. In this case, single-user security is ensured up to $C/\iota \approx C$ queries, while multi-user security is ensured up to $C/(M\iota) \approx C/M$ queries.

We further remark that although our multi-user bound Eq. (3) suffers from the multi-user loss (the factor of M), it remains non-trivial. As mentioned before, our method, once adapted, can yield the (tight) single-user bound Eq. (4). With this, the multi-user bound derived via the trivial hybrid argument is

$$\frac{4u\mu q_E(q_L+1)}{|\mathcal{R}|} + \frac{2u(\mu-1)\cdot q_C(q_L+1)}{|\mathcal{R}|} \tag{5}$$

Since $u \gg M$, this bound is inferior to Eq. (3). This improvement and the aforementioned tightness also demonstrate the usefulness of our complicated proof approach.

7.3 UC-mTCCRL Security of $\widehat{\mathsf{MMO}}^E$

We also focus on using *oracle-free* predicates for its key leaking oracle queries. Formally, we have

Theorem 2. *If $q_E \le 2^n/2$, σ is a linear orthomorphism and $E \in \mathcal{E}_{\{0,1\}^n,\{0,1\}^n}$ is modeled as an ideal cipher, then there exists a simulator $\mathcal{S}_1$ such that the* UC-mTCCRL *advantage of $\widehat{\mathsf{MMO}}^E$ for any $(q_C, q_E, M, q_{L,max}, \mu)$-bounded distinguisher $\mathcal{D}$ and any constant $T > 0$ has upper bound*

$$\mathbf{Adv}^{\text{UC-mTCCRL}}_{\widehat{\mathsf{MMO}}^E,\mathcal{R},\mathcal{P}_{\mathsf{free}},u}(\mathcal{D},\mathcal{S}_1) \le \frac{7\mu q_E(M+1)(q_{L,max}+1) + 4(\mu+1)^2 T q_C(M+1)(q_{L,max}+1)}{|\mathcal{R}|} + \frac{2}{T}, \tag{6}$$

In addition, $\mathcal{S}_1^{\mathsf{mKGuess}_{\mathbf{R}}}$ makes at most μq_E queries to its oracle $\mathsf{mKGuess}_{\mathbf{R}}$ and runs in time $O(\mu q_E + q_C)$.

Since the bound in Eq. (6) holds for any $T > 0$, there exists an optimal T. But setting $T = \sqrt{\frac{|\mathcal{R}|}{4(\mu+1)^2 q_C(M+1)(q_{L,max}+1)}}$ suffices to yield an asymptotically optimal bound:

$$\mathbf{Adv}^{\text{UC-mTCCRL}}_{\widehat{\mathsf{MMO}}^E,\mathcal{R},\mathcal{P}_{\mathsf{free}},u}(\mathcal{D},\mathcal{S}_1) \le \frac{7\mu q_E(M+1)(q_{L,max}+1)}{|\mathcal{R}|} + 6(\mu+1)\sqrt{\frac{q_C(M+1)(q_{L,max}+1)}{|\mathcal{R}|}}. \tag{7}$$

Theorem 2 is proved via a generic relation between muTCCRL and UC-mTCCRL among a class of hash constructions. We refer to Appendix B for the details.

8 Correlated Oblivious Transfer

We describe the correlated OT functionality in Figure 8.

We use the correlated oblivious transfer (COT) presented in SoftSpokenOT [41]. The following protocols are executed by two parties P_S and P_R. Define a parameter k that controls the communication–computation tradeoff of the scheme. A larger k uses less communication, but more computation.

This protocol consists of two parts: a VOLE that works over $\mathbb{F}_{2^k}$ that only is efficient when k is small (the small-field VOLE), and a larger subfield VOLE that combines n of these small-field VOLEs together to get a VOLE over $\mathbb{F}_{2^{nk}}$ (but with choice bits still defined over $\mathbb{F}_2$). If $nk \ge \kappa$, then these together define a correlated OT protocol, by converting (and possibly truncating) each row of n field elements over $\mathbb{F}_{2^k}$ into a single element of $\mathbb{F}_{2^\kappa}$.

Functionality $\mathcal{F}_{\mathsf{mCOT}}$

This functionality runs with parties $P_1, \ldots, P_n$. Let $\mathcal{M} \subset [n]$ be the subscript set of corrupted parties and [Xiaojie: $\mathcal{L} \subseteq \mathcal{P}(\mathbb{F}_{2^\lambda})$ be defined in XXX].

Parameter: A set $\{\mathcal{R}_i\}_{i\in[n]}$ of global-key distributions on $\mathbb{F}_{2^\lambda}$.

Initialization: Upon receiving $(\mathsf{Init}, \mathtt{sid}, i)$ from P_i,

1. If P_i is corrupted, receive $(\mathsf{GKs}, \mathtt{sid}, i, \{\Delta_{i,j}\}_{j\in[n]})$ from the adversary, where $\Delta_{i,j} \in \mathbb{F}_{2^\lambda}$ for each $j \in [n]$, and store $(\mathtt{sid}, i, \{\Delta_{i,j}\}_{j\in[n]})$. Otherwise, sample $\Delta_i \xleftarrow{\$} \mathcal{R}_i$ and store $(\mathtt{sid}, i, \Delta_i)$.
2. Ignore all subsequent $(\mathsf{Init}, \mathtt{sid}, i)$ calls with the same $(\mathtt{sid}, i)$.

Extension: Given stored $(\mathtt{sid}, i, \{\Delta_{i,j}\}_{j\in[n]})$ for each $i \in \mathcal{M}$ and $(\mathtt{sid}, i, \Delta_i)$ for each $i \in [n] \setminus \mathcal{M}$, this functionality outputs random COT tuples: Upon receiving $(\mathsf{Extend}, \mathtt{sid}, \mathtt{ssid}, i, j, \ell)$ from receiver P_i and sender P_j, where $\ell \in \mathbb{N}^+$,

1. If P_j is corrupted, receive $(\mathsf{S}, \mathtt{sid}, \mathtt{ssid}, i, j, \boldsymbol{x}^j, \mathsf{M}_i[\boldsymbol{x}^j])$ from the adversary, where $(\boldsymbol{x}^j, \mathsf{M}_i[\boldsymbol{x}^j]) \in (\mathbb{F}_2)^\ell \times (\mathbb{F}_{2^\lambda})^\ell$. Otherwise, sample $(\boldsymbol{x}^j, \mathsf{M}_i[\boldsymbol{x}^j]) \xleftarrow{\$} (\mathbb{F}_2)^\ell \times (\mathbb{F}_{2^\lambda})^\ell$.
2. If P_i is corrupted, receive $(\mathsf{R}, \mathtt{sid}, \mathtt{ssid}, i, j, \mathsf{K}_i[\boldsymbol{x}^j])$ from the adversary, where $\mathsf{K}_i[\boldsymbol{x}^j] \in (\mathbb{F}_{2^\lambda})^\ell$, and recompute $\mathsf{M}_i[\boldsymbol{x}^j] := \mathsf{K}_i[\boldsymbol{x}^j] \oplus \boldsymbol{x}^j \cdot \Delta_{i,j} \in (\mathbb{F}_{2^\lambda})^\ell$. Otherwise, compute $\Delta_{i,j} := \Delta_i \in \mathbb{F}_{2^\lambda}$ and $\mathsf{K}_i[\boldsymbol{x}^j] := \mathsf{M}_i[\boldsymbol{x}^j] \oplus \boldsymbol{x}^j \cdot \Delta_{i,j} \in (\mathbb{F}_{2^\lambda})^\ell$.
3. Output $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, j, \boldsymbol{x}^j, \mathsf{M}_i[\boldsymbol{x}^j])$ to P_j.
4. **Selective-failure queries:** If P_i is honest and P_j is corrupted, proceed as follows:
   (a) Receive $(\mathsf{Leak}, \mathtt{sid}, \mathtt{ssid}, i, j, L_{i,j})$ from the adversary, where $L_{i,j} \in \mathcal{L}$.
   (b) If $\Delta_i \notin L_{i,j}$, send $(\mathsf{Fail}, \mathtt{sid}, \mathtt{ssid}, i, j)$ to the adversary and abort; otherwise, do nothing.
5. Output $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, j, \Delta_{i,j}, \mathsf{K}_i[\boldsymbol{x}^j])$ to P_i.
6. Ignore all subsequent $(\mathsf{Extend}, \mathtt{sid}, \mathtt{ssid}, i, j, \ldots)$ calls with the same $(\mathtt{sid}, \mathtt{ssid}, i, j)$.

Figure 8: The functionality of multi-party correlated OT.

8.1 Small-field VOLE

[Xiao: are we proving the whole protocol together or in the small field VOEL hybrid? is latter we need an ideal func]

Figure 9 shows a small-field vector oblivious linear evaluation which returns $(\boldsymbol{u}, \boldsymbol{v}) \in \mathbb{F}_2^\ell \times \mathbb{F}_{2^k}^\ell$ to P_S and $(\Delta, \boldsymbol{w}) \in \mathbb{F}_{2^k} \times \mathbb{F}_{2^k}^\ell$ to P_R. It consists of three main steps:

1. $\binom{2^k}{2^k-1}$-oblivious transfer. It builds upon the puncturable pseudorandom functions (PPRF) [10] with $O(k\kappa)$ bits communication overhead. A length-doubling PRG G is known by both parties. P_R uniformly samples an index $\Delta \in \mathbb{F}_{2^k}$. P_S builds a depth-k GGM tree [25] and uses k $\binom{2}{1}$ OT to transmit nodes to P_R except for the nodes lie in the Δ-th path.

2. Consistency check. SoftSpokenOT employs the technique from Boyle et al. [9] to achieve malicious security for the above $\binom{2^k}{2^k-1}$-OT. The protocol extends the GGM tree for one more layer using a second PRG, $G' \colon \{0,1\}^\kappa \to \{0,1\}^{2\kappa} \times \{0,1\}^\kappa$, which is required to be collision-resistant in its left output. The left children in this extra layer are used to ensure that the $2^k - 1$ nodes received by P_R at layer k are consistent between different choices of the punctured path (i.e., choices of Δ).

Procedure π_{sfVOLE}

Define parameters k, ℓ. Define three pseudorandom generators $G\colon \{0,1\}^\kappa \to \{0,1\}^{2\kappa}$, $G'\colon \{0,1\}^\kappa \to \{0,1\}^{2\kappa} \times \{0,1\}^\kappa$, and $G''\colon \{0,1\}^\kappa \to \mathbb{F}_2^\ell$. G' must be collision-resistant in its first output. $H\colon \{0,1\}^{2^k \cdot \kappa} \to \{0,1\}^{2\kappa}$ is a collision-resistant hash function. Define a function Int which takes h bits $(b_0, \ldots, b_{h-1})$ and returns $\sum_{i=0}^{h-1} b_i \cdot 2^{h-1-i}$.

Puncturable Pseudorandom Function:

1. P_S samples $(s_0^1, s_1^1) \in \{0,1\}^{2\kappa}$ and sends $(\mathsf{send}, (s_0^1, s_1^1))$ to $\mathcal{F}_{\mathsf{OT}}$. For $j \in [2,k]$, P_S performs the following steps:
   (a) For $i \in [0, 2^{j-1})$, compute $(s_{2i}^j, s_{2i+1}^j) \xleftarrow{\$} G(s_i^{j-1})$.
   (b) For $\sigma \in \{0,1\}$, compute $K_\sigma^j = \bigoplus_{i=0}^{2^{j-1}} s_{2i+\sigma}^j$. Send $(\mathsf{send}, K_0^j, K_1^j)$ to $\mathcal{F}_{\mathsf{OT}}$.
2. P_R samples $\alpha \in \mathbb{F}_{2^k}$ and define the reverse bit-decomposition of α to be $(\alpha_1, \ldots, \alpha_k)$, where α_1 is the most significant bit of α. For $j \in [k]$, P_R performs the following steps:
   (a) Send $(\mathsf{recv}, \bar{\alpha}_j)$ to $\mathcal{F}_{\mathsf{OT}}$, which returns $K_{\bar{\alpha}_j}^j$.
   (b) Set $d = \mathsf{Int}(\alpha_1, \ldots, \alpha_{j-1})$.
   (c) If $j > 1$, for $i \in [0, 2^{j-1}) \setminus \{d\}$ compute $(s_{2i}^j, s_{2i+1}^j) \xleftarrow{\$} G(s_i^{j-1})$.
   (d) Compute $s_{2d+\bar{\alpha}_j}^j = K_{\bar{\alpha}_j}^j \oplus \left(\bigoplus_{i \in [0,2^{j-1}) \setminus \{d\}} s_{2i+\bar{\alpha}_j}^j\right)$.

Consistency Check:

3. P_S computes $(\tilde{s}_i \| y_i) \xleftarrow{\$} G'(s_i^k)$, for $i \in [0, 2^k)$. P_R computes $(\tilde{s}_i \| y_i) \xleftarrow{\$} G'(s_i^k)$ for $i \in [0, 2^k) \setminus \{\alpha\}$.
4. P_S computes $t = \bigoplus_{i=0}^{2^k-1} \tilde{s}_i$ and $s = H(\tilde{s}_0, \ldots, \tilde{s}_{2^k-1})$. P_S sends (t, s) to P_R, who computes $\tilde{s}_\alpha = t \oplus \left(\bigoplus_{i \in [0,2^k) \setminus \{\alpha\}} \tilde{s}_i\right)$ and $s' = H(\tilde{s}_0, \ldots, \tilde{s}_{2^k-1})$. If $s \neq s'$, P_R aborts.

Small-field VOLE:

5. P_S computes $\boldsymbol{r}_i \xleftarrow{\$} G''(y_i)$ for $i \in [0, 2^k)$. P_R computes $\boldsymbol{r}_i \xleftarrow{\$} G''(y_i)$ for $i \in [0, 2^k) \setminus \{\alpha\}$,
6. P_S returns $\boldsymbol{u} = \sum_{x \in \mathbb{F}_{2^k}} \boldsymbol{r}_x \in \mathbb{F}_2^\ell$ and $\boldsymbol{v} = -\sum_{x \in \mathbb{F}_{2^k}} \boldsymbol{r}_x \cdot x \in \mathbb{F}_{2^k}^\ell$.
7. P_R returns $\Delta = \alpha \in \mathbb{F}_{2^k}$ and $\boldsymbol{w} = \sum_{x \in \mathbb{F}_{2^k} \setminus \{\Delta\}} \boldsymbol{r}_x \cdot (\Delta - x) \in \mathbb{F}_{2^k}^\ell$.

Figure 9: Small-field vector oblivious linear evaluation.

3. Convert from $\binom{2^k}{2^k-1}$-OT to VOLE over $\mathbb{F}_{2^k}$. On P_S side, G' additionally outputs $\{y_i\}_{i \in [2^k]}$ while on P_R side, G' additionall outputs $\{y_i\}_{i \in [2^k] \setminus \{\Delta\}}$. Using the third PRG G'', parties expand these values to elements $\boldsymbol{r}_i$ in $\mathbb{F}_2^\ell$ and locally convert them to length ℓ subfield VOLE. This conversion produces a consistent VOLE correlation because

$$\boldsymbol{u} \cdot \Delta + \boldsymbol{v} = \sum_{x \in \mathbb{F}_{2^k}} \boldsymbol{r}_x \cdot \Delta - \sum_{x \in \mathbb{F}_{2^k}} \boldsymbol{r}_x \cdot x = \sum_{x \in \mathbb{F}_{2^k}} \boldsymbol{r}_x \cdot (\Delta - x) = \sum_{x \in \mathbb{F}_{2^k} \setminus \{\Delta\}} \boldsymbol{r}_x \cdot (\Delta - x) = \boldsymbol{w}.$$

The communication cost of this protocol consists of k chosen-message κ-bit $\binom{2}{1}$-OTs, plus an extra 4κ bits for the consistency check. If these OTs are implemented by derandomizing random OTs, the total communication is then $(2k+4)\kappa$ bits (plus the random OTs).

Protocol Π_{sVOLE}

Define parameters k, n, m, ℓ, where $nk \geq \kappa$. Let $\mathcal{R} \subset \mathbb{F}_2^{m \times \ell}$ be an ε-almost 1-universal hash family. Let $\mathsf{Hash}\colon \mathbb{F}_{2^k}^{m \times n} \to \{0,1\}^{2\kappa}$ be a collision-resistant hash family. For a vector $\boldsymbol{a}$ (resp. matrix A), define $\boldsymbol{a}_{[:\ell]}$ (resp. $A_{[:\ell]}$) to be its first ℓ elements (resp. rows). Define a function $\mathsf{Conv} : \mathbb{F}_{2^k}^n \to \mathbb{F}_{2^\kappa}$, which takes n $\mathbb{F}_{2^k}$-elements $(a_0, \ldots, a_{n-1})$ where $a_i = \sum_{j=0}^{k-1} a_{ij} X_k$, and outputs $\sum_{i=0}^{\kappa-1} a_{\lfloor \frac{i}{k} \rfloor\,(i \bmod k)} \cdot X_\kappa^i$. Here, X_N is the generator used to represent $\mathbb{F}_{2^N}$ over $\mathbb{F}_2$.

1. For $i \in [n]$ (in parallel), invoke Procedure 9 with parameters k, $\ell + m$, which returns $(\boldsymbol{u}_i, \boldsymbol{v}_i)$ to P_S and $(\Delta_i, \boldsymbol{w}'_i)$ to P_R. P_S defines $\boldsymbol{u} = \boldsymbol{u}_1$ and $V = (\boldsymbol{v}_1 \| \ldots \| \boldsymbol{v}_n) \in \mathbb{F}_{2^k}^{(\ell+m) \times n}$
2. For $i \in [2, n]$, P_S computes $\boldsymbol{d}_i = \boldsymbol{u} \oplus \boldsymbol{u}_i \in \mathbb{F}_2^{\ell+m}$ and sends $\boldsymbol{d}_i$ to P_R. P_R computes $\boldsymbol{w}_i \xleftarrow{\$} \boldsymbol{w}'_i \oplus \boldsymbol{d}_i \cdot \Delta_i$. P_R defines $\boldsymbol{\Delta}$ to be the column vector $[\Delta_1 \ \ldots \ \Delta_n]^\top \in \mathbb{F}_{2^k}^n$, and $W = (\boldsymbol{w}_1 \| \ldots \| \boldsymbol{w}_n) \in \mathbb{F}_{2^k}^{(\ell+m) \times n}$.
3. P_R uniformly samples $R \xleftarrow{\$} \mathcal{R}$ and sends it to P_S. P_S sends back $\tilde{\boldsymbol{u}} = [R \ \mathbb{1}_m]\, \boldsymbol{u} \in \mathbb{F}_{2^k}^m$ and $\tilde{V} = \mathsf{Hash}([R \ \mathbb{1}_m]\, V) \in \{0,1\}^{2\kappa}$. If $\tilde{V} \neq \mathsf{Hash}([R \ \mathbb{1}_m]\, W - \tilde{\boldsymbol{u}} \boldsymbol{\Delta}^\top)$, P_R aborts.
4. P_S updates $\boldsymbol{u} \xleftarrow{\$} \boldsymbol{u}_{[:\ell]} \in \mathbb{F}_2^\ell$ and $V \xleftarrow{\$} V_{[:\ell]} \in \mathbb{F}_{2^k}^{\ell \times n}$. It converts V to a $\mathbb{F}_{2^{nk}}$ vector $\boldsymbol{v}$ such that $w_i = \mathsf{Conv}(V[i])$ for $i \in [\ell]$. P_R updates $W \xleftarrow{\$} W_{[:\ell]} \in \mathbb{F}_{2^k}^{\ell \times n}$. It converts W to a $\mathbb{F}_{2^{nk}}$ vector $\boldsymbol{w}$ such that $v_i = \mathsf{Conv}(W[i])$ for $i \in [\ell]$, and sets $\Delta = \mathsf{Conv}(\Delta_1, \ldots, \Delta_n) \in \mathbb{F}_{2^\kappa}$.
5. P_S outputs $(\boldsymbol{u}, \boldsymbol{v})$ and P_R outputs $(\Delta, \boldsymbol{w})$.

Figure 10: Subfield Vector oblivious linear evaluation based on SoftSpokenOT [41].

8.2 Subfield VOLE

The protocol of subfield vector oblivious linear evaluation (VOLE) is presented in Figure 10.

In this protocol, the sender P_S receives $\boldsymbol{u} \in \mathbb{F}_2^h$ and $\boldsymbol{v} \in \mathbb{F}_{2^\kappa}^h$, whilst P_R receives $\boldsymbol{w} \in \mathbb{F}_{2^\kappa}^h$ and $\Delta \in \mathbb{F}_{2^\kappa}$. They satisfy $\boldsymbol{v} = \boldsymbol{w} + \boldsymbol{u} \cdot \Delta$. The protocol first invokes the procedure in Figure 9 for $n \geq \kappa/k$ times to collect $((\boldsymbol{u}_i, \boldsymbol{v}_i), (\Delta_i, \boldsymbol{w}'_i)) \in (\mathbb{F}_2^{\ell+m} \times \mathbb{F}_{2^k}^{\ell+m}) \times (\mathbb{F}_{2^k} \times \mathbb{F}_{2^k}^{\ell+m})$, for a cost of $n(2k+4)\kappa$ bits of communication and nk random $\binom{2}{1}$-OTs (assuming that it is instantiated with random OTs). Here, m is the number of extra entries used to make the consistency check preserve privacy.

Next, P_S derandomizes the $\boldsymbol{u}_i$ to all be the same $\boldsymbol{u}$, by sending the difference of $\boldsymbol{u} = \boldsymbol{u}_1$ and $\boldsymbol{u}_i$ for all $i \in [2, n]$ to P_R, who corrects the values of $\boldsymbol{w}'_i$ to $\boldsymbol{w}_i$ s.t. $\boldsymbol{v}_i = \boldsymbol{w}_i \oplus \boldsymbol{u} \cdot \Delta_i$. Sending these differences takes $(n-1)(\ell+m)$ bits of communication. For the consistency check, a universal hash family $\mathcal{R}$ (Definition 1) is employed to verify the relation $V = W + \boldsymbol{u}\boldsymbol{\Delta}^\top$. Depending on the universal hash output size m and family size $|\mathcal{R}|$, it costs $\log_2 |\mathcal{R}|$ bits for P_R to send the hash function (typically this can be κ, by compressing it with a pseudorandom generator), and $m + 2\kappa$ bits for P_S to return the response. If the check passes, the two parties discard the last m rows (i.e., correlated values), and for the remaining ℓ rows, they pack the n $\mathbb{F}_{2^k}$ elements at each row of V and W to get elements in $\mathbb{F}_{2^\kappa}$. Additionally, P_R defines Δ as the packing of $(\Delta_1, \ldots, \Delta_n)$ in $\mathbb{F}_{2^\kappa}$.

The correlated oblivious transfer (COT) can be realized by the COT sender acting as a VOLE Receiver and the COT receiver acting as a VOLE sender. The COT sender inteprets the output $\Delta \in \mathbb{F}_{2^\kappa}$ as a bit string $\Delta \in \{0,1\}^\kappa$ and $\boldsymbol{w} \in \mathbb{F}_{2^\kappa}^\ell$ as a bit matrix $\boldsymbol{w} \in \{0,1\}^{\ell \times \kappa}$. The COT receiver inteprets the output $\boldsymbol{u} \in \mathbb{F}_2$ as a bit string $\boldsymbol{u} \in \{0,1\}^\ell$ and $\boldsymbol{v} \in \mathbb{F}_{2^\kappa}^\ell$ as a bit matrix $\boldsymbol{v} \in \{0,1\}^{\ell \times \kappa}$.

[Xiao: the following should be somewhere else.]

[Xiaojie: this procedure is now inlined into protocol Π_{aShare}]

8.3 Fixing Procedure

We will use the macro $\mathsf{Fix}(x, [\![r]\!]_i^j, x)$ to denote the operation that changes the authentication of an arbitrary random bit into arbitrary given bit[3]. The operation is defined as follows.

- $[\![x]\!]_i^j \xleftarrow{\$} \mathsf{Fix}(x, [\![r]\!]_i^j)$: On input a private bit $x \in \mathbb{F}_2$ of P_i and a random bit $r \in \mathbb{F}_2$ authenticated by the global key Δ_j of P_j, two parties P_i and P_j execute the following:

  1. P_i sends $d := x \oplus r$ to P_j.
  2. Both parties set $[\![x]\!]_i^j := [\![r]\!]_i^j \oplus d$.

8.4 Amortized Opening Procedures

We present how to open authenticated bits/shares in an amortized way (i.e., it is possible to open ℓ authenticated bits with less than ℓ times the communication) using the standard techniques [39, 23]. In a naive approach, a party P_i can open $[\![x]\!]_i^j$ to P_j via just sending x and $\mathsf{M}_j[x]$ to P_j. Party P_j is able to verify the validity of x by checking that $\mathsf{M}_j[x] = \mathsf{K}_j[x] \oplus x\Delta_j$. As observed in previous work [39], authenticated bits/shares can be opened in the following amortized process.

- **aBits:** For each $i \in [n], j \neq i$, P_i can open ℓ two-party authenticated bits $[\![x_1]\!]_i^j, \ldots, [\![x_\ell]\!]_i^j$ to P_j as follows:

  1. P_i calls $\mathcal{F}_{\mathsf{RO}}$ with input $(\mathsf{M}_j[x_1], \ldots, \mathsf{M}_j[x_\ell])$ to get τ_j.
  2. P_i sends $x_1, \ldots, x_\ell$ and τ_j to P_j.
  3. P_j calls $\mathcal{F}_{\mathsf{RO}}$ with input $(\mathsf{K}_j[x_1] \oplus x_1\Delta_j, \ldots, \mathsf{K}_j[x_\ell] \oplus x_\ell\Delta_j)$ to get τ'_j.
  4. P_j checks that $\tau_j = \tau'_j$. If the check fails, P_j aborts.

  We use $\mathsf{Open}([x_k]_i^j)$ for each $k \in [\ell]$ to denote the above amortized opening process for two-party authenticated bits. P_i can also open ℓ multi-party authenticated bits $[\![x_1]\!]_i, \ldots, [\![x_\ell]\!]_i$ to all parties via opening $[\![x_1]\!]_i^j, \ldots, [\![x_\ell]\!]_i^j$ to P_j for each $j \neq i$.

- **aShares:** All parties can open ℓ authenticated shares $\langle\langle x_1 \rangle\rangle, \ldots, \langle\langle x_\ell \rangle\rangle$ by every party P_i opening its portion in the following way.

  1. For each $j \neq i$, P_i sends $(\mathsf{M}_j[x_1^i], \ldots, \mathsf{M}_j[x_\ell^i])$ to $\mathcal{F}_{\mathsf{RO}}$ to get $\tau_{i,j}$.
  2. For each $j \neq i$, P_i sends $x_1^i, \ldots, x_\ell^i$ along with $\tau_{i,j}$ to P_j.
  3. For each $j \neq i$, P_j sends $(\mathsf{K}_j[x_1^i] \oplus x_1^i\Delta_j, \ldots, \mathsf{K}_j[x_\ell^i] \oplus x_\ell^i\Delta_j)$ to $\mathcal{F}_{\mathsf{RO}}$ to get $\tau'_{i,j}$.
  4. For each $j \neq i$, P_j checks that $\tau_{i,j} = \tau'_{i,j}$ and aborts if the check fails.

  Let $\mathsf{Open}(\langle\langle x_k \rangle\rangle)$ for each $k \in [\ell]$ denote the above amortized opening process for authenticated shares.

Below, we prove that the above opening process in a batch is secure, even if the adversary can leak a few bits of global keys such that each bit leaked of global keys will be caught with probability 1/2. We focus on the case of two-party authenticated bits, where the security proof is easy to be generalized to multi-party authenticated bits and authenticated shares.

[3]The notation is inspired by the Mac'n'Cheese paper [3].

Lemma 10. *In the amortized opening process for two-party authenticated bits, either an honest party P_j aborts, or P_j receives the correct bits from a malicious party P_i except with probability $\frac{t+1}{2^\lambda}$ where t upper bounds the number of $\mathcal{F}_{\mathsf{RO}}$ queries of P_i. Let $\mathcal{A}$ be a probabilistic polynomial time (PPT) adversary, which corrupts the party P_i. Assuming that $\mathcal{A}$ leaks c bits of global key Δ_j for some $c \in [\lambda] \cup \{0\}$, and honest party P_j will abort with probability $1/2^c$.*

Proof. Let $x_1, \ldots, x_\ell$ be the correct bits that will be sent by semi-honest P_i and $x'_1, ..., x'_\ell$ be the bits that are actually sent. Define $e_i = x_i \oplus x'_i$ be the errors in the bits.

Let f be the random function that underlies the functionality $\mathcal{F}_{\mathsf{RO}}$. The receiver P_j checks that

$$\begin{aligned}\tau'_j &= f(\mathsf{K}[x_1] \oplus x'_1\Delta_j, ..., \mathsf{K}[x_\ell] \oplus x'_\ell\Delta_j)\\ &= f(\mathsf{M}[x_1] \oplus e_1\Delta_j, ..., \mathsf{M}[x_\ell] \oplus e_\ell\Delta_j)\end{aligned}$$

Assuming the input $\mathsf{M}[x_1] \oplus e_1\Delta_j, ..., \mathsf{M}[x_\ell] \oplus e_\ell\Delta_j$ has not been queried by P_i, then the probability of passing the check is $2^{-\lambda}$ since τ'_j is uniformly random over $\mathbb{F}_{2^\lambda}$. We bound the probability of P_i querying $\mathsf{M}[x_1] \oplus e_1\Delta_j, ..., \mathsf{M}[x_\ell] \oplus e_\ell\Delta_j$ (denoted as event Query) below.

Since event Query is equivalent to correctly guessing Δ_j, the probability of Query is at most $\frac{t}{2^{\lambda-c}} \cdot \frac{1}{2^c}$ where t bounds the number of queries to $\mathcal{F}_{\mathsf{RO}}$ by P_i and c is the number of leaked bits of Δ_j. Overall, we have

$$\begin{aligned}\Pr[P_j \text{ accepts}] &= \Pr[P_j \text{ accepts} \mid \mathsf{Query}] \cdot \Pr[\mathsf{Query}] + \Pr[P_j \text{ accepts} \mid \neg\mathsf{Query}] \cdot \Pr[\neg\mathsf{Query}]\\ &\leq \Pr[\mathsf{Query}] + \Pr[P_j \text{ accepts} \mid \neg\mathsf{Query}]\\ &\leq \frac{t+1}{2^\lambda}\end{aligned}$$

Overall, except with probability $\frac{t+1}{2^\lambda}$ P_j will reject incorrect bits and therefore only accepts the correct bits. □

9 Improved Preprocessing Protocols

In this section, we define the preprocessing protocols which will facilitate the online protocol. The functionalities of the preprocessing protocols are summarized as an ideal functionality $\mathcal{F}_{\mathsf{Prep}}$ in Figure 11. We follow the technical routes of previous works [34, 44] and construct the individual functionalities $\mathcal{F}_{\mathsf{aBit}}$, $\mathcal{F}_{\mathsf{aShare}}$, and $\mathcal{F}_{\mathsf{aAND}}$. As an intermediate step, we construct a leaky version of the authenticated AND triples functionality $\mathcal{F}_{\mathsf{LaAND}}$ and then removes the leakage with bucketing.

9.1 Basic Macros

In our functionalities and protocols, we will use the following macros as sub-procedures.

- $\mathsf{AuthShare}(\mathtt{sid}, \mathtt{ssid}, \boldsymbol{x}, \{\Delta_i\}_{i\in[n]})$. This macro is invoked by functionalities $\mathcal{F}_{\mathsf{aShare}}$, $\mathcal{F}_{\mathsf{LaAND}}$, and $\mathcal{F}_{\mathsf{Prep}}$. Let $\mathcal{M} \subset [n]$ be the subscript set of corrupted parties. Given a session ID $\mathtt{sid}$, a sub-session ID $\mathtt{ssid}$, a vector $\boldsymbol{x} \in (\mathbb{F}_2)^\ell$, and n global keys $\{\Delta_i\}_{i\in[n]} \in (\mathbb{F}_{2^\lambda})^n$:
  1. Receive $(\mathsf{Shr}, \mathtt{sid}, \mathtt{ssid}, \{\boldsymbol{x}^i\}_{i\in\mathcal{M}})$ from the adversary, where $\boldsymbol{x}^i \in (\mathbb{F}_2)^\ell$ for each $i \in \mathcal{M}$, and sample $\boldsymbol{x}^i \xleftarrow{\$} (\mathbb{F}_2)^\ell$ for each $i \notin \mathcal{M}$ such that $\bigoplus_{i\in[n]} \boldsymbol{x}^i = \boldsymbol{x}$.
  2. For each $i \in [n]$,
     (a) If P_i is corrupted, receive $(\mathsf{M}, \mathtt{sid}, \mathtt{ssid}, i, \{\mathsf{M}_j[\boldsymbol{x}^i]\}_{j\neq i})$ from the adversary, where $\mathsf{M}_j[\boldsymbol{x}^i] \in (\mathbb{F}_{2^\lambda})^\ell$ for each $j \neq i$, and compute $\mathsf{K}_j[\boldsymbol{x}^i] := \mathsf{M}_j[\boldsymbol{x}^i] \oplus \boldsymbol{x}^i \cdot \Delta_j \in (\mathbb{F}_{2^\lambda})^\ell$ for each $j \neq i$.

Functionality $\mathcal{F}_{\mathsf{Prep}}$

This functionality runs with parties $P_1, \ldots, P_n$. Let $\mathcal{M} \subset [n]$ be the subscript set of corrupted parties and [Xiaojie: $\mathcal{L} \subseteq \mathcal{P}(\mathbb{F}_{2^\lambda})$ be defined in XXX]. Let $\{\mathcal{R}_i\}_{i\in[n]}$ be the set of distributions on $\{0,1\}^\lambda$ such that $\mathcal{R}_i$ fixes the LSB of each sample to (n **mod** 2) for $i = 1$ and to 1 for each $i \neq 1$.

Initialization: Upon receiving $(\mathsf{Init}, \mathtt{sid}, i)$ from P_i,

1. If P_i is corrupted, receive $(\mathsf{GK}, \mathtt{sid}, i, \Delta_i)$ from the adversary, where $\Delta_i \in \{0,1\}^\lambda$. Otherwise, sample $\Delta_i \xleftarrow{\$} \mathcal{R}_i$.
2. Store $(\mathtt{sid}, i, \Delta_i)$, initialize an empty list A_i of outputs, and ignore all subsequent $(\mathsf{Init}, \mathtt{sid}, i)$ calls with the same $(\mathtt{sid}, i)$.

Authenticated shares & Authenticated AND triples: Given stored $(\mathtt{sid}, i, \Delta_i)$ for each $i \in [n]$, this functionality outputs such random tuples: Upon receiving $(\mathsf{Gen}, \mathtt{sid}, \mathtt{ssid}, \ell_1, \ell_2)$ from all parties, where $\ell_1, \ell_2 \in \mathbb{N}^+$ and $\ell_2 \geq \lambda$,

1. **Selective-failure queries:** For each $i \in [n] \setminus \mathcal{M}$ and $j \in \mathcal{M}$, proceed as follows in parallel:
   (a) Receive $(\mathsf{Leak}, \mathtt{sid}, \mathtt{ssid}, i, j, L_{i,j})$ from the adversary, where $L_{i,j} \in \mathcal{L}$.
   (b) If $\Delta_i \notin L_{i,j}$, send $(\mathsf{Fail}, \mathtt{sid}, \mathtt{ssid}, i, j)$ to the adversary and abort; otherwise, do nothing.
2. **Global-key extraction with abort:** Receive $(\mathsf{Quit}, \mathtt{sid}, \mathtt{ssid}, b)$ from the adversary. If $b = 1$, send $(\mathsf{Bye}, \mathtt{sid}, \mathtt{ssid}, \{\Delta_i\}_{i\in[n]\setminus\mathcal{M}})$ to the adversary and abort. Otherwise, do nothing.
3. Sample $\boldsymbol{x} \xleftarrow{\$} \{0,1\}^{\ell_1}$ and invoke macro $\{\langle\!\langle \boldsymbol{x} \rangle\!\rangle_i\}_{i\in[n]} \xleftarrow{\$} \mathsf{AuthShare}(\mathtt{sid}, \mathtt{ssid}, \boldsymbol{x}, \{\Delta_i\}_{i\in[n]})$.
4. Sample $\boldsymbol{a}, \boldsymbol{b} \xleftarrow{\$} \{0,1\}^{\ell_2}$, compute component-wise AND $\boldsymbol{c} := \boldsymbol{a} \wedge \boldsymbol{b} \in \{0,1\}^{\ell_2}$, and invoke macro $\{\langle\!\langle \boldsymbol{x} \rangle\!\rangle_i\}_{i\in[n]} \xleftarrow{\$} \mathsf{AuthShare}(\mathtt{sid}, \mathtt{ssid}, \boldsymbol{x}, \{\Delta_i\}_{i\in[n]})$ for each $\boldsymbol{x} \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$.
5. For each $i \in [n]$, output $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, \Delta_i, \langle\!\langle \boldsymbol{x} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{a} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{b} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{c} \rangle\!\rangle_i)$ to P_i.
6. For each $i \in [n]$, append $(\mathtt{sid}, \mathtt{ssid}, i, \langle\!\langle \boldsymbol{x} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{a} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{b} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{c} \rangle\!\rangle_i)$ to A_i.
7. Ignore all subsequent $(\mathsf{Gen}, \mathtt{sid}, \mathtt{ssid}, \ldots)$ calls with the same $(\mathtt{sid}, \mathtt{ssid})$.

Global-key queries: Given stored $(\mathtt{sid}, i, \Delta_i)$ for each $i \in [n] \setminus \mathcal{M}$, this functionality outputs the randomness of honest parties if the adversary can guess the global key of some honest party: Upon receiving $(\mathsf{Guess}, \mathtt{sid}, i, \Delta_i')$ from the adversary, where $\Delta_i' \in \{0,1\}^\lambda$, if P_i is honest and $\Delta_i' = \Delta_i$, send $(\mathsf{Success}, \mathtt{sid}, \{A_i\}_{i\in[n]\setminus\mathcal{M}})$ to the adversary; otherwise, do nothing.

Figure 11: The functionality of multi-party preprocessing.

   (b) Otherwise, receive $(\mathsf{K}, \mathtt{sid}, \mathtt{ssid}, i, \{\mathsf{K}_j[\boldsymbol{x}^i]\}_{j\in\mathcal{M}})$ from the adversary, where $\mathsf{K}_j[\boldsymbol{x}^i] \in (\mathbb{F}_{2^\lambda})^\ell$ for each $j \in \mathcal{M}$, sample $\mathsf{K}_j[\boldsymbol{x}^i] \xleftarrow{\$} (\mathbb{F}_{2^\lambda})^\ell$ for each $j \notin \mathcal{M}$ and $j \neq i$, and compute $\mathsf{M}_j[\boldsymbol{x}^i] := \mathsf{K}_j[\boldsymbol{x}^i] \oplus \boldsymbol{x}^i \cdot \Delta_j \in (\mathbb{F}_{2^\lambda})^\ell$ for each $j \neq i$.
3. For each $i \in [n]$, compute $\langle\!\langle \boldsymbol{x} \rangle\!\rangle_i := (\boldsymbol{x}^i, \{\mathsf{M}_j[\boldsymbol{x}^i], \mathsf{K}_i[\boldsymbol{x}^j]\}_{j\neq i}) \in (\mathbb{F}_2)^\ell \times (\mathbb{F}_{2^\lambda})^{2\ell(n-1)}$.
4. Return $\{\langle\!\langle \boldsymbol{x} \rangle\!\rangle_i\}_{i\in[n]}$.

- $\mathsf{OpenTo}(\mathtt{sid}, \mathtt{ssid}, \langle\!\langle \boldsymbol{x} \rangle\!\rangle, i)$. This macro is essentially invoked by all parties in protocols Π_{Prep} and Π_{MPC} (in Section 10). Given a session ID $\mathtt{sid}$, a sub-session ID $\mathtt{ssid}$, an authenticated share $\langle\!\langle \boldsymbol{x} \rangle\!\rangle$ among n parties for a vector $\boldsymbol{x} \in (\mathbb{F}_2)^\ell$, and a party subscript $i \in [n]$:
  1. For each $j \in [n]$ and $j \neq i$, P_j sends $(\mathsf{In}, \mathtt{sid}, \mathtt{ssid}, \mathsf{M}_i[\boldsymbol{x}^j])$ to $\mathcal{F}_{\mathsf{RO}}$, which outputs $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, h_{j,i})$ to P_j, and sends $(\boldsymbol{x}^j, h_{j,i}) \in (\mathbb{F}_2)^\ell \times \mathbb{F}_{2^\lambda}$ to P_i.
  2. For each $j \in [n]$ and $j \neq i$, P_i sends $(\mathsf{In}, \mathtt{sid}, \mathtt{ssid}, \mathsf{K}_i[\boldsymbol{x}^j] \oplus \boldsymbol{x}^j \cdot \Delta_i)$ to $\mathcal{F}_{\mathsf{RO}}$, which

outputs $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, h'_{j,i})$ to P_i, and checks if $h_{j,i} = h'_{j,i}$. If this check fails, P_i aborts; otherwise, P_i does nothing.

- $\mathsf{Open}(\mathtt{sid}, \mathtt{ssid}, \langle\langle \boldsymbol{x} \rangle\rangle)$. This macro is invoked by all parties in protocols Π_{Prep} and Π_{MPC} (in Section 10). Given a session ID $\mathtt{sid}$, a sub-session ID $\mathtt{ssid}$, and an authenticated share $\langle\langle \boldsymbol{x} \rangle\rangle$ among n parties for a vector $\boldsymbol{x} \in (\mathbb{F}_2)^\ell$:

  1. For each $i \in [n]$, invoke macro $\mathsf{OpenTo}(\mathtt{sid}, \mathtt{ssid}, \langle\langle \boldsymbol{x} \rangle\rangle, i)$.

9.2 Authenticated Sharings

We propose an efficient protocol Π_{aShare}, which allows n parties to compute authenticated shares of secret bits. Each party first generates authenticated bit $[\![x^j]\!]_j$ known to itself using the correlated oblivious transfer functionality $\mathcal{F}_{\mathsf{COT}}$, which serves as the additive share of a bit $x = \bigoplus_{j\in[n]} x^j$ unknown to all parties. In case of more than two parties, we need to check the following constraints.

- The consistent x^j value is authenticated across different parties.
- The consistent global keys are used to authenticate values of different parties.

Following the method in [44], we can perform the two tests with random linear combination

Based on a similar observation [43, 44], we note that since $\mathcal{F}_{\mathsf{aBit}}$ ensures the consistency of the global key between different aBit operations, it sufficies to ensure that between different calls to $\mathcal{F}_{\mathsf{aBit}}$, the global keys are consistent. And thus we only need to check that some of the generated authenticated shares have consistency global keys.

Before proving the security of Π_{aShare}, we show a lemma that states whenever the adversary uses inconsistent global keys, it will be caught with overwhelming probability.

Lemma 11. *If all honest parties do not abort in protocol Π_{aShare}, then for every corrupted party $P_i, i \in \mathcal{M}$, all the global keys $\Delta_{i,j}$ are consistent except with probability $1/2^\lambda$. I.e., $R_{i,j} = 0$ for each $j \notin \mathcal{M}$ holds except with probability $1/2^\lambda$.*

Proof. In Step **??** of protocol Π_{aShare}, every party P_i broadcasts $\widetilde{y}^i$ and computes $y := \sum_{i\in[n]} \widetilde{y}^i$. However, every malicious party $P_i \in \mathcal{M}$ may broadcast an adversarial value $\hat{y}^i$, such that $\hat{y} := \sum_{i\in\mathcal{M}} \hat{y}^i + \sum_{i\notin\mathcal{M}} \widetilde{y}^i = y + e_y$, where e_y is an additive error of the adversary's choice.

We define z_i^j to be the value committed by a party P_j when P_j behaves honestly. The corrupted parties may deviate the protocol by committing the values $\hat{z}_i^k$ for $k \in \mathcal{M}$, in such a way that $\sum_{k\in\mathcal{M}} \hat{z}_i^k = \sum_{k\in\mathcal{M}} z_i^k + E_i$, where E_i is an adversarially chosen error.

If a malicious party P_i tries to cheat, then it has to pass the check in Step **??** of protocol Π_{aShare}. Therefore, we have the following:

$$
\begin{aligned}
0 &= \textstyle\sum_{j\notin\mathcal{M}} z_i^j + \sum_{j\in\mathcal{M}} \hat{z}_i^j = z_i^i + \sum_{j\neq i} z_i^j + E_i \\
&= \Big(\textstyle\sum_{j\neq i} \mathsf{K}_i[y^j] + \big(y^i + y + e_y\big) \cdot \Delta_i \Big) + \sum_{j\neq i} \mathsf{M}_i[y^j] + E_i \\
&= \textstyle\sum_{j\neq i} \big(\mathsf{K}_i[y^j] + \mathsf{M}_i[y^j]\big) + \big(y^i + y + e_y\big) \cdot \Delta_i + E_i \\
&= \textstyle\sum_{j\neq i} y^j \cdot \Delta_{i,j} + \big(y^i + y + e_y\big) \cdot \Delta_i + E_i \\
&= \textstyle\sum_{j\neq i} y^j \cdot R_{i,j} + \big(y^i + y + e_y + \sum_{j\neq i} y^j\big) \cdot \Delta_i + E_i \\
&= \textstyle\sum_{j\neq i} y^j \cdot R_{i,j} + e_y \cdot \Delta_i + E_i.
\end{aligned}
$$

Functionality $\mathcal{F}_{\mathsf{aShare}}$

This functionality runs with parties $P_1, \ldots, P_n$. Let $\mathcal{M} \subset [n]$ be the subscript set of corrupted parties and [Xiaojie: $\mathcal{L} \subseteq \mathcal{P}(\mathbb{F}_{2^\lambda})$ be defined in XXX].

Parameter: A set $\{\mathcal{R}_i\}_{i\in[n]}$ of global-key distributions on $\mathbb{F}_{2^\lambda}$.

Initialization: Upon receiving $(\mathsf{Init}, \mathtt{sid}, i)$ from P_i,

1. If P_i is corrupted, receive $(\mathsf{GK}, \mathtt{sid}, i, \Delta_i)$ from the adversary, where $\Delta_i \in \{0,1\}^\lambda$. Otherwise, sample $\Delta_i \xleftarrow{\$} \mathcal{R}_i$.
2. Store $(\mathtt{sid}, i, \Delta_i)$ and ignore all subsequent $(\mathsf{Init}, \mathtt{sid}, i)$ calls with the same $(\mathtt{sid}, i)$.

Authenticated shares: Given stored $(\mathtt{sid}, i, \Delta_i)$ for each $i \in [n]$, this functionality outputs random authenticated shares: Upon receiving $(\mathsf{Gen}, \mathtt{sid}, \mathtt{ssid}, \ell_1)$ from all parties, where $\ell_1 \in \mathbb{N}^+$,

1. **Selective-failure queries:** For each $i \in [n] \setminus \mathcal{M}$ and $j \in \mathcal{M}$, proceed as follows in parallel:
   (a) Receive $(\mathsf{Leak}, \mathtt{sid}, \mathtt{ssid}, i, j, L_{i,j})$ from the adversary, where $L_{i,j} \in \mathcal{L}$.
   (b) If $\Delta_i \notin L_{i,j}$, send $(\mathsf{Fail}, \mathtt{sid}, \mathtt{ssid}, i, j)$ to the adversary and abort; otherwise, do nothing.
2. **Global-key extraction with abort:** Receive $(\mathsf{Quit}, \mathtt{sid}, \mathtt{ssid}, b)$ from the adversary. If $b = 1$, send $(\mathsf{Bye}, \mathtt{sid}, \mathtt{ssid}, \{\Delta_i\}_{i\in[n]\setminus\mathcal{M}})$ to the adversary and abort. Otherwise, do nothing.
3. Sample $\boldsymbol{x} \xleftarrow{\$} \{0,1\}^{\ell_1}$ and invoke macro $\{\langle\!\langle \boldsymbol{x} \rangle\!\rangle_i\}_{i\in[n]} \xleftarrow{\$} \mathsf{AuthShare}(\mathtt{sid}, \mathtt{ssid}, \boldsymbol{x}, \{\Delta_i\}_{i\in[n]})$.
4. For each $i \in [n]$, output $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, \Delta_i, \langle\!\langle \boldsymbol{x} \rangle\!\rangle_i)$ to P_i.
5. Ignore all subsequent $(\mathsf{Gen}, \mathtt{sid}, \mathtt{ssid}, \ldots)$ calls with the same $(\mathtt{sid}, \mathtt{ssid})$.

Figure 12: The functionality of authenticated shares.

If a malicious party P_i provides inconsistent global keys, then there exists $j_0, j_1 \notin \mathcal{M}$ such that $R_{i,j_0} \neq R_{i,j_1}$ and $j_0 \neq j_1$. Therefore, the adversary needs to set $E_i + e_y \cdot \Delta_i = y^{j_0} \cdot R_{i,j_0} + y^{j_1} \cdot R_{i,j_1}$. Due to the re-randomization by random zero-sharing, from the adversary's view, y^{j_0} and y^{j_1} are uniformly random additive shares of y. Thus, the equation holds except with probability $2^{-\lambda}$, and so is the probability of $\mathcal{A}$ to pass the check with inconsistent keys. □

Now we prove the security of Π_{aShare}.

Theorem 3. *Protocol Π_{aShare} (in Figure 13) UC-realizes functionality $\mathcal{F}_{\mathsf{aShare}}$ (in Figure 12) in the $(\mathcal{F}_{\mathsf{mCOT}}, \mathcal{F}_{\mathsf{Com}}, \mathcal{F}_{\mathsf{Rand}}, \mathcal{F}_{\mathsf{BC}})$-hybrid model.*

9.3 Leaky Authenticated AND Triples

Theorem 4. *In the ICM, given ICM-based $(\frac{n^2\ell_2}{4}, q_E + \frac{n^2\ell_2}{4}, n, n-1, n, \lambda-1, 1, \varepsilon_{\mathsf{H}})$-UC-mTCCRL-secure hash function $\mathsf{H} = \widehat{\mathsf{MMO}}^E$ and a family $\mathcal{H} = \{\mathsf{H}' : (\{0,1\}^\lambda)^{\ell_2} \to \{0,1\}^\lambda\}$ of linear ε-almost universal hash functions, protocol Π_{LaAND} (Figure 15) UC-realizes functionality $\mathcal{F}_{\mathsf{LaAND}}$ (Figure 14) in the $(\mathcal{F}_{\mathsf{aShare}}, \mathcal{F}_{\mathsf{Zero}}, \mathcal{F}_{\mathsf{Com}}, \mathcal{F}_{\mathsf{Rand}})$-hybrid model with advantage at most*

$$\varepsilon_{\mathsf{H}} + 2\varepsilon + \frac{5}{2^\lambda} + \frac{3q_E}{2^{\lambda-1}}$$

against any adversary making at most q_E ICM queries, when $(\mathsf{Gen}, \mathtt{sid}, \ldots)$ command is invoked only once per $\mathtt{sid}$.

Protocol Π_{aShare}

Initialization: All parties initialize their global keys in $\mathcal{F}_{\mathsf{mCOT}}$.

1. For each $i \in [n]$, P_i sends $(\mathsf{Init}, \mathtt{sid}, i)$ to $\mathcal{F}_{\mathsf{mCOT}}$, which is parameterized by $\{\mathcal{R}_i\}_{i\in[n]}$.

Authenticated shares: All parties pairwise generate $\ell' > \lambda + \ell_1$ random COT tuples.

2. For each $i, j \in [n]$ and $j \neq i$, P_i and P_j send $(\mathsf{Extend}, \mathtt{sid}, \mathtt{ssid}, i, j, \ell')$ to $\mathcal{F}_{\mathsf{mCOT}}$, which outputs $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, j, \Delta_i, \mathsf{K}_i[\boldsymbol{s}^{j,i}] \| \mathsf{K}_i[\boldsymbol{r}^{j,i}])$ to P_i and $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, j, \boldsymbol{s}^{j,i} \| \boldsymbol{r}^{j,i}, \mathsf{M}_i[\boldsymbol{s}^{j,i}] \| \mathsf{M}_i[\boldsymbol{r}^{j,i}])$ to P_j, where $\Delta_i \in \mathbb{F}_{2^\lambda}$ and

$$(\boldsymbol{s}^{j,i}, \mathsf{K}_i[\boldsymbol{s}^{j,i}], \mathsf{M}_i[\boldsymbol{s}^{j,i}]) \in (\mathbb{F}_2)^\lambda \times (\mathbb{F}_{2^\lambda})^\lambda \times (\mathbb{F}_{2^\lambda})^\lambda,$$
$$(\boldsymbol{r}^{j,i}, \mathsf{K}_i[\boldsymbol{r}^{j,i}], \mathsf{M}_i[\boldsymbol{r}^{j,i}]) \in (\mathbb{F}_2)^{\ell'-\lambda} \times (\mathbb{F}_{2^\lambda})^{\ell'-\lambda} \times (\mathbb{F}_{2^\lambda})^{\ell'-\lambda}.$$

All parties check the consistency of global keys. Let $\boldsymbol{g} := (1, X, \ldots, X^{\lambda-1}) \in (\mathbb{F}_{2^\lambda})^\lambda$ be gadget vector.

3. For each $i, j \in [n]$ and $j \neq i$, P_i computes $x^{i,j} := \langle \boldsymbol{g}, \boldsymbol{s}^{i,j} \rangle \in \mathbb{F}_{2^\lambda}$, $\mathsf{M}_j[x^{i,j}] := \langle \boldsymbol{g}, \mathsf{M}_j[\boldsymbol{s}^{i,j}] \rangle \in \mathbb{F}_{2^\lambda}$, and $\mathsf{K}_i[x^{j,i}] := \langle \boldsymbol{g}, \mathsf{K}_i[\boldsymbol{s}^{j,i}] \rangle \in \mathbb{F}_{2^\lambda}$.

4. All parties send $(\mathsf{Zero}, \mathtt{sid}, \mathtt{ssid}, n)$ to $\mathcal{F}_{\mathsf{Zero}}$, which outputs $(\mathsf{Val}, \mathtt{sid}, \mathtt{ssid}, \boldsymbol{u}^i)$ to P_i for each $i \in [n]$, where $\boldsymbol{u}^i \in (\mathbb{F}_{2^\lambda})^n$.

5. For each $i \in [n]$, P_i computes $\boldsymbol{y}^i := (x^{i,0}, \ldots, x^{i,i-1}, 0, x^{i,i+1}, \ldots, x^{i,n-1}) \oplus \boldsymbol{u}^i \in (\mathbb{F}_{2^\lambda})^n$ and sends $(\mathsf{BC}, \mathtt{sid}, \mathtt{ssid} \| 0, i, \boldsymbol{y}^i)$ to $\mathcal{F}_{\mathsf{BC}}$, which outputs $(\mathsf{Msg}, \mathtt{sid}, \mathtt{ssid} \| 0, i, \boldsymbol{y}^i)$ to all parties.

6. For each $i \in [n]$, P_i computes $\boldsymbol{y} := \bigoplus_{j\in[n]} \boldsymbol{y}^j \in (\mathbb{F}_{2^\lambda})^n$, $z_i^i := \bigoplus_{j\neq i} \mathsf{K}_i[x^{j,i}] \oplus y_i \cdot \Delta_i \in \mathbb{F}_{2^\lambda}$, and $\boldsymbol{z}^i := \left\{ \{ z_j^i := \mathsf{M}_j[x^{i,j}] \}_{j\neq i}, z_i^i \right\} \in (\mathbb{F}_{2^\lambda})^n$, and sends $(\mathsf{Commit}, \mathtt{sid}, \mathtt{ssid}, i, \boldsymbol{z}^i)$ to $\mathcal{F}_{\mathsf{Com}}$.

7. For each $i \in [n]$, after receiving $(\mathsf{Committed}, \mathtt{sid}, \mathtt{ssid}, j)$ from $\mathcal{F}_{\mathsf{Com}}$ for all $j \in [n]$ and $j \neq i$, P_i sends $(\mathsf{Open}, \mathtt{sid}, \mathtt{ssid}, i)$ to $\mathcal{F}_{\mathsf{Com}}$, which outputs $(\mathsf{Opened}, \mathtt{sid}, \mathtt{ssid}, i, \boldsymbol{z}^i)$ to all parties, and checks if $\bigoplus_{k\in[n]} z_j^k = 0$ for each $j \in [n]$. If this check fails, P_i aborts; otherwise, P_i does nothing.

All parties compute $\ell' - \lambda > \ell_1$ authenticated shares without correctness guarantee.

8. For each $i \in [n]$, P_i samples $\boldsymbol{x}^i \xleftarrow{\$} (\mathbb{F}_2)^{\ell'-\lambda}$.

9. For each $i, j \in [n]$ and $j \neq i$, P_i sends $\boldsymbol{d}^{i,j} := \boldsymbol{r}^{i,j} \oplus \boldsymbol{x}^i \in (\mathbb{F}_2)^{\ell'-\lambda}$ to P_j, P_i computes $\mathsf{M}_j[\boldsymbol{x}^i] := \mathsf{M}_j[\boldsymbol{r}^{i,j}] \in (\mathbb{F}_{2^\lambda})^{\ell'-\lambda}$, and P_j computes $\mathsf{K}_j[\boldsymbol{x}^i] := \mathsf{K}_j[\boldsymbol{r}^{i,j}] \oplus \boldsymbol{d}^{i,j} \cdot \Delta_j \in (\mathbb{F}_{2^\lambda})^{\ell'-\lambda}$.

All parties check the consistency of generated authenticated shares. Let $\mathcal{H} = \{\mathsf{H} : (\mathbb{F}_{2^\lambda})^{\ell'-\lambda} \to \mathbb{F}_{2^\lambda}\}$ be a family of $(\mathbb{F}_2)^{\ell_1}$-hiding 1-universal hash functions.

10. All parties send $(\mathsf{Rand}, \mathtt{sid}, \mathtt{ssid}, \mathcal{H})$ to $\mathcal{F}_{\mathsf{Rand}}$, which outputs $(\mathsf{Res}, \mathtt{sid}, \mathtt{ssid}, \mathsf{H})$ to all parties.

11. For each $i \in [n]$, P_i computes $w^i := \mathsf{H}(\boldsymbol{x}^i) \in \mathbb{F}_{2^\lambda}$ and sends $(\mathsf{BC}, \mathtt{sid}, \mathtt{ssid} \| 1, i, w^i)$ to $\mathcal{F}_{\mathsf{BC}}$, which outputs $(\mathsf{Msg}, \mathtt{sid}, \mathtt{ssid} \| 1, i, w^i)$ to all parties.

    For each $i, j \in [n]$ and $j \neq i$, P_i computes $\mathsf{M}_j[w^i] := \mathsf{H}(\mathsf{M}_j[\boldsymbol{x}^i]) \in \mathbb{F}_{2^\lambda}$ and sends $\mathsf{M}_j[w^i]$ to P_j.

12. For each $i, j \in [n]$ and $j \neq i$, P_i computes $\mathsf{K}_i[w^j] := \mathsf{H}(\mathsf{K}_i[\boldsymbol{x}^j]) \in \mathbb{F}_{2^\lambda}$ and checks if $\mathsf{M}_i[w^j] = \mathsf{K}_i[w^j] \oplus w^j \cdot \Delta_i$. If this check fails, P_i aborts; otherwise, P_i outputs

$$\left(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, \Delta_i, \boldsymbol{x}^i[0, \ell_1), \left\{ \mathsf{M}_j[\boldsymbol{x}^i][0, \ell_1), \mathsf{K}_i[\boldsymbol{x}^j][0, \ell_1) \right\}_{j\neq i}\right).$$

Figure 13: The protocol of authenticated shares.

Proof. The requirement that $(\mathsf{Gen}, \mathtt{sid}, \ldots)$ command is invoked only once per $\mathtt{sid}$ ensures that, for UC security, the dummy adversary (i.e., environment) cannot use the global keys of honest parties

Functionality $\mathcal{F}_{\mathsf{LaAND}}$

This functionality runs with parties $P_1, \ldots, P_n$. Let $\mathcal{M} \subset [n]$ be the subscript set of corrupted parties and [Xiaojie: $\mathcal{L} \subseteq \mathcal{P}(\mathbb{F}_{2^\lambda})$ be defined in XXX]. Let $\{\mathcal{R}_i\}_{i\in[n]}$ be the set of distributions on $\{0,1\}^\lambda$ such that $\mathcal{R}_i$ fixes the LSB of each sample to (n **mod** 2) for $i = 1$ and to 1 for each $i \neq 1$.

Initialization: Upon receiving $(\mathsf{Init}, \mathtt{sid}, i)$ from P_i,

1. If P_i is corrupted, receive $(\mathsf{GK}, \mathtt{sid}, i, \Delta_i)$ from the adversary, where $\Delta_i \in \{0,1\}^\lambda$. Otherwise, sample $\Delta_i \xleftarrow{\$} \mathcal{R}_i$.
2. Store $(\mathtt{sid}, i, \Delta_i)$, initialize an empty list A_i of outputs, and ignore all subsequent $(\mathsf{Init}, \mathtt{sid}, i)$ calls with the same $(\mathtt{sid}, i)$.

Authenticated shares & Leaky authenticated AND triples: Given stored $(\mathtt{sid}, i, \Delta_i)$ for each $i \in [n]$, this functionality outputs such random tuples: Upon receiving $(\mathsf{Gen}, \mathtt{sid}, \mathtt{ssid}, \ell_1, \ell_2)$ from all parties, where $\ell_1, \ell_2 \in \mathbb{N}^+$ and $\ell_2 \geq \lambda$,

1. **Selective-failure queries:** For each $i \in [n] \setminus \mathcal{M}$ and $j \in \mathcal{M}$, proceed as follows in parallel:
   (a) Receive $(\mathsf{Leak}, \mathtt{sid}, \mathtt{ssid}, i, j, L_{i,j})$ from the adversary, where $L_{i,j} \in \mathcal{L}$.
   (b) If $\Delta_i \notin L_{i,j}$, send $(\mathsf{Fail}, \mathtt{sid}, \mathtt{ssid}, i, j)$ to the adversary and abort; otherwise, do nothing.
2. **Global-key extraction with abort:** Receive $(\mathsf{Quit}, \mathtt{sid}, \mathtt{ssid}, b)$ from the adversary. If $b = 1$, send $(\mathsf{Bye}, \mathtt{sid}, \mathtt{ssid}, \{\Delta_i\}_{i\in[n]\setminus\mathcal{M}})$ to the adversary and abort. Otherwise, do nothing.
3. Sample $\boldsymbol{x} \xleftarrow{\$} \{0,1\}^{\ell_1}$ and invoke macro $\{\langle\!\langle \boldsymbol{x} \rangle\!\rangle_i\}_{i\in[n]} \xleftarrow{\$} \mathsf{AuthShare}(\mathtt{sid}, \mathtt{ssid}, \boldsymbol{x}, \{\Delta_i\}_{i\in[n]})$.
4. Sample $\boldsymbol{a}, \boldsymbol{b} \xleftarrow{\$} \{0,1\}^{\ell_2}$, compute component-wise AND $\boldsymbol{c} := \boldsymbol{a} \wedge \boldsymbol{b} \in \{0,1\}^{\ell_2}$, and invoke macro $\{\langle\!\langle \boldsymbol{x} \rangle\!\rangle_i\}_{i\in[n]} \xleftarrow{\$} \mathsf{AuthShare}(\mathtt{sid}, \mathtt{ssid}, \boldsymbol{x}, \{\Delta_i\}_{i\in[n]})$ for each $\boldsymbol{x} \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$.
5. **Share queries with selective failure (Leakage):**
   (a) Receive $(\mathsf{LeakA}, \mathtt{sid}, \mathtt{ssid}\|0, \{\boldsymbol{e}^j\}_{j\in[n]\setminus\mathcal{M}}, \boldsymbol{q})$ from the adversary, where $\boldsymbol{e}^j \in (\{0,1\}^\lambda)^{\ell_2}$ and $\boldsymbol{q} \in \{0,1\}^{\ell_2}$ for each $j \in [n] \setminus \mathcal{M}$.
   (b) Sample a function $\mathsf{H}' \xleftarrow{\$} \mathcal{H}$, where $\mathcal{H} = \{\mathsf{H}' : (\{0,1\}^\lambda)^{\ell_2} \to \{0,1\}^\lambda\}$ be a family of linear universal hash functions, and send $(\mathsf{Hash}, \mathtt{sid}, \mathtt{ssid}, \mathsf{H}')$ to the adversary.
   (c) Receive $(\mathsf{LeakA}, \mathtt{sid}, \mathtt{ssid}\|1, \delta)$ from the adversary, where $\delta \in \{0,1\}^\lambda$, and check
   $$\mathsf{H}'\left(\bigoplus_{j\in[n]\setminus\mathcal{M}} (\boldsymbol{a}^j \odot \boldsymbol{e}^j) \oplus \left(\boldsymbol{q} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j)\right) \cdot \bigoplus_{j\in[n]\setminus\mathcal{M}} \Delta_j\right) = \delta.$$
   If this check fails or $\mathsf{lsb}(\bigoplus_{i\in[n]} \Delta_i) = 0$, abort; otherwise, do nothing.
6. For each $i \in [n]$, output $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, \Delta_i, \langle\!\langle \boldsymbol{x} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{a} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{b} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{c} \rangle\!\rangle_i)$ to P_i.
7. For each $i \in [n]$, append $(\mathtt{sid}, \mathtt{ssid}, i, \langle\!\langle \boldsymbol{x} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{a} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{b} \rangle\!\rangle_i, \langle\!\langle \boldsymbol{c} \rangle\!\rangle_i)$ to A_i.
8. Ignore all subsequent $(\mathsf{Gen}, \mathtt{sid}, \mathtt{ssid}, \ldots)$ calls with the same $(\mathtt{sid}, \mathtt{ssid})$.

Global-key queries: Given stored $(\mathtt{sid}, i, \Delta_i)$ for each $i \in [n] \setminus \mathcal{M}$, this functionality outputs the randomness of honest parties if the adversary can guess the global key of some honest party: Upon receiving $(\mathsf{Guess}, \mathtt{sid}, i, \Delta_i')$ from the adversary, where $\Delta_i' \in \{0,1\}^\lambda$, if P_i is honest and $\Delta_i' = \Delta_i$, send $(\mathsf{Success}, \mathtt{sid}, \{A_i\}_{i\in[n]\setminus\mathcal{M}})$ to the adversary; otherwise, do nothing.

Figure 14: The functionality of leaky authenticated AND triples.

Protocol Π_{LaAND}

Initialization: All parties initialize their global keys in $\mathcal{F}_{\mathsf{aShare}}$.

1. For each $i \in [n]$, P_i sends $(\mathsf{Init}, \mathtt{sid}, i)$ to $\mathcal{F}_{\mathsf{aShare}}$, which is parameterized by $\{\mathcal{R}_i\}_{i\in[n]}$.

Authenticated shares & Leaky authenticated AND triples: All parties generate $\ell := \ell_1 + 3\ell_2$ random authenticated shares.

2. All parties send $(\mathsf{Gen}, \mathtt{sid}, \mathtt{ssid}, \ell)$ to $\mathcal{F}_{\mathsf{aShare}}$, which outputs $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, \Delta_i, \langle\langle \boldsymbol{x}\|\boldsymbol{a}\|\boldsymbol{b}\|\boldsymbol{r}\rangle\rangle_i)$ to P_i for each $i \in [n]$, where $\boldsymbol{x} \in \{0,1\}^{\ell_1}$ and $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{r} \in \{0,1\}^{\ell_2}$.

All parties share $\langle\langle \boldsymbol{c}\rangle\rangle = \langle\langle \boldsymbol{a} \wedge \boldsymbol{b}\rangle\rangle$. Let $\mathsf{H} : (\{0,1\}^\lambda)^2 \to \{0,1\}^\lambda$ be a UC-mTCCRL-secure hash function and $\mathcal{H} = \{\mathsf{H}' : (\{0,1\}^\lambda)^{\ell_2} \to \{0,1\}^\lambda\}$ be a family of linear uniform hash functions.

3. All parties send $(\mathsf{Zero}, \mathtt{sid}, \mathtt{ssid}, \ell_2 + 1)$ to $\mathcal{F}_{\mathsf{Zero}}$, which outputs $(\mathsf{Val}, \mathtt{sid}, \mathtt{ssid}, \boldsymbol{z}^i \| u^i)$ to P_i for each $i \in [n]$, where $\boldsymbol{z}^i \in (\{0,1\}^\lambda)^{\ell_2}$ and $u^i \in \{0,1\}^\lambda$.
4. For each $i \in [n]$, P_i computes $\boldsymbol{\Phi}^i := \boldsymbol{b}^i \cdot \Delta_i \oplus \bigoplus_{j\neq i} \left(\mathsf{K}_i[\boldsymbol{b}^j] \oplus \mathsf{M}_j[\boldsymbol{b}^i]\right) \oplus \boldsymbol{z}^i \in (\{0,1\}^\lambda)^{\ell_2}$.
5. For each $k \in [\ell_2]$, proceed as follows in parallel:
   (a) For each $i, j \in [n]$ and $j \neq i$, P_i computes

$$\begin{aligned} T_{i,j,k} &:= \mathtt{sid}\|\mathtt{ssid}\|i\|j\|k \in \{0,1\}^\lambda, \\ \mathsf{K}_i[a_k^j]_{\Phi_k^i} &:= \mathsf{H}(\mathsf{K}_i[a_k^j], T_{i,j,k}) \in \{0,1\}^\lambda, \\ u_k^{i,j} &:= \mathsf{K}_i[a_k^j]_{\Phi_k^i} \oplus \mathsf{H}(\mathsf{K}_i[a_k^j] \oplus \Delta_i, T_{i,j,k}) \oplus \Phi_k^i \in \{0,1\}^\lambda, \end{aligned}$$

   sends $u_k^{i,j}$ to P_j, and computes $\mathsf{M}_j[a_k^i]_{\Phi_k^j} := a_k^i \cdot u_k^{j,i} \oplus \mathsf{H}(\mathsf{M}_j[a_k^i], T_{i,j,k}) \in \{0,1\}^\lambda$.
   (b) For each $i \in [n]$, P_i computes

$$s_k^i := a_k^i \cdot \Phi_k^i \oplus \bigoplus_{j\neq i} \left(\mathsf{K}_i[a_k^j]_{\Phi_k^i} \oplus \mathsf{M}_j[a_k^i]_{\Phi_k^j}\right) \oplus r_k^i \cdot \Delta_i \oplus \bigoplus_{j\neq i} \left(\mathsf{K}_i[r_k^j] \oplus \mathsf{M}_j[r_k^i]\right) \in \{0,1\}^\lambda,$$

   and $d_k^i := \mathsf{lsb}(s_k^i) \in \{0,1\}$.
6. For each $i \in [n]$, P_i sends $(\mathsf{Commit}, \mathtt{sid}, \mathtt{ssid}\|0, i, \boldsymbol{d}^i)$ to $\mathcal{F}_{\mathsf{Com}}$.
7. For each $i \in [n]$, after receiving $(\mathsf{Committed}, \mathtt{sid}, \mathtt{ssid}\|0, j)$ from $\mathcal{F}_{\mathsf{Com}}$ for all $j \in [n]$ and $j \neq i$, P_i sends $(\mathsf{Open}, \mathtt{sid}, \mathtt{ssid}\|0, i)$ to $\mathcal{F}_{\mathsf{Com}}$, which outputs $(\mathsf{Opened}, \mathtt{sid}, \mathtt{ssid}\|0, i, \boldsymbol{d}^i)$ to all parties, and computes $\boldsymbol{d} := \bigoplus_{i\in[n]} \boldsymbol{d}^i \in \{0,1\}^{\ell_2}$ and $\boldsymbol{t}^i := \boldsymbol{s}^i \oplus \boldsymbol{d} \cdot \Delta_i \in (\{0,1\}^\lambda)^{\ell_2}$.
8. All parties send $(\mathsf{Rand}, \mathtt{sid}, \mathtt{ssid}, \mathcal{H})$ to $\mathcal{F}_{\mathsf{Rand}}$, which outputs $(\mathsf{Res}, \mathtt{sid}, \mathtt{ssid}, \mathsf{H}')$ to all parties.
9. For each $i \in [n]$, P_i computes $\alpha^i := \mathsf{H}'(\boldsymbol{t}^i) + u^i$ and sends $(\mathsf{Commit}, \mathtt{sid}, \mathtt{ssid}\|1, i, \alpha^i)$ to $\mathcal{F}_{\mathsf{Com}}$.
10. For each $i \in [n]$, after receiving $(\mathsf{Committed}, \mathtt{sid}, \mathtt{ssid}\|1, j)$ from $\mathcal{F}_{\mathsf{Com}}$ for all $j \in [n]$ and $j \neq i$, P_i sends $(\mathsf{Open}, \mathtt{sid}, \mathtt{ssid}\|1, i)$ to $\mathcal{F}_{\mathsf{Com}}$, which outputs $(\mathsf{Opened}, \mathtt{sid}, \mathtt{ssid}\|1, i, \alpha^i)$ to all parties, and checks if $\bigoplus_{i\in[n]} \alpha^i = 0$. If this check fails, P_i aborts; otherwise, P_i does nothing.
11. All parties compute $\langle\langle \boldsymbol{c}\rangle\rangle := \langle\langle \boldsymbol{r}\rangle\rangle \oplus \boldsymbol{d}$.
12. For each $i \in [n]$, P_i outputs $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, \Delta_i, \langle\langle \boldsymbol{x}\rangle\rangle_i, \langle\langle \boldsymbol{a}\rangle\rangle_i, \langle\langle \boldsymbol{b}\rangle\rangle_i, \langle\langle \boldsymbol{c}\rangle\rangle_i)$.

Figure 15: The protocol of leaky authenticated AND triples.

$\{\Delta_i\}_{i\in[n]\setminus\mathcal{M}}$ output at the end of one Gen execution to break the check in another execution. For a single Gen execution, we prove the UC security via the following hybrid argument:

- **Hybrid**$_0$. This is the real-world execution of protocol Π_{LaAND}.
- **Hybrid**$_1$. This hybrid is identical to the previous one, except that there is a machine that plays the role of all honest parties as per Π_{LaAND} and emulates all subroutine functionalities (including $\mathcal{F}_{\mathsf{aShare}}$, $\mathcal{F}_{\mathsf{Zero}}$, $\mathcal{F}_{\mathsf{Com}}$, and $\mathcal{F}_{\mathsf{Rand}}$) for all parties. This modification is conceptual so the joint distribution in this hybrid is perfectly indistinguishable from the previous one.
- **Hybrid**$_2$. This hybrid is identical to the previous one, except that the machine changes the way to emulate $\mathcal{F}_{\mathsf{aShare}}$ for all honest parties: it reversely samples uniform $\mathsf{M}_u[\boldsymbol{b}^j]$ and sets $\mathsf{K}_u[\boldsymbol{b}^j] := \mathsf{M}_u[\boldsymbol{b}^j] \oplus \boldsymbol{b}^j \cdot \Delta_u$ for each $j, u \in [n] \setminus \mathcal{M}$ and $u \neq j$.

  Clearly, this hybrid is perfectly indistinguishable from the previous one.
- **Hybrid**$_3$. This hybrid is identical to the previous one, except that the machine changes the way to compute the messages sent from honest parties to corrupted parties at step 5a:

  1. It defines $\mathbf{R} := \{\Delta_j\}_{j\in[n]\setminus\mathcal{M}}$, invokes the universal UC-mTCCRL simulator $\mathcal{S}_{\mathsf{H}}$ for hash function H to emulate the ideal cipher E and its inverse E^{-1}, and implements the oracle $\mathsf{mKGuess}_{\mathbf{R}}$ accessed by $\mathcal{S}_{\mathsf{H}}$.
  2. It samples uniform $\{u_k^{j,i}\}_{i\in\mathcal{M},j\in[n]\setminus\mathcal{M},k\in[\ell_2]}$ on behalf of all honest parties at step 5a and computes the following $\mathcal{Q}_F$ to switch $\mathcal{S}_{\mathsf{H}}$ from **Phase 1** into **Phase 2** before sending the message of any honest party:

$$\mathcal{Q}_F := \left\{ \left(j, \underbrace{\mathsf{M}_j[a_k^i]}, T_{j,i,k}, \underbrace{\bigoplus_{i\in\mathcal{M}} b_k^i} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} b_k^j \right), \gamma_{j,i,k} \right\}_{i\in\mathcal{M},j\in[n]\setminus\mathcal{M},k\in[\ell_2]},$$

$$\gamma_{j,i,k} := u_k^{j,i} \oplus \mathsf{H}(\underbrace{\mathsf{M}_j[a_k^i]}, T_{j,i,k}) \oplus \bigoplus_{u\in[n]\setminus\mathcal{M},u\neq j} \left(\mathsf{M}_j[b_k^u] \oplus \mathsf{M}_u[b_k^j]\right) \oplus z_k^j$$
$$\oplus \bigoplus_{u\in\mathcal{M}} \left(\underbrace{\mathsf{M}_j[b_k^u]} \oplus \underbrace{\mathsf{K}_u[b_k^j]} \oplus b_k^j \cdot \underbrace{\Delta_u}\right),$$

     where the underlined terms are the randomness of corrupted parties and are extracted from the emulated $\mathcal{F}_{\mathsf{aShare}}$.
  3. It follows Π_{LaAND} to compute all values used by the honest parties since step 5b.

  From Lemma 12, these two hybrids are indistinguishable with advantage at most ε_{H}.
- **Hybrid**$_4$. This hybrid is identical to the previous one, except that the machine additionally computes the following $(\{\boldsymbol{e}^j\}_{j\in[n]\setminus\mathcal{M}}, \boldsymbol{q}, \delta)$:

  1. For each $i \in \mathcal{M}, j \in [n] \setminus \mathcal{M}, k \in [\ell_2]$, it follows step 5a to compute $u_k^{i,j} \in \{0,1\}^\lambda$ and receives some $\widetilde{u_k^{i,j}} \in \{0,1\}^\lambda$ from the corrupted P_i in step 5a.
  2. For each $j \in [n] \setminus \mathcal{M}$, it computes an error $\boldsymbol{e}^j \in (\{0,1\}^\lambda)^{\ell_2}$, where for each $k \in [\ell_2]$,

$$e_k^j := \bigoplus_{i\in\mathcal{M}} \left(\widetilde{u_k^{i,j}} \oplus u_k^{i,j}\right) \in \{0,1\}^\lambda,$$

     and follows step 5a and 5b to compute $(\widetilde{\boldsymbol{s}^j}, \widetilde{\boldsymbol{d}^j})$ (resp., $(\boldsymbol{s}^j, \boldsymbol{d}^j)$) using $\{\widetilde{u_k^{i,j}}\}_{i\in\mathcal{M}}$ (resp., $\{u_k^{i,j}\}_{i\in\mathcal{M}}$). According to the steps, it holds that

$$\widetilde{\boldsymbol{s}^j} = \boldsymbol{s}^j \oplus \boldsymbol{a}^j \odot \boldsymbol{e}^j \in (\{0,1\}^\lambda)^{\ell_2},$$
$$\widetilde{\boldsymbol{d}^j} = \boldsymbol{d}^j \oplus \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j) \in \{0,1\},$$

where $\odot$ denotes component-wise multiplication. Note that for each $j \in [n] \setminus \mathcal{M}$, the machine uses $(\widetilde{\boldsymbol{s}^j}, \widetilde{\boldsymbol{d}^j})$ to proceed as an honest P_j after step 5b.

3. It follows step 5a and 5b to compute $\{\boldsymbol{s}^i, \boldsymbol{d}^i\}_{i\in\mathcal{M}}$, extracts $\{\widetilde{\boldsymbol{d}^i}\}_{i\in\mathcal{M}}$ from the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 6, computes

$$\boldsymbol{q} := \bigoplus_{i\in\mathcal{M}} \left(\widetilde{\boldsymbol{d}^i} \oplus \boldsymbol{d}^i\right) \in \{0,1\}^{\ell_2},$$

$$\boldsymbol{d} := \bigoplus_{i\in[n]} \boldsymbol{d}^i = \mathsf{lsb}\Big(\bigoplus_{i\in[n]} \boldsymbol{s}^i\Big) = (\boldsymbol{a}\wedge\boldsymbol{b}\oplus\boldsymbol{r}) \cdot \mathsf{lsb}\Big(\bigoplus_{i\in[n]} \Delta_i\Big) \in \{0,1\}^{\ell_2}, \tag{8}$$

$$\widetilde{\boldsymbol{d}} := \bigoplus_{i\in[n]} \widetilde{\boldsymbol{d}^i} = \boldsymbol{d} \oplus \boldsymbol{q} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j) \in \{0,1\}^{\ell_2}, \tag{9}$$

follows step 7 to compute $\{\boldsymbol{t}^i\}_{i\in\mathcal{M}}$ using $(\{\boldsymbol{s}^i, \Delta_i\}_{i\in\mathcal{M}}, \boldsymbol{d})$, extracts $\{\widetilde{\alpha^i}\}_{i\in\mathcal{M}}$ from the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 9, and computes

$$\delta := \bigoplus_{i\in\mathcal{M}} \left(\widetilde{\alpha^i} \oplus \mathsf{H}'(\boldsymbol{t}^i)\right) \in \{0,1\}^{\lambda}.$$

This modification has nothing to do with the joint distribution so that the two hybrids are perfectly indistinguishable.

- **Hybrid**$_5$. This hybrid is identical to the previous one, except that the machine changes the way to compute the messages committed by all honest parties at step 6:

  According to the public LSBs of all honest parties and the global keys of corrupted parties extracted from the emulated $\mathcal{F}_{\mathsf{aShare}}$,

  – If $\mathsf{lsb}(\bigoplus_{i\in[n]} \Delta_i) = 0$, it samples uniform $\{\widetilde{\boldsymbol{d}^j}\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 7 on behalf of all honest parties conditioned on

  $$\bigoplus_{j\in[n]\setminus\mathcal{M}} \widetilde{\boldsymbol{d}^j} = \bigoplus_{i\in\mathcal{M}} \widetilde{\boldsymbol{d}^i} \oplus \boldsymbol{q} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j).$$

  – If $\mathsf{lsb}(\bigoplus_{i\in[n]} \Delta_i) = 1$, it samples uniform $\{\widetilde{\boldsymbol{d}^j}\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 7 on behalf of all honest parties conditioned on

  $$\bigoplus_{j\in[n]\setminus\mathcal{M}} \widetilde{\boldsymbol{d}^j} = \bigoplus_{i\in\mathcal{M}} \widetilde{\boldsymbol{d}^i} \oplus \boldsymbol{a}\wedge\boldsymbol{b} \oplus \boldsymbol{r} \oplus \boldsymbol{q} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j).$$

  It follows from Equation 8 and Equation 9 that the distribution of $\bigoplus_{j\in[n]\setminus\mathcal{M}} \widetilde{\boldsymbol{d}^j}$ is identical in both two hybrids. In addition, if there are at least two honest parties, the distribution of the $\widetilde{\boldsymbol{d}^j}$ per honest party P_j is uniform in the previous hybrid due to the uniform mask $\mathsf{K}_j[\boldsymbol{r}^i]$ for another honest party P_i in $\widetilde{\boldsymbol{s}^j}$, having the same distribution as in this hybrid. Therefore, this hybrid is perfectly indistinguishable from the previous one.

- **Hybrid**$_6$. This hybrid is identical to the previous one, except that the machine changes the abort condition of all honest parties in step 10 as follows:

  – Let abort denote the event that $\mathsf{lsb}(\bigoplus_{i\in[n]} \Delta_i) = 0$ or

  $$\mathsf{H}'\left(\underbrace{\bigoplus_{j\in[n]\setminus\mathcal{M}} (\boldsymbol{a}^j \odot \boldsymbol{e}^j) \oplus \left(\boldsymbol{q} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j)\right) \cdot \bigoplus_{j\in[n]\setminus\mathcal{M}} \Delta_j}_{\text{denoted by } \boldsymbol{m}}\right) \neq \delta.$$

  The machine aborts all honest parties at step 10 if and only if the event abort happens.

We claim that, after changing the abort condition, the probability of abort at step 10 in this hybrid has a negligible difference with its counterpart in the previous hybrid. As H' is linear, the check at step 10 in the previous hybrid can be equivalently written as

$$
\begin{aligned}
0 &= \bigoplus_{i\in[n]} \widetilde{\alpha^i} \\
&= \bigoplus_{j\in[n]\setminus\mathcal{M}} \widetilde{\alpha^j} \oplus \bigoplus_{i\in\mathcal{M}} \mathsf{H}'(\boldsymbol{t}^i) \oplus \delta \\
&= \bigoplus_{j\in[n]\setminus\mathcal{M}} \mathsf{H}'\Big(\widetilde{\boldsymbol{s}^j} \oplus \widetilde{\boldsymbol{d}} \cdot \Delta_j\Big) \oplus \bigoplus_{i\in\mathcal{M}} \mathsf{H}'\Big(\boldsymbol{s}^i \oplus \boldsymbol{d} \cdot \Delta_i\Big) \oplus \delta \\
&= \bigoplus_{j\in[n]\setminus\mathcal{M}} \mathsf{H}'\Big(\boldsymbol{s}^j \oplus \boldsymbol{a}^j \odot \boldsymbol{e}^j \oplus \Big(\boldsymbol{d} \oplus \boldsymbol{q} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j)\Big) \cdot \Delta_j\Big) \\
&\qquad \oplus \bigoplus_{i\in\mathcal{M}} \mathsf{H}'\Big(\boldsymbol{s}^i \oplus \boldsymbol{d} \cdot \Delta_i\Big) \oplus \delta \qquad (10) \\
&= \mathsf{H}'\Big(\bigoplus_{i\in[n]} \boldsymbol{s}^i \oplus \boldsymbol{d} \cdot \bigoplus_{i\in[n]} \Delta_i\Big) \\
&\qquad \oplus \mathsf{H}'\Big(\bigoplus_{j\in[n]\setminus\mathcal{M}} (\boldsymbol{a}^j \odot \boldsymbol{e}^j) \oplus \Big(\boldsymbol{q} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j)\Big) \cdot \bigoplus_{j\in[n]\setminus\mathcal{M}} \Delta_j\Big) \oplus \delta \\
&= \mathsf{H}'\Big(\bigoplus_{i\in[n]} \boldsymbol{s}^i \oplus \boldsymbol{d} \cdot \bigoplus_{i\in[n]} \Delta_i\Big) \oplus \mathsf{H}'(\boldsymbol{m}) \oplus \delta \\
&= \mathsf{H}'\Big(\overline{\mathsf{lsb}(\textstyle\bigoplus_{i\in[n]} \Delta_i)} \cdot (\boldsymbol{a} \wedge \boldsymbol{b} \oplus \boldsymbol{r}) \cdot \bigoplus_{i\in[n]} \Delta_i\Big) \oplus \mathsf{H}'(\boldsymbol{m}) \oplus \delta.
\end{aligned}
$$

If $\mathsf{lsb}(\bigoplus_{i\in[n]} \Delta_i) = 1$, all honest parties behave identically for abort in both two hybrids. If $\mathsf{lsb}(\bigoplus_{i\in[n]} \Delta_i) = 0$, Equation 10 in the previous hybrid is further equivalent to

$$
\begin{aligned}
&\mathsf{H}'\left(\boldsymbol{a} \wedge \boldsymbol{b} \oplus \boldsymbol{r} \oplus \boldsymbol{q} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j)\right) \cdot \bigoplus_{j\in[n]\setminus\mathcal{M}} \Delta_j \\
&\qquad = \delta \oplus \mathsf{H}'\left(\bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \odot \boldsymbol{e}^j\right) \oplus \mathsf{H}'(\boldsymbol{a} \wedge \boldsymbol{b} \oplus \boldsymbol{r}) \cdot \bigoplus_{i\in\mathcal{M}} \Delta_i
\end{aligned} \qquad (11)
$$

where H' is applied to vectors over $\{0,1\}^{\ell_2}$ by first casting them to $(\{0,1\}^{\lambda})^{\ell_2}$.

Without loss of generality, consider there is only one honest party P_{j_0} as the adversary has a lower advantage to ensure Equation 11 if there are at least two honest parties. First, the randomness $\boldsymbol{r}$ guarantees that the left-hand hash input is zero with probability $2^{-\ell_2} \leq 2^{-\lambda}$. Second, conditioned on this input is non-zero, the hash output is zero with probability at most ε since H' is ε-almost universal. Third, conditioned on both the hash input and output are non-zero, Equation 11 can hold with probability at most $\frac{1}{2^{\lambda-1}}$.

To see the third probability, on the one hand, we move to compute the probability that the adversary successfully guesses exact $0 \leq c \leq \lambda - 1$ non-LSB bits of Δ_{j_0}. This probability upper bounds the probability that δ or $\{\boldsymbol{e}^j\}_{j\in[n]\setminus\mathcal{M}}$ can depend on c non-LSB bits of Δ_{j_0}. We stress that, in the previous hybrid, the adversary can only guess some bits of Δ_{j_0} via the selective-failure queries in the emulated $\mathcal{F}_{\mathsf{aShare}}$ as its received messages are now independent of Δ_{j_0} before step 10. Thus, this probability is exactly $\frac{1}{2^c}$. On the other hand, the remaining $\lambda - 1 - c$ non-LSB bits of Δ_{j_0} are uniform and independent of δ or $\{\boldsymbol{e}^j\}_{j\in[n]\setminus\mathcal{M}}$ on the right hand (otherwise, the adversary must successfully guess more than c non-LSB bits of Δ_{j_0}), this randomness ensures that these bits comply with Equation 11 with probability $\frac{1}{2^{\lambda-1-c}}$. Putting the two cases together, for any c, the third probability is at most $\frac{1}{2^c} \cdot \frac{1}{2^{\lambda-1-c}} = \frac{1}{2^{\lambda-1}}$.

Therefore, if $\mathsf{lsb}(\bigoplus_{i\in[n]}\Delta_i)=0$, the check at step 10 passes with probability at most $\frac{3}{2^\lambda}+\varepsilon$, while it passes with zero probability in this hybrid.

In conclusion, the two hybrids are statistically indistinguishable with an statistical distance at most $\frac{3}{2^\lambda}+\varepsilon$.

- **Hybrid**$_7$. This hybrid is identical to the previous one, except that the machine proceeds as follows if and only if $\mathsf{lsb}(\bigoplus_{i\in[n]}\Delta_i)=0$ (this event is identified by the machine at step 1):

  1. It samples uniform $\{\boldsymbol{a}^j\}_{j\in[n]\setminus\mathcal{M}}$ for all honest parties, instead of using the values from the emulated $\mathcal{F}_{\mathsf{aShare}}$.
  2. It does not compute $\{(\widetilde{\boldsymbol{s}^j},\widetilde{\boldsymbol{d}^j})\}_{j\in[n]\setminus\mathcal{M}}$ for all honest parties at step 5b but uses uniform $\{\widetilde{\boldsymbol{d}^j}\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 7 conditioned on
  $$\bigoplus_{j\in[n]\setminus\mathcal{M}}\widetilde{\boldsymbol{d}^j}=\bigoplus_{i\in\mathcal{M}}\widetilde{\boldsymbol{d}^i}\oplus\boldsymbol{q}\oplus\bigoplus_{j\in[n]\setminus\mathcal{M}}\boldsymbol{a}^j\wedge\mathsf{lsb}(\boldsymbol{e}^j).$$
  3. It computes $\boldsymbol{a}:=\bigoplus_{i\in[n]}\boldsymbol{a}^i$, where $\{\boldsymbol{a}^i\}_{i\in\mathcal{M}}$ are extracted from the emulated $\mathcal{F}_{\mathsf{aShare}}$, and samples $\boldsymbol{b},\boldsymbol{r}\xleftarrow{\$}\{0,1\}^{\ell_2}$ instead.
  4. It samples $\{\Delta_j\}_{j\in[n]\setminus\mathcal{M}}\xleftarrow{\$}\{\mathcal{R}_j\}_{j\in[n]\setminus\mathcal{M}}$ conditioned on that $\{\Delta_j\}_{j\in[n]\setminus\mathcal{M}}$ comply with the selective-failure queries in the emulated $\mathcal{F}_{\mathsf{aShare}}$.
  5. It does not compute $\{\widetilde{\boldsymbol{t}^j}\}_{j\in[n]\setminus\mathcal{M}}$ (resp., $\{\widetilde{\alpha^j}\}_{j\in[n]\setminus\mathcal{M}}$) at step 7 (resp., 9) for all honest parties, but uses uniform $\{\widetilde{\alpha^j}\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 10 conditioned on
  $$\bigoplus_{j\in[n]\setminus\mathcal{M}}\widetilde{\alpha^j}=\bigoplus_{i\in\mathcal{M}}\widetilde{\alpha^i}\oplus\mathsf{H}'\Big((\boldsymbol{a}\wedge\boldsymbol{b}\oplus\boldsymbol{r})\cdot\bigoplus_{i\in[n]}\Delta_i\Big)\oplus\mathsf{H}'(\boldsymbol{m})\oplus\delta.$$

  In this and the previous hybrid, all honest parties abort without any output in the case of $\mathsf{lsb}(\bigoplus_{i\in[n]}\Delta_i)=0$. The circumstances where the view of the adversary could differ include the selective-failure queries against $\{\Delta_j\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{aShare}}$, $\{\widetilde{\boldsymbol{d}^j}\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 7, and $\{\widetilde{\alpha^j}\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 10.

  First, the sampling of $\{\Delta_j\}_{j\in[n]\setminus\mathcal{M}}$ in this hybrid can be consistent with the selective-failure queries as in the previous hybrid. Second, the distribution of $\bigoplus_{j\in[n]\setminus\mathcal{M}}\widetilde{\boldsymbol{d}^j}$ is identical in the two hybrids due to Equation 9 and the identical distribution of $\{\boldsymbol{a}^j\}_{j\in[n]\setminus\mathcal{M}}$. If there are at least two honest parties, the $\widetilde{\boldsymbol{d}^j}:=\mathsf{lsb}(\widetilde{\boldsymbol{s}^j})$ of per honest party P_j is uniform in the previous hybrid due to the uniform mask $\mathsf{K}_j[\boldsymbol{r}^i]$ for another honest party P_i in $\widetilde{\boldsymbol{s}^j}$, having the same distribution as in this hybrid. Third, the distribution of $\bigoplus_{j\in[n]\setminus\mathcal{M}}\widetilde{\alpha^j}$ is identical in the two hybrids due to Equation 10 and the identical distribution of $\{(\boldsymbol{a}^j,\Delta_j)\}_{j\in[n]\setminus\mathcal{M}}$ and $(\boldsymbol{a},\boldsymbol{b},\boldsymbol{r})$. If there are at least two honest parties, the $\widetilde{\alpha^j}$ of per honest party P_j is uniform in previous hybrid due to the uniform zero share u^i, having the same distribution as in this hybrid.

  Putting this argument of all honest parties' outputs and the adversarial view together, we can see that these two hybrids have perfectly indistinguishable joint distribution.

- **Hybrid**$_8$. This hybrid is identical to the previous one, except that the machine proceeds as follows if and only if $\mathsf{lsb}(\bigoplus_{i\in[n]}\Delta_i)=1$ (this event is identified by the machine at step 1):

  1. It does not compute $\{(\widetilde{\boldsymbol{s}^j},\widetilde{\boldsymbol{d}^j})\}_{j\in[n]\setminus\mathcal{M}}$ for all honest parties at step 5b but uses uniform $\{\widetilde{\boldsymbol{d}^j}\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 7.

2. It does not compute $\{\widetilde{\boldsymbol{t}^j}\}_{j\in[n]\setminus\mathcal{M}}$ (resp., $\{\widetilde{\alpha^j}\}_{j\in[n]\setminus\mathcal{M}}$) at step 7 (resp., 9) for all honest parties, but proceeds as follows:

   Case 1: If the event $\mathsf{H}'(\boldsymbol{m}) \neq \delta$ is identified by the machine at step 9,

   (a) It samples uniform $\{\boldsymbol{a}^j\}_{j\in[n]\setminus\mathcal{M}}$ for all honest parties, instead of using the values from the emulated $\mathcal{F}_{\mathsf{aShare}}$.

   (b) It samples $\{\Delta_j\}_{j\in[n]\setminus\mathcal{M}} \xleftarrow{\$} \{\mathcal{R}_j\}_{j\in[n]\setminus\mathcal{M}}$ conditioned on that (i) $\{\Delta_j\}_{j\in[n]\setminus\mathcal{M}}$ comply with the selective-failure queries in the emulated $\mathcal{F}_{\mathsf{aShare}}$, and (ii) $\mathsf{H}'(\boldsymbol{m}) \oplus \delta$ from the sampled $\{(\boldsymbol{a}^j, \Delta_j)\}_{j\in[n]\setminus\mathcal{M}}$ is non-zero. Let $X \neq 0$ denote such a value.

   (c) It uses uniform $\{\widetilde{\alpha^j}\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 10 conditioned on

$$\bigoplus_{j\in[n]\setminus\mathcal{M}} \widetilde{\alpha^j} = \bigoplus_{i\in\mathcal{M}} \widetilde{\alpha^i} \oplus X.$$

   Case 2: If the event $\mathsf{H}'(\boldsymbol{m}) = \delta$ is identified by the machine at step 9,

   (a) It uses uniform $\{\widetilde{\alpha^j}\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 10 conditioned on

$$\bigoplus_{j\in[n]\setminus\mathcal{M}} \widetilde{\alpha^j} = \bigoplus_{i\in\mathcal{M}} \widetilde{\alpha^i}.$$

   (b) Instead of using the computation over authenticated sharing at step 11, it directly uses $\{\Delta_j\}_{j\in[n]\setminus\mathcal{M}}$ at step 12 to sample $\{\boldsymbol{c}^j\}_{j\in[n]\setminus\mathcal{M}}$ such that

$$\bigoplus_{i\in[n]} \boldsymbol{c}^i = \bigoplus_{i\in[n]} \boldsymbol{a}^i \wedge \bigoplus_{i\in[n]} \boldsymbol{b}^i,$$

   and generates $\langle\!\langle \boldsymbol{c} \rangle\!\rangle_j = (\boldsymbol{c}^j, \{\mathsf{M}_i[\boldsymbol{c}^j], \mathsf{K}_j[\boldsymbol{c}^i]\}_{i\neq j})$ for each $j \in [n] \setminus \mathcal{M}$ such that

$$\mathsf{K}_j[\boldsymbol{c}^i] := \begin{cases} \mathsf{M}_j[\boldsymbol{c}^i] \oplus \boldsymbol{c}^i \cdot \Delta_j, & P_i \text{ is corrupted and} \\ & \quad (\boldsymbol{c}^i, \mathsf{M}_j[\boldsymbol{c}^i]) \text{ is honestly computed from } P_i\text{'s view} \\ \text{uniform}, & P_i \text{ is honest} \end{cases}$$

$$\mathsf{M}_i[\boldsymbol{c}^j] := \begin{cases} \mathsf{K}_i[\boldsymbol{c}^j] \oplus \boldsymbol{c}^j \cdot \Delta_i, & P_i \text{ is corrupted and} \\ & \quad (\Delta_i, \mathsf{K}_i[\boldsymbol{c}^j]) \text{ is honestly computed from } P_i\text{'s view} \\ \mathsf{K}_i[\boldsymbol{c}^j] \oplus \boldsymbol{c}^j \cdot \Delta_i, & P_i \text{ is honest} \end{cases}$$

First, the event $\mathsf{lsb}(\bigoplus_{i\in[n]} \Delta_i) = 1$ implies that Equation 9 in the previous hybrid equals

$$\bigoplus_{j\in[n]\setminus\mathcal{M}} \widetilde{\boldsymbol{d}^j} = (\boldsymbol{a} \wedge \boldsymbol{b}) \oplus \boldsymbol{r} \oplus \bigoplus_{i\in\mathcal{M}} \widetilde{\boldsymbol{d}^i} \oplus \boldsymbol{q} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j),$$

which is uniform as $\boldsymbol{r}$ is uniform. In this hybrid, this sum is uniform as well. If there are at least two honest parties, the $\widetilde{\boldsymbol{d}^j} := \mathsf{lsb}(\widetilde{\boldsymbol{s}^j})$ of per honest party P_j is uniform in the previous hybrid due to the uniform mask $\mathsf{K}_j[\boldsymbol{r}^i]$ for another honest party P_i in $\widetilde{\boldsymbol{s}^j}$, having the same distribution as in this hybrid.

Then, consider **Case 1**. In this case, all honest parties abort without any output in two hybrids. Conditioned on the argued same distribution of $\{\widetilde{\boldsymbol{d}^j}\}_{j\in[n]\setminus\mathcal{M}}$, the circumstances where the view of the adversary may differ include the selective-failure queries against $\{\Delta_j\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{aShare}}$ and $\{\widetilde{\alpha^j}\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 10. On the one hand, the sampling of $\{\Delta_j\}_{j\in[n]\setminus\mathcal{M}}$ in this hybrid yields the same distribution as in the previous hybrid, i.e., respecting the selective-failure queries under the event $\mathsf{H}'(\boldsymbol{m}) \neq \delta$. On the other

hand, the distribution of $\bigoplus_{j\in[n]\setminus\mathcal{M}} \widetilde{\alpha^j}$ is identical in the two hybrids due to Equation 10 and the same distribution of $\{(\boldsymbol{a}^j, \Delta_j)\}_{j\in[n]\setminus\mathcal{M}}$. If there are at least two honest parties, the $\widetilde{\alpha^j}$ of per honest party P_j is uniform in previous hybrid due to the uniform zero share u^i, having the same distribution as in this hybrid.

Finally, consider **Case 2**. In this case, all honest parties do not abort in both two hybrids. Conditioned on the argued same distribution of $\{\widetilde{\boldsymbol{d}^j}\}_{j\in[n]\setminus\mathcal{M}}$, the circumstances where the view of the adversary may differ include $\{\widetilde{\alpha^j}\}_{j\in[n]\setminus\mathcal{M}}$ in the emulated $\mathcal{F}_{\mathsf{Com}}$ at step 10 and the joint distribution. The distributions of $\bigoplus_{j\in[n]\setminus\mathcal{M}} \widetilde{\alpha^j}$ and each $\widetilde{\alpha^j}$ are identical in the two hybrids based on a similar argument in the **Case 1**. For the joint distribution, let bad denote the event that all honest parties' outputs yield $\bigoplus_{i\in[n]} \boldsymbol{c}^i \neq \bigoplus_{i\in[n]} \boldsymbol{a}^i \wedge \bigoplus_{i\in[n]} \boldsymbol{b}^i$. As $\bigoplus_{i\in[n]} \boldsymbol{c}^i = \bigoplus_{i\in[n]} \boldsymbol{a}^i \wedge \bigoplus_{i\in[n]} \boldsymbol{b}^i \oplus \boldsymbol{q} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j)$ in the previous hybrid due to Equation 9, bad occurs in the previous hybrid if and only if $\boldsymbol{q} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j) \neq \boldsymbol{0}$. Using this equivalence and a similar argument in $\mathbf{Hybrid}_6$, we have in the previous hybrid

$$
\begin{aligned}
\Pr[\mathsf{bad}] &= \Pr\left[\mathsf{bad} \cap \left(\mathsf{H}'(\boldsymbol{m}) = \delta\right) \cap \left(\mathsf{lsb}(\textstyle\bigoplus_{i\in[n]} \Delta_i) = 1\right)\right] \\
&= \Pr\left[\mathsf{H}'(\boldsymbol{m}) = \delta \;\middle|\; \mathsf{bad} \cap \left(\mathsf{lsb}(\textstyle\bigoplus_{i\in[n]} \Delta_i) = 1\right)\right] \cdot \Pr\left[\mathsf{bad} \cap \left(\mathsf{lsb}(\textstyle\bigoplus_{i\in[n]} \Delta_i) = 1\right)\right] \\
&\leq \Pr\left[\mathsf{H}'(\boldsymbol{m}) = \delta \;\middle|\; \mathsf{bad} \cap \left(\mathsf{lsb}(\textstyle\bigoplus_{i\in[n]} \Delta_i) = 1\right)\right] \\
&= \Pr\left[\mathsf{H}'(\boldsymbol{m}) = \delta \;\middle|\; \left(\boldsymbol{q} \oplus \textstyle\bigoplus_{j\in[n]\setminus\mathcal{M}} \boldsymbol{a}^j \wedge \mathsf{lsb}(\boldsymbol{e}^j) \neq \boldsymbol{0}\right) \cap \left(\mathsf{lsb}(\textstyle\bigoplus_{i\in[n]} \Delta_i) = 1\right)\right] \\
&\leq \varepsilon + \frac{1}{2^{\lambda-1}}.
\end{aligned}
$$

That is, in the previous hybrid, all honest parties have erroneous outputs with probability at most $\varepsilon + \frac{1}{2^{\lambda-1}}$. In this hybrid, this probability is zero. Conditioned on the consistent outputs, the joint distribution in previous hybrid is identical to that in this hybrid given the uniform $\{\langle\!\langle \boldsymbol{r} \rangle\!\rangle_j\}_{j\in[n]\setminus\mathcal{M}}$ to compute the shares $\{\langle\!\langle \boldsymbol{c} \rangle\!\rangle_j\}_{j\in[n]\setminus\mathcal{M}}$. As a corollary, the statistical distance between the two hybrids in this case is at most $\varepsilon + \frac{1}{2^{\lambda-1}}$.

In summary, in either case, the two hybrids are statistically indistinguishable with advantage at most $\varepsilon + \frac{1}{2^{\lambda-1}}$.

- $\mathbf{Hybrid}_9$. This hybrid is identical to the previous one, except that the machine no longer plays the roles of all honest parties but interacts with functionality $\mathcal{F}_{\mathsf{LaAND}}$ to complete the execution. The machine in this hybrid differs from the previous one as follows:

  1. It uses the interfaces given in functionality $\mathcal{F}_{\mathsf{LaAND}}$ for the initialization, the selective-failure queries, and the global-key extraction with abort in the emulated $\mathcal{F}_{\mathsf{aShare}}$. Meanwhile, it still emulates $\mathcal{F}_{\mathsf{Zero}}$ for both honest and corrupted parties.
  2. It uses the share queries with selective failure in functionality $\mathcal{F}_{\mathsf{LaAND}}$ to possibly abort all honest parties. Specifically, it sends $(\mathsf{LeakA}, \mathtt{sid}, \mathtt{ssid}\|0, \{\boldsymbol{e}^j\}_{j\in[n]\setminus\mathcal{M}}, \boldsymbol{q})$ to $\mathcal{F}_{\mathsf{LaAND}}$ at step 7, obtains $(\mathsf{Hash}, \mathtt{sid}, \mathtt{ssid}, \mathsf{H}')$, sends $(\mathsf{Res}, \mathtt{sid}, \mathtt{ssid}, \mathsf{H}')$ in the emulated $\mathcal{F}_{\mathsf{Rand}}$, and sends $(\mathsf{LeakA}, \mathtt{sid}, \mathtt{ssid}\|1, \delta)$ to $\mathcal{F}_{\mathsf{LaAND}}$. As the machine can observe whether all honest parties abort in this interaction, it can simulate the messages sent by all honest parties as in the previous hybrid depending on this event.
  3. It delays and changes the computation of the UC-mTCCRL transcript $\mathcal{Q}_F$ to switch $\mathcal{S}_{\mathsf{H}}$ from **Phase 1** into **Phase 2**. Instead of knowing the outputs of all honest parties (in

particular, $\{\boldsymbol{b}^j\}_{j\in[n]\setminus\mathcal{M}}$) to compute $\mathcal{Q}_F$, it extracts them via the global-key queries in $\mathcal{F}_{\mathsf{LaAND}}$ and the ICM queries made by the adversary after step 12, i.e., after that the outputs of all honest parties are given to the adversary. More specifically,

- If there is only one honest party P_{j_0}, for each $i \in \mathcal{M}$ and $k \in [\ell_2]$, the machine computes every Δ'_{j_0} from both (i) every forward ICM query $E(T_{j_0,i,k}, X)$ such that $\Delta'_{j_0} := \sigma^{-1}(X) \oplus \mathsf{M}_{j_0}[a^i_k]$, and (ii) every possible $(b_k, b^{j_0}_k) \in \{0,1\}^2$ and backward ICM query $E^{-1}(T_{j_0,i,k}, X)$ such that

$$\Delta'_{j_0} := \begin{cases} \sigma^{-1}(\phi_{j_0,i,k}), & b_k = 0 \\ \sigma'^{-1}(\phi_{j_0,i,k}), & b_k = 1 \end{cases},$$
$$\phi_{j_0,i,k} := X \oplus u^{j_0,i}_k \oplus E(T_{j_0,i,k}, \sigma(\mathsf{M}_{j_0}[a^i_k])) \oplus \bigoplus\nolimits_{u\in\mathcal{M}} \left(\mathsf{M}_{j_0}[b^u_k] \oplus \mathsf{K}_u[b^{j_0}_k] \oplus b^{j_0}_k \cdot \Delta_u \oplus z^u_k\right)$$

  if the adversary has queried $E(T_{j_0,i,k}, \sigma(\mathsf{M}_{j_0}[a^i_k]))$.
- If there are at least two honest parties, for each $i \in \mathcal{M}, j \in [n] \setminus \mathcal{M}, k \in [\ell_2]$, the machine computes Δ'_j from every two distinct forward ICM queries $E(T_{j,i,k}, X)$ and $E(T_{j,i,k}, Y)$ such that $\Delta'_j := \sigma^{-1}(X \oplus Y)$.

For each computed Δ'_j for some $j \in [n] \setminus \mathcal{M}$, the machine sends $(\mathsf{Guess}, \mathtt{sid}, j, \Delta'_j)$ to $\mathcal{F}_{\mathsf{LaAND}}$. If it receives $(\mathsf{Success}, \mathtt{sid}, \{A_j\}_{j\in[n]\setminus\mathcal{M}})$ from $\mathcal{F}_{\mathsf{LaAND}}$, it uses $\{A_j\}_{j\in[n]\setminus\mathcal{M}}$ to compute $\mathcal{Q}_F$ as in the previous hybrid and switches $\mathcal{S}_{\mathsf{H}}$ from **Phase 1** into **Phase 2**; otherwise, it waits for new ICM queries to make further global-key queries.

We note that the joint distribution in the two hybrids can only be distributed differently in the ICM query-response pairs due to the changed switch of $\mathcal{S}_{\mathsf{H}}$. So, the statistical distance of the joint distribution is bounded by the statistical distance of ICM query-response pairs conditioned on other identically distributed transcripts in the joint distribution.

According to Lemma 13, the latter statistical distance is at most $\frac{q_E}{2^{\lambda-1}}$ if there is only one honest party. If there are more than two honest parties, from Lemma 14, this statistical distance is at most $\frac{3q_E}{2^{\lambda-1}}$. Putting these two cases together, we can see that this hybrid is statistically indistinguishable from the previous one with advantage at most $\frac{3q_E}{2^{\lambda-1}}$.

One can check that $\mathbf{Hybrid}_9$ is essentially the ideal-world execution where the machine plays as the simulator. Therefore, the theorem is proved. □

Lemma 12. *In the ICM, given ICM-based* $(\frac{n^2\ell_2}{4}, q_E + \frac{n^2\ell_2}{4}, n, n-1, n, \lambda-1, 1, \varepsilon_{\mathsf{H}})$*-UC-mTCCRL-secure hash function* $\mathsf{H} = \widehat{\mathsf{MMO}}^E$*,* $\mathbf{Hybrid}_2$ *and* $\mathbf{Hybrid}_3$ *in the proof of Theorem 4 are computationally indistinguishable with advantage at most* ε_{H}*.*

Proof. For contradiction, assume that there exists an environment $\mathcal{Z}$ that can distinguish the two hybrids with advantage more than ε_{H}. We can construct a distinguisher $\mathcal{D}$ to break the UC-mTCCRL security of H. Let $\mathcal{S}_{\mathsf{H}}$ denote the universal simulator in the UC-mTCCRL experiment (in Figure 7) regarding H and $\overrightarrow{\mathcal{R}} := \{\mathcal{R}_j\}_{j\in[n]\setminus\mathcal{M}}$. Each component of $\overrightarrow{\mathcal{R}}$ has min-entropy $\lambda - 1$. According to the experiment, $\mathcal{D}$ has oracle access to $(\mathcal{O}_1, \mathcal{O}_2, \mathcal{O}, \mathsf{mL}_{\mathbf{R}})$, where $\mathbf{R} \xleftarrow{\$} \overrightarrow{\mathcal{R}}$ and

$$(\mathcal{O}_1, \mathcal{O}_2, \mathcal{O}) = \begin{cases} (E, E^{-1}, \mathsf{mTCK}^{\mathsf{H}^E}_{\mathbf{R}}), & \text{if } \beta = 0 \\ (\mathcal{S}^{\mathsf{mKGuess}_{\mathbf{R}}}_{\mathsf{H}}.O_1, \mathcal{S}^{\mathsf{mKGuess}_{\mathbf{R}}}_{\mathsf{H}}.O_2, \mathbf{F}), & \text{if } \beta = 1 \end{cases}$$

It invokes $\mathcal{Z}$ in a black-box way and simulates the execution in either hybrid (depending on β) to interacts with $\mathcal{Z}$ as follows:

1. $\mathcal{D}$ emulates $\mathcal{F}_{\mathsf{aShare}}$, $\mathcal{F}_{\mathsf{Zero}}$, $\mathcal{F}_{\mathsf{Com}}$, and $\mathcal{F}_{\mathsf{Rand}}$ for the parties corrupted by $\mathcal{Z}$. In particular, it emulates the selective-failure queries in $\mathcal{F}_{\mathsf{aShare}}$ using its given $\mathsf{mL}_{\mathbf{R}}$ and aborts the emulated $\mathcal{F}_{\mathsf{aShare}}$ if the oracle $\mathsf{mL}_{\mathbf{R}}$ aborts. If $\mathcal{Z}$ asks for the global-key extraction with abort in the emulated $\mathcal{F}_{\mathsf{aShare}}$, $\mathcal{D}$ switches from **Phase 1** to **Phase 2** to obtain $\mathbf{R} = \{\Delta_j\}_{j\in[n]\setminus\mathcal{M}}$, sends $(\mathsf{Bye}, \mathtt{sid}, \mathtt{ssid}, \mathbf{R})$ to $\mathcal{Z}$, and aborts the emulated $\mathcal{F}_{\mathsf{aShare}}$.

2. $\mathcal{D}$ uses $\mathcal{O}_1$ and $\mathcal{O}_2$ to emulate ideal cipher E and its inverse E^{-1} for the parties corrupted by $\mathcal{Z}$, respectively, during the whole reduction.

3. $\mathcal{D}$ behaves as the machine in $\mathbf{Hybrid}_2$ for all honest parties except that (i) in the emulation of $\mathcal{F}_{\mathsf{aShare}}$ for all honest parties, it only samples

$$\{(\boldsymbol{a}^j, \boldsymbol{b}^j)\}_{j\in[n]\setminus\mathcal{M}}, \{\mathsf{M}_u[\boldsymbol{b}^j]\}_{j,u\in[n]\setminus\mathcal{M},u\neq j},$$

   (ii) for each $i \in \mathcal{M}$, $j \in [n] \setminus \mathcal{M}$, and $k \in [\ell_2]$, it computes

$$u_k^{j,i} := \mathcal{O}\Big(j, \underbrace{\mathsf{M}_j[a_k^i]}, T_{j,i,k}, \underbrace{\bigoplus_{i\in\mathcal{M}} b_k^i} \oplus \bigoplus_{j\in[n]\setminus\mathcal{M}} b_k^j\Big) \oplus \mathsf{H}(\underbrace{\mathsf{M}_j[a_k^i]}, T_{j,i,k})$$
$$\oplus \bigoplus_{u\in[n]\setminus\mathcal{M},u\neq j} \Big(\mathsf{M}_j[b_k^u] \oplus \mathsf{M}_u[b_k^j]\Big) \oplus z_k^j \oplus \bigoplus_{u\in\mathcal{M}} \Big(\underbrace{\mathsf{M}_j[b_k^u]} \oplus \underbrace{\mathsf{K}_u[b_k^j]} \oplus b_k^j \cdot \underbrace{\Delta_u}\Big),$$

   where the underlined terms are the randomness of corrupted parties extracted from the emulated $\mathcal{F}_{\mathsf{aShare}}$, and (iii) before sending the message of any honest party at step 5a, it switches itself from **Phase 1** to **Phase 2** to get $\mathbf{R} = \{\Delta_j\}_{j\in[n]\setminus\mathcal{M}}$, uses $\mathbf{R}$ to sample the undefined parts in the emulated $\mathcal{F}_{\mathsf{aShare}}$ for all honest parties, and honestly plays all honest parties as per protocol Π_{LaAND}.

4. $\mathcal{D}$ outputs whatever $\mathcal{Z}$ outputs.

In this construction, $\mathcal{D}$ makes at most $q_E + |\mathcal{M}| \cdot (n - |\mathcal{M}|) \cdot \ell_2 \leq q_E + \frac{n^2\ell_2}{4}$ queries to $\mathcal{O}_1$ and $\mathcal{O}_2$ since $\mathcal{Z}$ only makes at most q_E queries to E and E^{-1}, at most $|\mathcal{M}| \cdot (n - |\mathcal{M}|) \cdot \ell_2 \leq \frac{n^2\ell_2}{4}$ queries to $\mathcal{O}$, and at most $|\mathcal{M}| \leq n - 1$ queries to the $\mathsf{mL}_{\mathbf{R}}$ oracle for at most $n - |\mathcal{M}| \leq n$ distinct honest parties. That is, $\mathcal{D}$ is $(\frac{n^2\ell_2}{4}, q_E + \frac{n^2\ell_2}{4}, n, n-1)$-bounded.

When $\beta = 0$, one can check that

$$\begin{aligned} u_k^{j,i} &:= \mathsf{H}(\mathsf{M}_j[a_k^i], T_{j,i,k}) \oplus \mathsf{H}(\mathsf{M}_j[a_k^i] \oplus \Delta_j, T_{j,i,k}) \oplus b_k \cdot \Delta_j \\ &\quad \oplus \bigoplus_{u\in[n]\setminus\mathcal{M},u\neq j} \Big(\mathsf{K}_j[b_k^u] \oplus b_k^u \cdot \Delta_j \oplus \mathsf{M}_u[b_k^j]\Big) \oplus z_k^j \\ &\quad \oplus \bigoplus_{u\in\mathcal{M}} \Big(\mathsf{K}_j[b_k^u] \oplus b_k^u \cdot \Delta_j \oplus \mathsf{M}_u[b_k^j]\Big) \\ &= \mathsf{H}(\mathsf{K}_j[a_k^i], T_{j,i,k}) \oplus \mathsf{H}(\mathsf{K}_j[a_k^i] \oplus \Delta_j, T_{j,i,k}) \oplus b_k^j \cdot \Delta_j \oplus \bigoplus_{u\neq j} \Big(\mathsf{K}_j[b_k^u] \oplus \mathsf{M}_u[b_k^j]\Big) \oplus z_k^j, \end{aligned}$$

which is defined as in $\mathbf{Hybrid}_2$. Moreover, according to the identical distribution in the emulated functionalities and the fact that $\mathbf{R}$ can never be used before step 5a in $\mathbf{Hybrid}_2$, the interaction between $\mathcal{D}$ and $\mathcal{Z}$ when $\beta = 0$ yields the same joint distribution as $\mathbf{Hybrid}_2$.

When $\beta = 1$, each response from the oracle $\mathcal{O}$ is independently uniform as $\mathcal{O} = \mathbf{F}$ is a random function in this case and the tweaks are pairwise distinct. As a result, the interaction between $\mathcal{D}$ and $\mathcal{Z}$ when $\beta = 1$ yields the same joint distribution as $\mathbf{Hybrid}_3$.

Therefore, this $(\frac{n^2\ell_2}{4}, q_E + \frac{n^2\ell_2}{4}, n, n-1)$-bounded distinguisher $\mathcal{D}$ can break the UC-mTCCRL security of H with same advantage as that the q_E-bounded $\mathcal{Z}$ can distinguish the two hybrids, i.e., at least ε_{H}, leading to a contradiction and completing this proof. □

Lemma 13. *If there is only one honest party,* $\mathbf{Hybrid}_8$ *and* $\mathbf{Hybrid}_9$ *in the proof of Theorem 4 are statistically indistinguishable with advantage at most* $\frac{q_E}{2^{\lambda-1}}$ *conditioned on all fixed transcripts except the ICM query-response pairs in the joint distribution.*

Proof. Let P_{j_0} denote the only one honest party. For simplicity, we can consider a more powerful $(q_E + 2(n-1)\ell_2)$-bounded adversary, which works as the original q_E-bounded adversary but, for each $i \in \mathcal{M}$ and $k \in [\ell_2]$, additionally makes a forward ICM query $E(T_{j_0,i,k}, \sigma(\mathsf{M}_{j_0}[a_k^i]))$ before step 5a and either of the following ICM queries as soon as it learns the output of P_{j_0} at step 12:

- A forward ICM query $E(T_{j_0,i,k}, \sigma(\mathsf{M}_{j_0}[a_k^i] \oplus \Delta_{j_0}))$.
- A backward ICM query

$$E^{-1}\left(T_{j_0,i,k}, \begin{array}{c} u_k^{j_0,i} \oplus E(T_{j_0,i,k}, \sigma(\mathsf{M}_{j_0}[a_k^i])) \oplus b_k \cdot \Delta_{j_0} \oplus \sigma(\Delta_{j_0}) \\ \oplus \bigoplus_{u\in\mathcal{M}} \left(\mathsf{M}_{j_0}[b_k^u] \oplus \mathsf{K}_u[b_k^{j_0}] \oplus b_k^{j_0} \cdot \Delta_u \oplus z_k^u\right) \end{array}\right).$$

This adversary is more powerful than the original one since it can simply discard the additional $2(n-1)\ell_2$ ICM responses. Therefore, it is sufficient to bound the statistical distance against this more powerful adversary. Without loss of generality, we can assume that this adversary only makes distinct ICM queries. For each $i \in \mathcal{M}$ and $k \in [\ell_2]$, let $q_1^{i,k}, q_2^{i,k}, q_3^{i,k}$ respectively be the number of distinct ICM queries (except the additional ones) with $\mathsf{idx} = T_{j_0,i,k}$ before step 5a, after step 5a but before step 12, and after step 12. We have $q_E \geq \sum_{i\in\mathcal{M},k\in[\ell_2]}(q_1^{i,k} + q_2^{i,k} + q_3^{i,k})$.

We use the H-coefficient technique to bound the statistical distance. Let $\Pr[\cdot]$ (resp., $\Pr'[\cdot]$) be the probability measure in $\mathbf{Hybrid}_8$ (resp., $\mathbf{Hybrid}_9$). A list of ICM query-response pairs is bad if and only if, in this list,

- For some $i \in \mathcal{M}$ and $k \in [\ell_2]$, there exists a forward ICM query $E(T_{j_0,i,k}, \sigma(\mathsf{M}_{j_0}[a_k^i] \oplus \Delta_{j_0}))$ or there exists a backward ICM query (after the forward ICM query $E(T_{j_0,i,k}, \sigma(\mathsf{M}_{j_0}[a_k^i]))$)

$$E^{-1}\left(T_{j_0,i,k}, \begin{array}{c} u_k^{j_0,i} \oplus E(T_{j_0,i,k}, \sigma(\mathsf{M}_{j_0}[a_k^i])) \oplus b_k \cdot \Delta_{j_0} \oplus \sigma(\Delta_{j_0}) \\ \oplus \bigoplus_{u\in\mathcal{M}} \left(\mathsf{M}_{j_0}[b_k^u] \oplus \mathsf{K}_u[b_k^{j_0}] \oplus b_k^{j_0} \cdot \Delta_u \oplus z_k^u\right) \end{array}\right)$$

  before step 12.

Let bad denote the event that a list of ICM query-response pairs is bad. From the randomness of Δ_{j_0} and the fact that $\sigma'(x) := \sigma(x) \oplus x$ is a permutation, we have

$$\Pr'[\mathsf{bad}] \leq \frac{|\cap_{i\in\mathcal{M}} L_{i,j_0}|}{2^{\lambda-1}} \cdot \frac{\sum_{i\in\mathcal{M},k\in[\ell_2]}(q_1^{i,k} + q_2^{i,k})}{|\cap_{i\in\mathcal{M}} L_{i,j_0}|} \leq \frac{q_E}{2^{\lambda-1}},$$

where $L_{i,j_0} \in \mathcal{L}$ is the selective-failure query made by the adversary of behalf of corrupted P_i and against honest P_{j_0}.

For any fixed good list τ of ICM query-response pairs, we define $\mathsf{good}(\tau)$ as the event that the interaction in a hybrid produces τ. In $\mathbf{Hybrid}_8$, it follows from the switch at step 5a that

$$\Pr[\mathsf{good}(\tau)] = \prod_{i\in\mathcal{M},k\in[\ell_2]} \frac{1}{(2^\lambda)_{q_1^{i,k}+1}} \cdot \frac{1}{(2^\lambda - (q_1^{i,k}+1) - \underbrace{1}_{\substack{\text{the only one ICM entry}\\ \text{fixed by one pair in } \mathcal{Q}_F\\ \text{indexed by } (j_0,i,k)}})_{q_2^{i,k}+q_3^{i,k}}}$$
$$= \prod_{i\in\mathcal{M},k\in[\ell_2]} \frac{2^\lambda - (q_1^{i,k}+1)}{(2^\lambda)_{q_1^{i,k}+q_2^{i,k}+q_3^{i,k}+2}},$$

where the probability is taken over the randomness of ICM.

In $\mathbf{Hybrid}_9$, the extraction step of the machine ensure that it can always compute a correct $\Delta'_{j_0} = \Delta_{j_0}$ from the additional ICM queries and then obtain the output of honest party P_{j_0} from the global-key queries to $\mathcal{F}_{\mathsf{LaAND}}$ at step 12. Therefore, the machine can compute the same[4] $\mathcal{Q}_F$ at step 12 and switch $\mathcal{S}_{\mathsf{H}}$ from **Phase 1** into **Phase 2**. The delayed switch leads to

$$\Pr'[\mathsf{good}(\tau)] = \prod_{i\in\mathcal{M},k\in[\ell_2]} \frac{1}{(2^\lambda)_{q_1^{i,k}+q_2^{i,k}+1}} \cdot \frac{1}{(2^\lambda - (q_1^{i,k}+q_2^{i,k}+1) - \underbrace{1}_{\substack{\text{the only one ICM entry}\\ \text{fixed by one pair in } \mathcal{Q}_F\\ \text{indexed by } (j_0,i,k)}})_{q_3^{i,k}}}$$
$$= \prod_{i\in\mathcal{M},k\in[\ell_2]} \frac{2^\lambda - (q_1^{i,k}+q_2^{i,k}+1)}{(2^\lambda)_{q_1^{i,k}+q_2^{i,k}+q_3^{i,k}+2}}$$

according to the randomness of ICM. It is clear that

$$\frac{\Pr[\mathsf{good}(\tau)]}{\Pr'[\mathsf{good}(\tau)]} = \prod_{i\in\mathcal{M},k\in[\ell_2]} \frac{2^\lambda - (q_1^{i,k}+1)}{2^\lambda - (q_1^{i,k}+q_2^{i,k}+1)} \geq 1.$$

From the H-coefficient technique, the statistical distance between the list of ICM query-response pairs in $\mathbf{Hybrid}_8$ and $\mathbf{Hybrid}_9$ is at most $\frac{q_E}{2^{\lambda-1}}$ against the more powerful adversary, leading to an upper bound against the original adversary. □

Lemma 14. *If there are at least two honest parties,* $\mathbf{Hybrid}_8$ *and* $\mathbf{Hybrid}_9$ *in the proof of Theorem 4 are statistically indistinguishable with advantage at most* $\frac{3q_E}{2^{\lambda-1}}$ *conditioned on all fixed transcripts except the ICM query-response pairs in the joint distribution.*

Proof. For simplicity, we consider a more powerful $(q_E + 2(n-|\mathcal{M}|)\cdot|\mathcal{M}|\cdot\ell_2)$-bounded adversary, which works as the original q_E-bounded adversary but, for each $i \in \mathcal{M}, j \in [n]\setminus\mathcal{M}, k \in [\ell_2]$, additionally makes a forward ICM query $E(T_{j,i,k}, \sigma(\mathsf{M}_j[a_k^i]))$ before step 5a and a forward ICM query $E(T_{j,i,k}, \sigma(\mathsf{M}_j[a_k^i] \oplus \Delta_j))$ as soon as it learns the output of P_j at step 12.

[4]This is because, when there is only one honest party, the randomness in $\mathcal{Q}_F$ is fixed by the additional ICM response $E(T_{j_0,i,k}, \sigma(\mathsf{M}_{j_0}[a_k^i]))$ fixed in τ and the fixed other transcripts in the joint distribution.

This adversary is more powerful than the original one as it can simply discard the additional $2(n-|\mathcal{M}|)\cdot|\mathcal{M}|\cdot\ell_2$ ICM responses. So, it is sufficient to bound the statistical distance against this more powerful adversary. We assume that this adversary only makes distinct ICM queries. For each $i\in\mathcal{M}, j\in[n]\setminus\mathcal{M}, k\in[\ell_2]$, let $q_1^{j,i,k}, q_2^{j,i,k}, q_3^{j,i,k}$ respectively be the number of distinct ICM queries (except the additional ones) with $\mathsf{idx}=T_{j,i,k}$ before step 5a, after step 5a but before step 12, and after step 12. We have $q_E \geq \sum_{i\in\mathcal{M},j\in[n]\setminus\mathcal{M},k\in[\ell_2]}(q_1^{j,i,k}+q_2^{j,i,k}+q_3^{j,i,k})$.

We use the H-coefficient technique to bound the statistical distance. Let $\Pr[\cdot]$ (resp., $\Pr'[\cdot]$) be the probability measure in $\mathbf{Hybrid}_8$ (resp., $\mathbf{Hybrid}_9$). A list of ICM query-response pairs is bad if and only if, in this list, at least one of the following events holds:

- $\mathsf{B1}$. For some $i\in\mathcal{M}, j\in[n]\setminus\mathcal{M}, k\in[\ell_2]$, there exists a forward ICM query
$$E(T_{j,i,k}, \sigma(\mathsf{M}_j[a_k^i]\oplus\Delta_j))$$
before step 12.
- $\mathsf{B2}$. For some $i\in\mathcal{M}, j\in[n]\setminus\mathcal{M}, k\in[\ell_2]$, there exists a backward ICM query (after the forward ICM query $E(T_{j,i,k},\sigma(\mathsf{M}_j[a_k^i]))$)
$$E^{-1}\left(T_{j,i,k}, \begin{array}{c} u_k^{j,i}\oplus E(T_{j,i,k},\sigma(\mathsf{M}_j[a_k^i]))\oplus b_k\cdot\Delta_j\oplus\sigma(\Delta_j) \\ \oplus\bigoplus_{u\in[n]\setminus\mathcal{M},u\neq j}\left(\mathsf{M}_j[b_k^u]\oplus\mathsf{M}_u[b_k^j]\right)\oplus z_k^j \\ \oplus\bigoplus_{u\in\mathcal{M}}\left(\mathsf{M}_j[b_k^u]\oplus\mathsf{K}_u[b_k^j]\oplus b_k^j\cdot\Delta_u\right)\end{array}\right)$$
during the whole execution.

Let $\mathsf{bad} := \mathsf{B1}\wedge\mathsf{B2}$ denote the event that a list of ICM query-response pairs is bad. [Cui: $\mathsf{bad} := \mathsf{B1}\vee\mathsf{B2}$?] It follows from the randomness of per Δ_j and a union bound that
$$\Pr'[\mathsf{B1}] \leq \sum_{j\in[n]\setminus\mathcal{M}} \frac{|\cap_{i\in\mathcal{M}}L_{i,j}|}{2^{\lambda-1}}\cdot\frac{\sum_{i\in\mathcal{M},k\in[\ell_2]}(q_1^{j,i,k}+q_2^{j,i,k})}{|\cap_{i\in\mathcal{M}}L_{i,j}|} \leq \frac{q_E}{2^{\lambda-1}},$$
where $L_{i,j}\in\mathcal{L}$ is the selective-failure query made by the adversary of behalf of corrupted P_i and against honest P_j. Meanwhile, the uniformness of per zero share z_k^j ensures that
$$\Pr'[\mathsf{B2}] \leq \frac{\sum_{i\in\mathcal{M},j\in[n]\setminus\mathcal{M},k\in[\ell_2]}(q_1^{j,i,k}+q_2^{j,i,k}+q_3^{j,i,k})}{2^\lambda} \leq \frac{q_E}{2^\lambda}.$$
Therefore, we have $\Pr'[\mathsf{bad}]\leq\Pr'[\mathsf{B1}]+\Pr'[\mathsf{B2}]\leq\frac{3q_E}{2^\lambda}$.

For any fixed good list τ of ICM query-response pairs, we define $\mathsf{good}(\tau)$ as the event that the interaction in a hybrid produces τ. In $\mathbf{Hybrid}_8$, it follows from the switch at step 5a that
$$\begin{aligned}\Pr[\mathsf{good}(\tau)] &= \prod_{i\in\mathcal{M},j\in[n]\setminus\mathcal{M},k\in[\ell_2]} \frac{1}{(2^\lambda)_{q_1^{j,i,k}+1}} \\ &\quad\cdot\frac{1}{(2^\lambda-(q_1^{j,i,k}+1)-\underbrace{1}_{\substack{\text{the only one ICM entry}\\ \text{fixed by one pair in } \mathcal{Q}_F\\ \text{indexed by } (j,i,k)}})_{q_2^{j,i,k}+q_3^{j,i,k}}} \\ &= \prod_{i\in\mathcal{M},j\in[n]\setminus\mathcal{M},k\in[\ell_2]} \frac{2^\lambda-(q_1^{j,i,k}+1)}{(2^\lambda)_{q_1^{j,i,k}+q_2^{j,i,k}+q_3^{j,i,k}+2}},\end{aligned}$$

where the probability is taken over the randomness of ICM.

In $\mathbf{Hybrid}_9$, the extraction step of the machine ensure that it can always compute a correct $\Delta'_j = \Delta_j$ from the additional ICM queries and then obtain the output of all honest parties from the global-key queries to $\mathcal{F}_{\mathsf{LaAND}}$ at step 12. Additionally conditioned on the identically distributed zero shares $\{z_k^j\}_{j\in[n]\setminus\mathcal{M}}$, the machine can compute the same[5] $\mathcal{Q}_F$ at step 12 and switch $\mathcal{S}_{\mathsf{H}}$ from **Phase 1** into **Phase 2**. The delayed switch leads to

$$\begin{aligned}\Pr'[\mathsf{good}(\tau)] &= \prod_{i\in\mathcal{M},j\in[n]\setminus\mathcal{M},k\in[\ell_2]} \frac{1}{(2^\lambda)_{q_1^{j,i,k}+q_2^{j,i,k}+1}} \\ &\qquad \cdot \frac{1}{(2^\lambda - (q_1^{j,i,k}+q_2^{j,i,k}+1) - \underbrace{1}_{\substack{\text{the only one ICM entry}\\ \text{fixed by one pair in } \mathcal{Q}_F\\ \text{indexed by } (j,i,k)}})_{q_3^{j,i,k}}} \\ &= \prod_{i\in\mathcal{M},j\in[n]\setminus\mathcal{M},k\in[\ell_2]} \frac{2^\lambda - (q_1^{j,i,k}+q_2^{j,i,k}+1)}{(2^\lambda)_{q_1^{j,i,k}+q_2^{j,i,k}+q_3^{j,i,k}+2}},\end{aligned}$$

according to the randomness of ICM. Therefore, we have

$$\frac{\Pr[\mathsf{good}(\tau)]}{\Pr'[\mathsf{good}(\tau)]} = \prod_{i\in\mathcal{M},j\in[n]\setminus\mathcal{M},k\in[\ell_2]} \frac{2^\lambda - (q_1^{j,i,k}+1)}{2^\lambda - (q_1^{j,i,k}+q_2^{j,i,k}+1)} \geq 1.$$

From the H-coefficient technique, the statistical distance between the list of ICM query-response pairs in $\mathbf{Hybrid}_8$ and $\mathbf{Hybrid}_9$ is at most $\frac{3q_E}{2^{\lambda-1}}$ against the more powerful adversary, leading to an upper bound against the original adversary. □

9.4 Authenticated AND Triples

Finally, we employ the bucketing technique to remove the leakage on the x elements of the triples generated by $\mathcal{F}_{\mathsf{LaAND}}$, realizing the authenticated AND triple functionality $\mathcal{F}_{\mathsf{aAND}}$ in Figure **??**. Here we use the rotation-based bucketing to reduce the cache-miss probability which greatly affects the overall running time.

Theorem 5. *The protocol Π_{aAND} in Figure 16 securely realizes functionality $\mathcal{F}_{\mathsf{aAND}}$ in Figure **??** with statistical error $\max(\frac{1}{2^\rho} + \frac{2B+1}{\ell^{B-1}}, \frac{1}{2^\rho} + \frac{1}{2^{\lambda-\rho}}, \frac{1}{2^{B\ell}})$ in the $\mathcal{F}_{\mathsf{LaAND}}$-hybrid model.*

Before proving the theorem we recall a simple lemma on the probability of a random matrix being full-rank.

Claim 1. *Let $\boldsymbol{A} :=_{\$} \mathbb{F}_2^{n\times(n+\rho)}$ be a uniformly random binary matrix, where $n, \rho \in \mathbb{N}$. Then the probability that $\boldsymbol{A}$ being full rank is at least $\frac{1}{2^\rho}$.*

[5] When there are at least two honest parties, the randomness in $\mathcal{Q}_F$ is fixed by the fixed zero shares $\{z_k^j\}_{j\in[n]\setminus\mathcal{M}}$, the additional ICM response $E(T_{j,i,k}, \sigma(\mathsf{M}_j[a_k^i]))$ fixed in τ, and the fixed other transcripts in the joint distribution.

The claim follows since let $m = n + \rho$, we have

$$\begin{aligned}\Pr[\mathrm{rank}(\boldsymbol{A}) = n] &= \frac{2^m - 2^0}{2^m} \cdot \frac{2^m - 2^1}{2^m} \cdot \ldots \cdot \frac{2^m - 2^{n-1}}{2^m} \\ &= \prod_{i \in [0, n-1)} (1 - \frac{2^i}{2^m}) \\ &\geq 1 - \frac{2^n}{2^m} \\ &= 1 - \frac{1}{2^\rho}.\end{aligned}$$

Proof of Theorem 5. The only interaction among the parties in Π_{aAND} is the Open command in Step **??**. Also since there is no leakage in the y component in the $\mathcal{F}_{\mathsf{LaAND}}$ protocol, therefore, the simulator can simply use uniformly random d values to simulate the open command. The statistical distance between the real world and the ideal world captures the probability that the adversary learns information about x in the real world.

Recall that in $\mathcal{F}_{\mathsf{LaAND}}$, if the event $\mathtt{bad\text{-}key}$ occurs then for each leaky triple generated there is a $\frac{1}{2}$ probability of abort. Therefore, we do not consider this case since both in two worlds abort happens except with probability $\frac{1}{2^{B\ell}}$.

Now we only consider Type 2 queries. Notice that in $\mathcal{F}_{\mathsf{LaAND}}$ the functionality aborts if $Q \neq \bigoplus_{i \in \mathcal{H}, t \in [\ell]} \chi_t \cdot R_{t,i} \cdot x_t^i$. Consider the following two cases.

- $\ell \geq \lambda - \rho$. By Claim 1 there exists a $(\lambda - \rho)$ sized subset of the set $\{\chi_t\}_{t \in [\ell], \exists i \in \mathcal{H}, R_{t,i} \neq 0}$ which is linearly independent when viewing its elements as vectors in $\mathbb{F}_2^\lambda$ except with probability $2^{-\lambda}$.
- $\ell < \lambda - \rho$. In this case we have the set $\{\chi_t\}_{t \in [\ell], \exists i \in \mathcal{H}, R_{t,i} \neq 0}$ itself is linearly independent when viewing its elements as vectors in $\mathbb{F}_2^\lambda$ except with probability $2^{-\rho}$.

In the first case, the value $\bigoplus_{i \in \mathcal{H}, t \in [\ell]} \chi_t \cdot R_{t,i} \cdot x_t^i$ has at least $(\lambda - \rho)$ bits of entropy, making the abort probability at least $\frac{1}{2^\rho} + \frac{1}{2^{\lambda - \rho}}$.

In the second case, each $t \in [\ell], R_{t,i} \neq 0$ contributes to $\frac{1}{2}$ probability of abort. The leakage happens if the corresponding bucket only contains leaky triples. Following the proof technique of TinyOT [39], we first fix the index of the bucket and the set of leaky triples that fill the bucket, and then bound the overall probability via a union bound.

Let γ be the total number of leaky triples, i be the bucket index, and S be the set of triples that got mapped to the i-th bucket via rotation . The probability that all of elements in S mapped to the i-th bucket is $\frac{1}{\ell^{B-1}}$ since the first block is not rotated. In total, there are $\min(\ell, \gamma)$ number of indices and the number of S is bounded by $\binom{\gamma}{B}$. Therefore, let Leak be the event that the $\mathcal{A}$ successfully gets a leaky AND triple, we have that

$$\Pr[\mathsf{Leak}] \leq \frac{1}{2^\gamma} \cdot \binom{\gamma}{B} \cdot \gamma \cdot \frac{1}{\ell^{B-1}}.$$

Let the right-hand term be $\alpha(\ell, B, \gamma)$, we would like to find the maximum of $\alpha(\ell, B, \gamma)$ with respect to γ. Notice that

$$\frac{\alpha(\ell, B, \gamma + 1)}{\alpha(\ell, B, \gamma)} = \frac{\frac{1}{2^{\gamma+1}} \cdot \binom{\gamma+1}{B} \cdot (\gamma + 1) \cdot \frac{1}{\ell^{B-1}}}{\frac{1}{2^\gamma} \cdot \binom{\gamma}{B} \cdot \gamma \cdot \frac{1}{\ell^{B-1}}} = \frac{(\gamma + 1)^2}{2\gamma \cdot (\gamma - B + 1)}.$$

Protocol Π_{Prep}

Initialization: All parties initialize their global keys in $\mathcal{F}_{\mathsf{LaAND}}$.

1. For each $i \in [n]$, P_i sends $(\mathsf{Init}, \mathtt{sid}, i)$ to $\mathcal{F}_{\mathsf{LaAND}}$.

Authenticated shares & Authenticated AND triples: All parties generate ℓ_1 random authenticated shares and $\ell := B \cdot \ell_2$ random leaky authenticated AND triples, where $B \in \mathbb{N}^+$ is for bucketing.

2. All parties send $(\mathsf{Gen}, \mathtt{sid}, \mathtt{ssid}, \ell_1, \ell)$ to $\mathcal{F}_{\mathsf{LaAND}}$, which outputs

$$(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, \Delta_i, \langle\langle \boldsymbol{x} \rangle\rangle_i, \langle\langle \boldsymbol{a}' \rangle\rangle_i, \langle\langle \boldsymbol{b}' \rangle\rangle_i, \langle\langle \boldsymbol{c}' \rangle\rangle_i)$$

to P_i for each $i \in [n]$, where $\boldsymbol{x} \in \{0,1\}^{\ell_1}$ and $\boldsymbol{a}', \boldsymbol{b}', \boldsymbol{c}' \in \{0,1\}^{\ell}$.

All parties eliminate the leakage on $\langle\langle \boldsymbol{a}' \rangle\rangle$ via bucketing.

3. All parties rearrange all $\ell = B \cdot \ell_2$ leaky triples into a B-by-ℓ_2 matrix. For each $k \in [2, B]$, all parties send $(\mathsf{Rand}, \mathtt{sid}, \mathtt{ssid}, [\ell_2])$ to $\mathcal{F}_{\mathsf{Rand}}$, which outputs $(\mathsf{Res}, \mathtt{sid}, \mathtt{ssid}, r_k)$ to all parties, and perform a circular right shift on the ℓ_2 leaky triples of the k-th row by step size r_k.

   For each $j \in [\ell_2]$, the j-th column of the matrix is the j-th bucket of B leaky triples.

4. For each $j \in [\ell_2]$, all parties combine the B leaky triples in the j-th bucket into a non-leaky one. Let $(\langle\langle a_j \rangle\rangle, \langle\langle b_j \rangle\rangle, \langle\langle c_j \rangle\rangle)$ register this resulting non-leaky triple and be initialized as the first leaky triple in the j-th bucket. Then, for each next leaky triple $(\langle\langle a^* \rangle\rangle, \langle\langle b^* \rangle\rangle, \langle\langle c^* \rangle\rangle)$ in the j-th bucket,

   (a) All parties invoke macro $d := \mathsf{Open}(\mathtt{sid}, \mathtt{ssid}, \langle\langle b_j \rangle\rangle \oplus \langle\langle b^* \rangle\rangle) \in \{0,1\}$.

   (b) All parties update $\langle\langle a_j \rangle\rangle := \langle\langle a_j \rangle\rangle \oplus \langle\langle a^* \rangle\rangle$ and $\langle\langle c_j \rangle\rangle := \langle\langle c_j \rangle\rangle \oplus \langle\langle c^* \rangle\rangle \oplus d \cdot \langle\langle a^* \rangle\rangle$.

5. For each $i \in [n]$, P_i outputs $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, \Delta_i, \langle\langle \boldsymbol{x} \rangle\rangle_i, \langle\langle \boldsymbol{a} \rangle\rangle_i, \langle\langle \boldsymbol{b} \rangle\rangle_i, \langle\langle \boldsymbol{c} \rangle\rangle_i)$.

Figure 16: The protocol of multi-party preprocessing.

Let $\frac{(\gamma+1)^2}{2\gamma \cdot (\gamma - B + 1)} > 1$, we have

$$\begin{aligned} 2\gamma \cdot (\gamma - B + 1) &< (\gamma + 1)^2 \\ \gamma^2 - 2B\gamma + B^2 &< B^2 + 1. \end{aligned}$$

Therefore, $\frac{(\gamma+1)^2}{2\gamma \cdot (\gamma - B + 1)} > 1$ is equivalent to $B - \sqrt{B^2+1} < \gamma < B + \sqrt{B^2+1}$, which implies the maximum of $\alpha(\ell, B, \gamma)$ is $\alpha'(\ell, B) = \alpha(\ell, B, 2B+1)$.

Notice that

$$\begin{aligned} \alpha'(\ell, B) &= \frac{1}{2^{2B+1}} \cdot \binom{2B+1}{B} \cdot (2B+1) \cdot \frac{1}{\ell^{B-1}} \\ &\leq \frac{2B+1}{\ell^{B-1}}, \end{aligned}$$

which is the desired bound. □

10 Authenticated Garbling

In this section, we introduce the multi-party authenticated garbling protocol as in [44]. We begin with the multiparty computation functionality in Figure 17

Functionality $\mathcal{F}_{\mathsf{MPC}}$

This functionality runs with parties $P_1, \ldots, P_n$.

Parameter: A Boolean circuit $f : \{0,1\}^{|\mathcal{I}_1|} \times \cdots \times \{0,1\}^{|\mathcal{I}_n|} \to \{0,1\}^{|\mathcal{O}_1|} \times \cdots \times \{0,1\}^{|\mathcal{O}_n|}$, where (i) $|f| \geq \lambda$ and (ii) $\mathcal{I}_i$ (resp., $\mathcal{O}_i$) is the set of circuit-input (resp., -output) wires of P_i for each $i \in [n]$.

Input: Upon receiving $(\mathsf{Input}, \mathtt{sid}, \mathtt{ssid}, i, \boldsymbol{x}^i)$ from P_i, where $\boldsymbol{x}^i \in \{0,1\}^{|\mathcal{I}_i|}$, store $(\mathtt{sid}, \mathtt{ssid} \| 0, i, \boldsymbol{x}^i)$ and ignore all subsequent $(\mathsf{Input}, \mathtt{sid}, \mathtt{ssid}, i, \ldots)$ calls with the same $(\mathtt{sid}, \mathtt{ssid}, i)$.

Eval: Given stored $(\mathtt{sid}, \mathtt{ssid} \| 0, i, \boldsymbol{x}^i)$ for each $i \in [n]$, this functionality computes f: Upon receiving $(\mathsf{Eval}, \mathtt{sid}, \mathtt{ssid})$ from all parties, compute $(\boldsymbol{y}^1, \ldots, \boldsymbol{y}^n) := f(\boldsymbol{x}^1, \ldots, \boldsymbol{x}^n)$, store $(\mathtt{sid}, \mathtt{ssid} \| 1, i, \boldsymbol{y}^i)$ for each $i \in [n]$, and ignore all subsequent $(\mathsf{Eval}, \mathtt{sid}, \mathtt{ssid})$ calls with the same $(\mathtt{sid}, \mathtt{ssid})$.

Output: Given stored $(\mathtt{sid}, \mathtt{ssid} \| 1, i, \boldsymbol{y}^i)$ for each $i \in [n]$, this functionality outputs all results: Upon receiving $(\mathsf{Output}, \mathtt{sid}, \mathtt{ssid})$ from all parties, output $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, \boldsymbol{y}^i)$ to P_i for each $i \in [n]$.

Figure 17: The functionality of multi-party computation.

10.1 Multiparty Authenticated Garbling

We then present the protocol that implements $\mathcal{F}_{\mathsf{MPC}}$ in the $\mathcal{F}_{\mathsf{Prep}}$-hybrid model. Recall that in the preprocessing functionality (Figure **??**), the returned global key satisfies that $\mathsf{lsb}(\Delta_i) = 1$ for $i \neq 1$. This fact comes in handy when using the half-gate technique since it allows the evaluator (P_1) to extract the masked wire value from the least significant bits from the wire labels.

We introduce the procedures for function independent processing in Figure **??** and function-dependent processing in Figure **??**. The online procedures is given in Figure **??**.

Remark 2. *We note that in Step* **??** *an authenticated share $\langle\langle \lambda_w^i \rangle\rangle$ is prepared for each input wire $w \in \mathcal{I}_i, i \in [n]$ while in [44] the share $\lambda_{w,j}^i$ for $j \neq i$ is zero. The change is necessary so that the active path is independent of the actual input values in the view of the adversary, which allows us to simulate an indistinguishable garbled circuit ciphertexts using the* muTCCRL *property of the hash function. In [44] the security is proved under the random oracle model so this requirement becomes irrelevant.*

10.2 Security Analysis

In this subsection, we analyze the security of the protocol in Figure **??**, Figure **??**, and Figure **??**. We follow the approach in [44] by separately proving lemmas regarding the security properties of the garbling scheme, and then combining them together into a main theorem.

References

[1] Elena Andreeva, Andrey Bogdanov, Yevgeniy Dodis, Bart Mennink, and John P. Steinberger. On the indifferentiability of key-alternating ciphers. In Ran Canetti and Juan A. Garay, editors, CRYPTO 2013, Part I, volume 8042 of LNCS, pages 531–550, Santa Barbara, CA, USA, August 18–22, 2013. Springer, Heidelberg, Germany.

[2] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. LNCS, pages 581–615, Santa Barbara, CA, USA, August 2023. Springer, Heidelberg, Germany.

Protocol Π_{MPC}

Input: All parties require a circuit-dependent preprocessing. Let $\mathcal{W}$ be the set of the output wires of AND gates, $\mathcal{I} := \cup_{i\in[n]}\mathcal{I}_i$ be the set of circuit-input wires, and $\mathsf{H} : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda$ be an muTCCRL secure hash function. Note that $|\mathcal{W}| = |f| \geq \lambda$.

1. For each $i \in [n]$, P_i sends $(\mathsf{Init}, \mathtt{sid}, i)$ to $\mathcal{F}_{\mathsf{Prep}}$.
2. All parties send $(\mathsf{Gen}, \mathtt{sid}, \mathtt{ssid}, |\mathcal{W}| + |\mathcal{I}|, |\mathcal{W}|)$ to $\mathcal{F}_{\mathsf{Prep}}$, which outputs
$$(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, \Delta_i, \langle\langle \boldsymbol{\lambda}\rangle\rangle_i, \langle\langle \boldsymbol{a}\rangle\rangle_i, \langle\langle \boldsymbol{b}\rangle\rangle_i, \langle\langle \boldsymbol{c}\rangle\rangle_i)$$
to P_i for each $i \in [n]$, where $\boldsymbol{\lambda} \in \{0,1\}^{|\mathcal{W}|+|\mathcal{I}|}$ and $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c} \in \{0,1\}^{|\mathcal{W}|}$.
Let all parties use wires as the unique subscripts of vectors (e.g., $\boldsymbol{\lambda}$, $\boldsymbol{a}$, $\boldsymbol{b}$, $\boldsymbol{c}$, $\boldsymbol{x}^i$, and $\boldsymbol{y}^i$).
3. For each $i \in [2,n]$ and each $w \in \mathcal{I}$, P_i samples $\mathsf{L}^i_{w,0} \xleftarrow{\$} \{0,1\}^\lambda$.
4. For each XOR gate $(\alpha, \beta, \gamma, \oplus)$ in a topology order,
   (a) All parties compute $\langle\langle \lambda_\gamma\rangle\rangle := \langle\langle \lambda_\alpha\rangle\rangle \oplus \langle\langle \lambda_\beta\rangle\rangle$.
   (b) For each $i \in [2,n]$, P_i computes $\mathsf{L}^i_{\gamma,0} := \mathsf{L}^i_{\alpha,0} \oplus \mathsf{L}^i_{\beta,0} \in \{0,1\}^\lambda$.
5. For each AND gate $(\alpha, \beta, \gamma, \wedge)$ in parallel,
   (a) All parties compute $\langle\langle d_\gamma\rangle\rangle := \langle\langle \lambda_\alpha\rangle\rangle \oplus \langle\langle a_\gamma\rangle\rangle$ and $\langle\langle e_\gamma\rangle\rangle := \langle\langle \lambda_\beta\rangle\rangle \oplus \langle\langle b_\gamma\rangle\rangle$.
   (b) All parties invoke macro $(d_\gamma, e_\gamma) := \mathsf{Open}(\mathtt{sid}, \mathtt{ssid}, (\langle\langle d_\gamma\rangle\rangle, \langle\langle e_\gamma\rangle\rangle)) \in \{0,1\}^2$.
   (c) All parties compute $\langle\langle \lambda_{\alpha\beta}\rangle\rangle = \langle\langle \lambda_\alpha \cdot \lambda_\beta\rangle\rangle := \langle\langle c_\gamma\rangle\rangle \oplus d_\gamma \cdot \langle\langle b_\gamma\rangle\rangle \oplus e_\gamma \cdot \langle\langle a_\gamma\rangle\rangle \oplus d_\gamma \cdot e_\gamma$.
6. For each $i \in [2,n]$ and each AND gate $(\alpha, \beta, \gamma, \wedge)$ in parallel, P_i computes $\mathsf{L}^i_{\alpha,1} := \mathsf{L}^i_{\alpha,0} \oplus \Delta_i \in \{0,1\}^\lambda$, $\mathsf{L}^i_{\beta,1} := \mathsf{L}^i_{\beta,0} \oplus \Delta_i \in \{0,1\}^\lambda$, and
$$\begin{aligned}
T_{\gamma,i,i} &:= \mathtt{sid}\|\mathtt{ssid}\|\gamma\|i\|i \in \{0,1\}^\lambda,\\
G^i_{\gamma,0} &:= \mathsf{H}(\mathsf{L}^i_{\alpha,0}, T_{\gamma,i,i}) \oplus \mathsf{H}(\mathsf{L}^i_{\alpha,1}, T_{\gamma,i,i}) \oplus (\textstyle\bigoplus_{j\neq i} \mathsf{K}_i[\lambda^j_\beta]) \oplus \lambda^i_\beta \cdot \Delta_i \in \{0,1\}^\lambda,\\
G^i_{\gamma,1} &:= \mathsf{H}(\mathsf{L}^i_{\beta,0}, T_{\gamma,i,i}) \oplus \mathsf{H}(\mathsf{L}^i_{\beta,1}, T_{\gamma,i,i}) \oplus \mathsf{L}^i_{\alpha,0} \oplus (\textstyle\bigoplus_{j\neq i} \mathsf{K}_i[\lambda^j_\alpha]) \oplus \lambda^i_\alpha \cdot \Delta_i \in \{0,1\}^\lambda,\\
\mathsf{L}^i_{\gamma,0} &:= \mathsf{H}(\mathsf{L}^i_{\alpha,0}, T_{\gamma,i,i}) \oplus \mathsf{H}(\mathsf{L}^i_{\beta,0}, T_{\gamma,i,i})\\
&\qquad \oplus (\textstyle\bigoplus_{j\neq i} \mathsf{K}_i[\lambda^j_{\alpha\beta}]) \oplus \lambda^i_{\alpha\beta} \cdot \Delta_i \oplus (\textstyle\bigoplus_{j\neq i} \mathsf{K}_i[\lambda^j_\gamma]) \oplus \lambda^i_\gamma \cdot \Delta_i \in \{0,1\}^\lambda,
\end{aligned}$$
   and for each $j \neq i, 1$, computes
$$\begin{aligned}
T_{\gamma,i,j} &:= \mathtt{sid}\|\mathtt{ssid}\|\gamma\|i\|j \in \{0,1\}^\lambda,\\
S^{i,j}_{\gamma,0} &:= \mathsf{H}(\mathsf{L}^i_{\alpha,0}, T_{\gamma,i,j}) \oplus \mathsf{H}(\mathsf{L}^i_{\alpha,1}, T_{\gamma,i,j}) \oplus \mathsf{M}_j[\lambda^i_\beta] \in \{0,1\}^\lambda,\\
S^{i,j}_{\gamma,1} &:= \mathsf{H}(\mathsf{L}^i_{\beta,0}, T_{\gamma,i,j}) \oplus \mathsf{H}(\mathsf{L}^i_{\beta,1}, T_{\gamma,i,j}) \oplus \mathsf{M}_j[\lambda^i_\alpha] \in \{0,1\}^\lambda,\\
S^{i,j}_{\gamma,2} &:= \mathsf{H}(\mathsf{L}^i_{\alpha,0}, T_{\gamma,i,j}) \oplus \mathsf{H}(\mathsf{L}^i_{\beta,0}, T_{\gamma,i,j}) \oplus \mathsf{M}_j[\lambda^i_\gamma] \oplus \mathsf{M}_j[\lambda^i_{\alpha\beta}] \in \{0,1\}^\lambda.
\end{aligned}$$
7. For each $i \in [2,n]$ and each wire $w \in \mathcal{W}$, P_i sends $((G^i_{w,0}, G^i_{w,1}), \{(S^{i,j}_{w,0}, S^{i,j}_{w,1}, S^{i,j}_{w,2})\}_{j\neq i,1})$ to P_1. In addition, for each $w \in \mathcal{W}$, P_2 sends $b_w := \mathsf{lsb}(\mathsf{L}^2_{w,0}) \in \{0,1\}$ to P_1.

Figure 18: The protocol of multi-party computation based on authenticated garbling (Part I).

[3] Carsten Baum, Alex J. Malozemoff, Marc B. Rosen, and Peter Scholl. Mac'n'cheese: Zero-knowledge proofs for boolean and arithmetic circuits with nested disjunctions. In Tal Malkin and Chris Peikert, editors, CRYPTO 2021, Part IV, volume 12828 of LNCS, pages 92–122,

Protocol Π_{MPC}

All parties use the preprocessed tuples to input to the protocol.

8. For each $i \in [n]$ and each $w \in \mathcal{I}_i$, all parties proceed as follows:
   (a) All parties invoke macro $\mathsf{OpenTo}(\mathtt{sid}, \mathtt{ssid}, \langle\langle \lambda_w \rangle\rangle, i)$ to output $\lambda_w \in \{0,1\}$ to P_i.
   (b) P_i computes $\Lambda_w := \lambda_w \oplus x_w^i \in \{0,1\}$ and sends $(\mathsf{BC}, \mathtt{sid}, \mathtt{ssid}, i, \Lambda_w)$ to $\mathcal{F}_{\mathsf{BC}}$, which outputs $(\mathsf{Msg}, \mathtt{sid}, \mathtt{ssid}, i, \Lambda_w)$ to all parties.
9. For each $i \in [2,n]$, P_i sends $\mathsf{L}^i_{w,\Lambda_w} := \mathsf{L}^i_{w,0} \oplus \Lambda_w \cdot \Delta_i \in \{0,1\}^\lambda$ to P_1.

Eval: P_1 evaluates the circuit in a topology order.

10. For each gate $(\alpha, \beta, \gamma, \mathsf{type})$, P_1 proceeds as follows:

- If $\mathsf{type} = \oplus$, compute $\Lambda_\gamma := \Lambda_\alpha \oplus \Lambda_\beta$ and for each $i \in [2,n]$, $\mathsf{L}^i_{\gamma,\Lambda_\gamma} := \mathsf{L}^i_{\alpha,\Lambda_\alpha} \oplus \mathsf{L}^i_{\beta,\Lambda_\beta}$.
- If $\mathsf{type} = \wedge$, compute for each $i, j \in [2,n]$ and $i \neq j$,

$$\begin{aligned} T_{\gamma,i,j} &:= \mathtt{sid} \| \mathtt{ssid} \| \gamma \| i \| j \in \{0,1\}^\lambda, \\ \mathsf{M}_j[r^i] &:= \mathsf{H}(\mathsf{L}^i_{\alpha,\Lambda_\alpha}, T_{\gamma,i,j}) \oplus \mathsf{H}(\mathsf{L}^i_{\beta,\Lambda_\beta}, T_{\gamma,i,j}) \oplus \Lambda_\alpha \cdot S^{i,j}_{\gamma,0} \oplus \Lambda_\beta \cdot S^{i,j}_{\gamma,1} \oplus S^{i,j}_{\gamma,2} \in \{0,1\}^\lambda, \end{aligned}$$

  and for each $i \in [2,n]$,

$$\begin{aligned} \mathsf{M}_i[r^1] &:= \Lambda_\alpha \cdot \mathsf{M}_i[\lambda^1_\beta] \oplus \Lambda_\beta \cdot \mathsf{M}_i[\lambda^1_\alpha] \oplus \mathsf{M}_i[\lambda^1_{\alpha\beta}] \oplus \mathsf{M}_i[\lambda^1_\gamma] \in \{0,1\}^\lambda, \\ T_{\gamma,i,i} &:= \mathtt{sid} \| \mathtt{ssid} \| \gamma \| i \| i \in \{0,1\}^\lambda, \\ \mathsf{L}^i_{\gamma,\Lambda_\gamma} &:= \mathsf{H}(\mathsf{L}^i_{\alpha,\Lambda_\alpha}, T_{\gamma,i,i}) \oplus \mathsf{H}(\mathsf{L}^i_{\beta,\Lambda_\beta}, T_{\gamma,i,i}) \\ &\qquad \oplus \Lambda_\alpha \cdot G^i_{\gamma,0} \oplus \Lambda_\beta (G^i_{\gamma,1} \oplus \mathsf{L}^i_{\alpha,\Lambda_\alpha}) \oplus (\textstyle\bigoplus_{j \neq i} \mathsf{M}_i[r^j]) \in \{0,1\}^\lambda, \end{aligned}$$

  and $\Lambda_\gamma := b_\gamma \oplus \mathsf{lsb}(\mathsf{L}^2_{\gamma,\Lambda_\gamma}) \in \{0,1\}$.

Figure 19: The protocol of multi-party computation based on authenticated garbling (Part II).

Virtual Event, August 16–20, 2021. Springer, Heidelberg, Germany.

[4] Mihir Bellare, Viet Tung Hoang, Sriram Keelveedhi, and Phillip Rogaway. Efficient garbling from a fixed-key blockcipher. In 2013 IEEE Symposium on Security and Privacy, pages 478–492, Berkeley, CA, USA, May 19–22, 2013. IEEE Computer Society Press.

[5] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, ACM CCS 2012, pages 784–796, Raleigh, NC, USA, October 16–18, 2012. ACM Press.

[6] Aner Ben-Efraim, Yehuda Lindell, and Eran Omri. Optimizing semi-honest secure multiparty computation for the internet. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, ACM CCS 2016, pages 578–590, Vienna, Austria, October 24–28, 2016. ACM Press.

[7] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In Kenneth G. Paterson, editor, EUROCRYPT 2011, volume 6632 of LNCS, pages 169–188, Tallinn, Estonia, May 15–19, 2011. Springer, Heidelberg, Germany.

Protocol Π_{MPC}

Output: All parties check the correctness of circuit outputs. Let $\mathcal{H} = \{\mathsf{H}' : (\{0,1\}^\lambda)^{|\mathcal{W}|} \to \{0,1\}^\lambda\}$ be a family of linear 1-universal hash functions.

11. P_1 samples $\mathsf{H}' \xleftarrow{\$} \mathcal{H}$.
12. For each $i \in [2,n]$, P_i checks the correctness as follows:
    (a) P_1 sends $(\mathsf{In}, \mathtt{sid}, \mathtt{ssid}, \{\mathsf{L}^i_{w,\Lambda_w}\}_{w\in\mathcal{W}})$ to $\mathcal{F}_{\mathsf{RO}}$, which outputs $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, h_i)$ to P_1, and sends $(\{\Lambda_w\}_{w\in\mathcal{W}}, h_i, \mathsf{H}')$ to P_i.
    (b) P_i sends $(\mathsf{In}, \mathtt{sid}, \mathtt{ssid}, \{\mathsf{L}^i_{w,0} \oplus \Lambda_w \cdot \Delta_i\}_{w\in\mathcal{W}})$ to $\mathcal{F}_{\mathsf{RO}}$, which outputs $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, h'_i)$ to P_i, and checks if $h_i = h'_i$. If this check fails, P_i aborts; otherwise, for each XOR gate $(\alpha, \beta, \gamma, \oplus)$ in a topology order, P_i computes $\Lambda_\gamma := \Lambda_\alpha \oplus \Lambda_\beta$.
13. P_1 checks the correctness by checking $t_\gamma = (\Lambda_\alpha \oplus \lambda_\alpha) \wedge (\Lambda_\beta \oplus \lambda_\beta) \oplus (\Lambda_\gamma \oplus \lambda_\gamma) = 0$ for each AND gate $(\alpha, \beta, \gamma, \wedge)$ in batch as follows:
    (a) For each AND gate $(\alpha, \beta, \gamma, \wedge)$ in parallel, P_1 computes

$$t^1_\gamma := (\Lambda_\alpha \cdot \Lambda_\beta \oplus \Lambda_\gamma) \oplus \Lambda_\alpha \cdot \lambda^1_\beta \oplus \Lambda_\beta \cdot \lambda^1_\alpha \oplus \lambda^1_{\alpha\beta} \oplus \lambda^1_\gamma \in \{0,1\},$$
$$\mathsf{M}_1[t^1_\gamma] := t^1_\gamma \cdot \Delta_1 \oplus \bigoplus_{i\in[2,n]} \left(\Lambda_\alpha \cdot \mathsf{K}_1[\lambda^i_\beta] \oplus \Lambda_\beta \cdot \mathsf{K}_1[\lambda^i_\alpha] \oplus \mathsf{K}_1[\lambda^i_{\alpha\beta}] \oplus \mathsf{K}_1[\lambda^i_\gamma]\right) \in \{0,1\}^\lambda,$$

    and for each $i \in [2,n]$, P_i computes

$$\mathsf{M}_1[t^i_\gamma] := \Lambda_\alpha \cdot \mathsf{M}_1[\lambda^i_\beta] \oplus \Lambda_\beta \cdot \mathsf{M}_1[\lambda^i_\alpha] \oplus \mathsf{M}_1[\lambda^i_{\alpha\beta}] \oplus \mathsf{M}_1[\lambda^i_\gamma] \in \{0,1\}^\lambda.$$

    (b) For each $i \in [2,n]$, P_i sends $z_i := \mathsf{H}'(\{\mathsf{M}_1[t^i_w]\}_{w\in\mathcal{W}}) \in \{0,1\}^\lambda$ to P_1.
    (c) P_1 computes $z_1 := \mathsf{H}'(\{\mathsf{M}_1[t^1_w]\}_{w\in\mathcal{W}}) \in \{0,1\}^\lambda$ and checks if $\bigoplus_{i\in[n]} z_i = 0$. If this check fails, P_1 aborts; otherwise, P_1 does nothing.

All parties decode their circuit outputs.

14. For each $i \in [n]$ and each $w \in \mathcal{O}_i$, all parties invoke macro $\mathsf{OpenTo}(\mathtt{sid}, \mathtt{ssid}, \langle\langle \lambda_w \rangle\rangle, i)$ to output $\lambda_w \in \{0,1\}$ to P_i, and P_i computes $y^i_w := \Lambda_w \oplus \lambda_w \in \{0,1\}$.
15. For each $i \in [n]$, P_i outputs $(\mathsf{Out}, \mathtt{sid}, \mathtt{ssid}, i, \boldsymbol{y}^i)$.

Figure 20: The protocol of multi-party computation based on authenticated garbling (Part III).

[8] Jürgen Bierbrauer, Thomas Johansson, Gregory Kabatianskii, and Ben Smeets. On families of hash functions via geometric codes and concatenation. In Douglas R. Stinson, editor, CRYPTO'93, volume 773 of LNCS, pages 331–342, Santa Barbara, CA, USA, August 22–26, 1994. Springer, Heidelberg, Germany.

[9] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure computation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, ACM CCS 2019, pages 291–308, London, UK, November 11–15, 2019. ACM Press.

[10] Elette Boyle, Niv Gilboa, and Yuval Ishai. Group-based secure computation: Optimizing rounds, communication, and computation. In Jean-Sébastien Coron and Jesper Buus Nielsen,

editors, EUROCRYPT 2017, Part II, volume 10211 of LNCS, pages 163–193, Paris, France, April 30 – May 4, 2017. Springer, Heidelberg, Germany.

[11] Ran Canetti. Security and composition of multiparty cryptographic protocols. Journal of Cryptology, 13(1):143–202, January 2000.

[12] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In 42nd FOCS, pages 136–145, Las Vegas, NV, USA, October 14–17, 2001. IEEE Computer Society Press.

[13] J Lawrence Carter and Mark N Wegman. Universal classes of hash functions. In Proceedings of the ninth annual ACM symposium on Theory of computing, pages 106–112, 1977.

[14] Ignacio Cascudo, Ivan Damgård, Bernardo David, Nico Döttling, and Jesper Buus Nielsen. Rate-1, linear time and additively homomorphic UC commitments. In Matthew Robshaw and Jonathan Katz, editors, CRYPTO 2016, Part III, volume 9816 of LNCS, pages 179–207, Santa Barbara, CA, USA, August 14–18, 2016. Springer, Heidelberg, Germany.

[15] Shan Chen, Rodolphe Lampe, Jooyoung Lee, Yannick Seurin, and John P. Steinberger. Minimizing the two-round Even-Mansour cipher. Journal of Cryptology, 31(4):1064–1119, October 2018.

[16] Shan Chen and John P. Steinberger. Tight security bounds for key-alternating ciphers. In Phong Q. Nguyen and Elisabeth Oswald, editors, EUROCRYPT 2014, volume 8441 of LNCS, pages 327–350, Copenhagen, Denmark, May 11–15, 2014. Springer, Heidelberg, Germany.

[17] Weikeng Chen and Raluca Ada Popa. Metal: A metadata-hiding file-sharing system. In NDSS 2020, San Diego, CA, USA, February 23–26, 2020. The Internet Society.

[18] Yu Long Chen and Stefano Tessaro. Better security-efficiency trade-offs in permutation-based two-party computation. In Mehdi Tibouchi and Huaxiong Wang, editors, ASIACRYPT 2021, Part II, volume 13091 of LNCS, pages 275–304, Singapore, December 6–10, 2021. Springer, Heidelberg, Germany.

[19] Seung Geol Choi, Jonathan Katz, Ranjit Kumaresan, and Hong-Sheng Zhou. On the security of the "free-XOR" technique. In Ronald Cramer, editor, TCC 2012, volume 7194 of LNCS, pages 39–53, Taormina, Sicily, Italy, March 19–21, 2012. Springer, Heidelberg, Germany.

[20] Benoit Cogliati and Yannick Seurin. On the provable security of the iterated Even-Mansour cipher against related-key and chosen-key attacks. In Elisabeth Oswald and Marc Fischlin, editors, EUROCRYPT 2015, Part I, volume 9056 of LNCS, pages 584–613, Sofia, Bulgaria, April 26–30, 2015. Springer, Heidelberg, Germany.

[21] Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. Merkle-Damgård revisited: How to construct a hash function. In Victor Shoup, editor, CRYPTO 2005, volume 3621 of LNCS, pages 430–448, Santa Barbara, CA, USA, August 14–18, 2005. Springer, Heidelberg, Germany.

[22] Jean-Sébastien Coron, Thomas Holenstein, Robin Künzler, Jacques Patarin, Yannick Seurin, and Stefano Tessaro. How to build an ideal cipher: The indifferentiability of the Feistel construction. Journal of Cryptology, 29(1):61–114, January 2016.

[23] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, CRYPTO 2012, volume 7417 of LNCS, pages 643–662, Santa Barbara, CA, USA, August 19–23, 2012. Springer, Heidelberg, Germany.

[24] Oded Goldreich. Foundations of Cryptography: Basic Applications, volume 2. Cambridge University Press, Cambridge, UK, 2004.

[25] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions (extended abstract). In 25th FOCS, pages 464–479, Singer Island, Florida, October 24–26, 1984. IEEE Computer Society Press.

[26] Shafi Goldwasser and Yehuda Lindell. Secure multi-party computation without agreement. Journal of Cryptology, 18(3):247–287, July 2005.

[27] Chun Guo, Jonathan Katz, Xiao Wang, Chenkai Weng, and Yu Yu. Better concrete security for half-gates garbling (in the multi-instance setting). In Daniele Micciancio and Thomas Ristenpart, editors, CRYPTO 2020, Part II, volume 12171 of LNCS, pages 793–822, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Heidelberg, Germany.

[28] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. Cryptology ePrint Archive, Report 2019/074, 2019. https://eprint.iacr.org/2019/074.

[29] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In 2020 IEEE Symposium on Security and Privacy, pages 825–841, San Francisco, CA, USA, May 18–21, 2020. IEEE Computer Society Press.

[30] Chun Guo, Xiao Wang, Kang Yang, and Yu Yu. On tweakable correlation robust hashing against key leakages. Cryptology ePrint Archive, Paper 2024/163, 2024.

[31] Carmit Hazay, Peter Scholl, and Eduardo Soria-Vazquez. Low cost constant round MPC combining BMR and oblivious transfer. In Tsuyoshi Takagi and Thomas Peyrin, editors, ASIACRYPT 2017, Part I, volume 10624 of LNCS, pages 598–628, Hong Kong, China, December 3–7, 2017. Springer, Heidelberg, Germany.

[32] Viet Tung Hoang and Stefano Tessaro. Key-alternating ciphers and key-length extension: Exact bounds and multi-user security. In Matthew Robshaw and Jonathan Katz, editors, CRYPTO 2016, Part I, volume 9814 of LNCS, pages 3–32, Santa Barbara, CA, USA, August 14–18, 2016. Springer, Heidelberg, Germany.

[33] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, CRYPTO 2003, volume 2729 of LNCS, pages 145–161, Santa Barbara, CA, USA, August 17–21, 2003. Springer, Heidelberg, Germany.

[34] Jonathan Katz, Samuel Ranellucci, Mike Rosulek, and Xiao Wang. Optimizing authenticated garbling for faster secure two-party computation. In Hovav Shacham and Alexandra Boldyreva, editors, CRYPTO 2018, Part III, volume 10993 of LNCS, pages 365–391, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Heidelberg, Germany.

[35] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson,

Anna Ingólfsdóttir, and Igor Walukiewicz, editors, ICALP 2008, Part II, volume 5126 of LNCS, pages 486–498, Reykjavik, Iceland, July 7–11, 2008. Springer, Heidelberg, Germany.

[36] Eyal Kushilevitz, Yehuda Lindell, and Tal Rabin. Information-theoretically secure protocols and security under composition. In Jon M. Kleinberg, editor, 38th ACM STOC, pages 109–118, Seattle, WA, USA, May 21–23, 2006. ACM Press.

[37] Enrique Larraia, Emmanuela Orsini, and Nigel P. Smart. Dishonest majority multi-party computation for binary circuits. In Juan A. Garay and Rosario Gennaro, editors, CRYPTO 2014, Part II, volume 8617 of LNCS, pages 495–512, Santa Barbara, CA, USA, August 17–21, 2014. Springer, Heidelberg, Germany.

[38] Jooyoung Lee, Martijn Stam, and John P. Steinberger. The security of Tandem-DM in the ideal cipher model. Journal of Cryptology, 30(2):495–518, April 2017.

[39] Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, and Sai Sheshank Burra. A new approach to practical active-secure two-party computation. In Reihaneh Safavi-Naini and Ran Canetti, editors, CRYPTO 2012, volume 7417 of LNCS, pages 681–700, Santa Barbara, CA, USA, August 19–23, 2012. Springer, Heidelberg, Germany.

[40] Jacques Patarin. The "coefficients H" technique (invited talk). In Roberto Maria Avanzi, Liam Keliher, and Francesco Sica, editors, SAC 2008, volume 5381 of LNCS, pages 328–345, Sackville, New Brunswick, Canada, August 14–15, 2009. Springer, Heidelberg, Germany.

[41] Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, CRYPTO 2022, Part I, volume 13507 of LNCS, pages 657–687, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Heidelberg, Germany.

[42] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient MultiParty computation toolkit. https://github.com/emp-toolkit, 2016.

[43] Xiao Wang, Samuel Ranellucci, and Jonathan Katz. Global-scale secure multiparty computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, ACM CCS 2017, pages 39–56, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.

[44] Kang Yang, Xiao Wang, and Jiang Zhang. More efficient MPC from improved triple generation and authenticated garbling. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, ACM CCS 2020, pages 1627–1646, Virtual Event, USA, November 9–13, 2020. ACM Press.

[45] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In 27th FOCS, pages 162–167, Toronto, Ontario, Canada, October 27–29, 1986. IEEE Computer Society Press.

[46] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole - reducing data transfer in garbled circuits using half gates. In Elisabeth Oswald and Marc Fischlin, editors, EUROCRYPT 2015, Part II, volume 9057 of LNCS, pages 220–250, Sofia, Bulgaria, April 26–30, 2015. Springer, Heidelberg, Germany.